University of Warsaw
Faculty of Mathematics, Informatics and Mechanics

Piotr Styczyński
Student no. 386038

An integrated and practical database solution for storage and efficient processing of large genomic datasets

Master's thesis
in COMPUTER SCIENCE

Warsaw, 2026-07-28

Abstract

Genomic workflows commonly connect specialized programs through files. This arrangement separates sequences from relational metadata, repeats parsing and indexing work, and prevents the database planner from representing the cost of alignment. This thesis presents BioSeqDB, a PostgreSQL extension that stores compact biological sequences and runs a minimap2-derived seeding, chaining, and extension pipeline during database scans.

BioSeqDB provides alphabet-generic packed sequence views, scalar and single instruction, multiple data (SIMD) minimizer extraction, and Vulkan implementations for selected sketching and chaining operations. Bounded CPU dynamic-programming kernels and GPU alignment routes perform final extension. Optional SneakySnake and GateKeeper filters reject unsuitable candidate regions before that stage. The database layer adds persistent minimizer indexes, bounded ingestion, external-provider relation definitions, coordinated caching, and generated PostgreSQL boundary declarations.

Alignment is not evaluated as an independent scalar function for every row. Planner support introduces custom paths whose execution nodes batch sequences, execute the explicitly selected implementation when the input and hardware pass admission, and place alignment results into planned expressions. The same operation can therefore participate in joins, filtering, metadata projection, and persistent storage. The thesis describes how the mapper's data and control flow become a database execution primitive while retaining explicit resource bounds and support for heterogeneous processors.

Keywords

sequence alignment, bioinformatics, minimap2, PostgreSQL, databases, genomic data management, SIMD, GPU computing

Thesis domain (Socrates-Erasmus subject area codes)

11.3 Informatics and Computer Science

Subject classification

Applied computing, life and medical sciences, bioinformatics

Tytuł pracy w języku polskim

Zintegrowane i praktyczne rozwiązanie bazodanowe do przechowywania i wydajnego przetwarzania dużych zbiorów danych genomicznych

Contents

Introduction . . . 7
Timeline . . . 10

1. Sequence representations and lazy evaluation . . . 11
1.1. Designing sequence abstraction . . . 11
1.2. Packed symbols and a common C++ interface . . . 15
1.3. Slices keep logical coordinates . . . 17
1.4. Composing sequences without joining their payloads . . . 19
1.5. Reading only what the algorithm requires . . . 20
1.5.1. Where a flat value is required . . . 22
1.6. Shared storage and bounded lifetimes . . . 24

2. PostgreSQL API and custom scans . . . 27
2.1. The engineering problem: a search returns hits . . . 27
2.1.1. Why not pass a query as text? . . . 28
2.1.2. Why not save the last result in a function? . . . 29
2.1.3. A set-returning function is a valid alternative . . . 29
2.2. From a Boolean marker to a custom scan . . . 30
2.2.1. Inspiration from PostgreSQL extensions . . . 30
2.2.2. Paths, plans, and live state . . . 31
2.2.3. What the hooks must establish . . . 31
2.3. One current hit supplies every result field . . . 32
2.3.1. Typed values in the expression tree . . . 33
2.3.2. Placement during planning is a separate problem . . . 33
2.3.3. Requested fields can require more alignment work . . . 34
2.4. Batch work inside a row-at-a-time executor . . . 34
2.4.1. Row stability during a search . . . 35
2.4.2. Filter placement and early stopping . . . 35
2.4.3. Initialization, rescans, and cleanup . . . 36
2.5. Joins give sequences distinct search roles . . . 36
2.5.1. Two searches of the same stored sequence . . . 38
2.5.2. Understanding the selected plan . . . 39
2.6. Sharing completed query results . . . 40
2.6.1. One producer, waiting readers . . . 40
2.6.2. Describe coverage, then compute the difference . . . 41
2.6.3. An overlapping range example . . . 42

3. Physical sequence storage in PostgreSQL . . . 45
3.1. Store sequences together with their metadata . . . 45
3.2. Read the needed columns and row groups . . . 46
3.2.1. What analytical benchmarks motivate . . . 47
3.2.2. Group rows into stripes and chunks . . . 48
3.2.3. Two independent ways to avoid reads . . . 48
3.2.4. A column chunk separates missing values from values . . . 49
3.3. Store byte streams in PostgreSQL relation pages . . . 50
3.3.1. Reserve space, write bytes, publish metadata . . . 50
3.3.2. Why use relations instead of private payload files? . . . 51
3.4. A locator addresses packed sequence fragments . . . 51
3.4.1. Symbols, packed units, fragments, and pages . . . 52
3.4.2. Length buckets separate short and long values . . . 52
3.5. Keep old readers stable when a row changes . . . 53
3.5.1. Deletion changes visibility before it changes storage . . . 54
3.5.2. Locks protect different parts of the operation . . . 55
3.5.3. Removing bytes is a separate maintenance operation . . . 55
3.6. Load fragments when a consumer needs them . . . 56
3.6.1. Decoded column chunks have a different cache . . . 57
3.7. Physical storage of shared query results . . . 57
3.8. From stable sources to persistent indexes . . . 57

4. Persistent sequence indexes . . . 59
4.1. Keep the lookup work between queries . . . 59
4.2. What existing sequence indexes store . . . 60
4.2.1. Minimap2: buckets, singletons, and position lists . . . 60
4.2.2. MMseqs2: words point back to target sequences . . . 61
4.3. The same minimizers in two orders . . . 62
4.3.1. MPF follows source intervals . . . 62
4.3.2. MPS follows hash ranges . . . 63
4.3.3. A smaller index answers symbol counts . . . 64
4.4. Build a large index with bounded work buffers . . . 64
4.5. Read counts before large position lists . . . 66
4.6. A saved index needs a source version . . . 67
4.6.1. Publish complete generations . . . 67
4.6.2. From stored index rows to a CPU reader file . . . 68
4.7. Choose reuse without changing the search meaning . . . 76
4.7.1. Keep useful bytes while their owners remain valid . . . 78
4.8. What the database integration adds . . . 79

5. BioSeqDB architecture . . . 81
5.1. Connecting sequence processing to the database . . . 81
5.2. Contracts describe the operations a stage needs . . . 82
5.3. Ownership completes the interface . . . 83
5.4. Runtime choices and compiled implementations . . . 84
5.4.1. Turn SQL settings into a request . . . 84
5.4.2. Separate algorithm selection from hardware selection . . . 85
5.4.3. Require compatible indexes and execution routes . . . 86
5.5. Schedule independent work within a backend . . . 86

5.5.1. PostgreSQL processes and oneTBB threads . . . 87
5.5.2. Choose the unit of parallel work . . . 88
5.5.3. Admit memory before increasing overlap . . . 89
5.5.4. Completion, cancellation, and progress . . . 90
5.6. Load sequence files through a reusable parser . . . 91
5.6.1. Separate byte sources from record framing . . . 92
5.6.2. Deliver chunks while a record remains open . . . 92
5.6.3. Route header fields into SQL columns . . . 92
5.7. Remote sequences and local persistence . . . 94
5.8. Generation and validation . . . 95
5.8.1. Deriving the PostgreSQL boundary . . . 95
5.8.2. Controlling specialization cost . . . 96
5.8.3. Benchmark specifications and validation . . . 97
5.9. Authorship, development and AI assistance . . . 98
5.9.1. Where AI tools were used . . . 98
5.10. Validation limits of the complete system . . . 99

6. Conclusions . . . 101

References . . . 103

Introduction

Sequencing instruments turn biological material into collections of strings. The useful interpretation of those strings rarely exists in isolation. A read has an instrument and sample provenance. A reference interval belongs to an assembly and annotation release. An alignment has a score, a strand, a position, an edit path, and a relationship to experimental metadata. Conventional workflows keep the strings in FASTA and FASTQ sequence files, Sequence Alignment/Map (SAM) and Binary Alignment/Map (BAM) alignment files, or mapper-specific index files while keeping much of the surrounding information in tables or ad-hoc scripts. Each tool sees only the portion of the workflow represented by its command-line arguments.

Relational database systems solve a different part of the problem. PostgreSQL provides durable storage, transactions, typed values, indexing, joins, extensible operators, and a cost-based planner. It does not, by itself, provide the representations or execution kernels required for billions of packed symbols and computationally expensive sequence searches. A naive user-defined function is not sufficient. Evaluating one alignment function independently for every row prevents batching, hides accelerator opportunities, and gives the planner little information about a predicate that can dominate the query.

BioSeqDB investigates the boundary between these two systems. It is implemented as a PostgreSQL extension whose non-database core is written in C++26. Biological symbols are represented by generic compact ranges rather than by a deoxyribonucleic acid (DNA)-only byte string. The alignment path is derived from minimap2: representative k-mers are extracted as minimizers, matching minimizers become anchors, compatible anchors are chained, and CPU or GPU alignment routes extend the selected regions. The port is not a line-for-line translation. The implementation introduces template-selected alphabets and layouts, Highway SIMD kernels, task-based parallel execution, Vulkan compute paths, persistent PostgreSQL-backed indexes, and prefilters for candidate regions. These changes must preserve the semantics that make minimap2 useful while operating on data that may be lazy, fragmented, cached, or obtained from an external provider.

The database boundary is equally important. An SQL expression such as `mmsearch(a, b)` has a conventional function definition so that PostgreSQL can type and plan it. The ordinary function body is not an alignment fallback: it fails closed if no BioSeqDB custom executor owns the expression. Its planner-support function assigns the unowned expression a high startup cost, while structural planner hooks can construct an owning custom path with explicit nonnegative costs. The resulting custom scan batches input, invokes the alignment pipeline, and makes its results available to planned expressions through injected PostgreSQL nodes. This is what permits alignment to participate in filters, projections, and joins while retaining the execution structure required by central processing unit (CPU) vector units and graphics processing units (GPUs).

The central research question is therefore not whether an existing aligner can be called from SQL. It is whether the aligner's data and control flow can become a database execution

primitive without losing algorithmic fidelity, resource bounds, or the ability to exploit modern processors. The investigation is organized around four supporting questions:

RQ1. Which parts of seeding, chaining, and extension can be expressed over a common compact sequence abstraction without eager conversion?

RQ2. Which stages admit SIMD or GPU implementations, and which input and resource constraints govern their selection?

RQ3. How can PostgreSQL plan and execute a set-valued, batch-oriented search while presenting an ordinary typed SQL interface?

RQ4. How should sequences, indexes, ingestion, external data, caches, and generated boundary code fit around the kernels so that the resulting system remains inspectable and reliable?

This thesis answers these questions through implementation and analysis of algorithmic and execution contracts. Historical designs appear only when they explain a constraint or a rejected optimization. They are not presented as simultaneous production architectures. Comparisons with minimap2, SIMD-Minimizers, the G3SA GPU seed-chain-extend system, Block Aligner, SneakySnake, GateKeeper, and Citus Columnar distinguish inherited mechanisms from BioSeqDB-specific changes.

The main contributions developed in the following chapters are:

C1. An alphabet-generic sequence representation that supports packed, fragmented, lazily composed, and PostgreSQL-backed data through a common range contract.

C2. A staged minimap2-derived mapper with scalar, SIMD, and Vulkan components and explicit runtime dispatch.

C3. Optional GPU-capable SneakySnake and GateKeeper candidate filters integrated without changing the meaning of an accepted alignment.

C4. Persistent sequence and minimizer-index storage derived from a columnar PostgreSQL access method, with bounded loading, caching, and selective cold-index reads.

C5. A planner and custom-scan interface that connects SQL expressions and joins to batch-oriented alignment.

C6. Structural generation and validation of SQL installation definitions and PostgreSQL function-manager boundary declarations from annotated C++ sources.

The document first establishes the biological and algorithmic context, then states the shortcomings that motivate an integrated system. The algorithm chapters follow data through minimizer extraction and chaining. They then describe the expensive downstream alignment stage before stepping back to the prefilter that guards that stage. This expository inversion makes the filter's safety and cost contract concrete without changing the runtime order, which remains chaining, prefiltering, and alignment. It then moves outward through sequence abstractions, PostgreSQL planning, physical storage, persistent indexes, ingestion, external providers, and generated SQL boundary code.

- Chapter 1 develops compact sequence ranges and bit-level views. It then presents fragmented and lazy sequences with PostgreSQL-backed adapters and alphabet dispatch.

- Chapter 2 follows an mmsearch expression through planning and batched execution. It explains joins and per-row result substitution.
- Chapter 3 explains the columnar-derived storage layout and fragment metadata. It also covers caching and request coordination.
- Chapter 4 describes index construction and storage with compatibility checks and selective cold loading. It also explains updates and differences from minimap2 indexes.
- Chapter 5 connects algorithm selection and oneTBB scheduling to a reusable processing core. It also covers sequence ingestion; remote access; code generation; validation and authorship.
- Chapter 6 answers the research questions and summarizes the final system. It identifies the principal limitations and possible extensions.

Timeline

This thesis required several years of exploration and experiments. I began without a formal background in computational biology, so learning the field was part of the research. I proposed the database idea and prepared the initial design for our internal team. I directed the 2022 prototype, whose implementation was written by colleagues.[1] The MMseqs2 discussions did not lead to a collaboration. The Python prototypes that led to the implementation studied here began in December 2023.



Selected milestones from my project history. Spacing is not proportional to elapsed time. Linked entries point to the related discussion and prototype repositories.

[1] The current BioSeqDB is a separate project, designed and written by me. It contains no code or intellectual property from the early BWA-MEM integration experiments.

[2] I was a co-founder (shareholder) until 15 July 2022 (KRS update on 3 Jun 2022) in Covid Genomics Sp. z o.o. (registered in Poland in Jun 2020, KRS 0000846289) which then changed its legal name to CGen Sp. z o.o. (on 3 Jun 2022) and then changed its branding name to Deepflare (on Jul 2022). I was also a Chief Engineering Officer from 1 Apr 2020 till 31 Aug 2022.

Chapter 1

Sequence representations and lazy evaluation

1.1. Designing sequence abstraction

Suppose user stores large genome sequence in one row, but only wants to align specific small part of it (let's say we know location of one protein-coding human gene and are interested only in finding occurences of that gene). It would be very wasteful if I didn't provide built-in functionality to load only that small region. This region could be stored on disk, so we only have it's locator poiting to a specific chunk in the table. On the other hand, we can load sequences from fasta file, where parser can be writing to RAM, so our sequence could refer to location in memory, where all sequences are pooled into one memory region to avoid constant allocation/reallocation. In yet another case, in tests we would like to use standard library containers to prepare and examine sequences (potentially even somebody could directly call some internal BioseqDB functions as standalone alignment library). This created a design challenge: same sequence can arrive as relationship locator, region in memory pool in RAM, can be provided as a whole as standrad container or as only a fragment (if user wants to operate only on specific slice or if it does not fit in RAM or RAM is limited, so we cannot fit entire sequence directly there), while these fragments should also be shareable, so parallel alignment to the same sequence doesn't have to load them multiple times.

There are several existing libraries for interfacing with genomic sequences. SeqAn[1] is a C++ library that provides modern interface for composing efficient sequence-analysis software from generic C++ components. Its library includes sequence representations, indexes and configurable alignment algorithms, with SIMD alignment, but it has it's own limitations. However SeqAn SIMD algorithms are mostly limited to auto-vectoriztion provided by the compiler (which doesn't have to be portable in most cases) and offers very narrow set of features for custom pipelines. I would be hard to directly reuse it for implementation of mentioned feautres. Using SeqAn would require deep integration, that's why I decided to address the design challenges by designing own C++ system of templates with different specializations driven by C++ concepts (inspired by nice-looking HPC MPI code Uhl, Schimek, and Brommer, Concepts in Practice: C++ MPI Bindings for the HPC Ecosystem. From a Standardizable Core to a Composable Interface).

[1] Reinert et al., "The SeqAn C++ template library for efficient sequence analysis: A resource for programmers"; SeqAn Project, SeqAn Manual.

SeqAn compatibility

Because the architecture of BioseqDB pipeline is very custom, I decided to only use SeqAn's alphabet layer[a] and utilities exposed by its public API. This means that libraries using SeqAn code as base, could be slightly adjusted to be able to work with BioseqDB pipelines.

[a] SeqAn Project, Alphabets in SeqAn.

Table 1.1 lists every specialization of all BioseqDB sequence representations, to showcase how they are used in different scenarios. SeqAn offers abstractions over lazy sequences as well SeqAn Project, SeqAn Tutorial: Sequences In-Depth, but they're not as sophisticates as in my case.

Table 1.1: Iterfaces for sequence representations in BioSeqDB.

Template	Specialization, and when it is selected	Description and usage context
`compact_sequence_range<R>` This is the core type representing sequence that resides completely in memory. Template allows only four available specializations depending on what `R` is and each of them have entirely separate handling logic.	`R` is a container of alphabet values, such as `std::vector<dna4>`	This is only adapter for existing sequence containers to provide compatibility with SeqAn and standard library containers. There is no alphabet packing and the type borrows `const R&` Adapter just provides common sequence interface. Adapting one costs $O(1)$ as we proxy all operations to the underlying container
	`R` is a packing configuration `sequence_custom_alphabet<A, U,b>`	Bitpacked vector that stores sequences according to alphabet rules passed as type paramter. Owns storage. Can convert to and from other bitpacked or not-bitpacked sequences, but if constructed from non-bitpacked sequence, then constructor costs $O(N)$
	`R` is an SeqAn alphabet type, as in `compact_sequence_range<dna4>`	The same as `sequence_custom_alphabet<A,U,b>`, but has sensible defaults for used underlying alphabet's storage unit width etc. Commonly used in tests. Code interfacing directly with BioseqDB C++ API should use this specialization.
	`R` is `compact_sequence_stdc_instance<...>`	Holds only reference to the underlying object. Adapter to convert `compact_sequence_stdc_instance<...>` to `compact_sequence_range<R>`. `compact_sequence_stdc_instance<...>` is a type that comes from `compact_sequence_stdc<...>` - described below Used internally

Template	Specialization, and when it is selected	Description and usage context
`compact_sequence_stdc<...>` Represents standard layout C type cppreference.com contributors, C++ named requirements: StandardLayoutType that is used to type-erase `compact_sequence_range<R>` types i.e allow user to have one type for storing possibly multiple different sequences types. Metadata like alphabet type and storage unit width are stored inside the struct and specialized template dispatch lambda is called to convert it back to the `compact_sequence_range<R>` type. This is heavily used by Postgres code to store sequences as C structures.		Owns everything and stores fixed header that records struct (self) size in bytes (compatible with Postgres varlena), alphabet code, unit width, padding, and symbol count Used heavy by Postgres code as we cannot store template instances directly and need to hold them in one object that converts type parameters of templates to runtime values (with correct runtime dispatch). If you want to store sequences of different alphabets in one single container, you should use this.
	`compact_sequence_stdc_instance<Stdc,A,W>`, internal type obtained when dispatching the type at runtime	Carries nothing, this is primarily runtime tag Shouldn't be used directly, one should receive `compact_sequence_range<compact_sequence_stdc_instance<...>>` when using dispatch methods of the stdc template
`lazy_sequence<R, Scanner>` Represents sequence that can be loaded in fragments by some storage provider represented by type parameter `Scanner`	`Scanner` has static constexpr method `is_fragmented()` that returns false (this means that Scanner always holds entire sequence in memory and there's only one fragment). This is used as compatibility layer so you can directly create lazy sequence from compact sequence range instance. I have `resident_sequence_scanner<U>` for trivial implementation of non-fragmented scanners so you can write `lazy_sequence<R,resident_sequence_scanner<R>>` and don't have to implement scanner type on your own.	Holds single non-owning pointer to `compact_sequence_range<R>` Used as adapter to convert compact sequence range to lazy sequence. All primary pipeline operations should accept it as parameter

Template	Specialization, and when it is selected	Description and usage context
	`Scanner` is fragmented (`is_fragmented()` returns true)	Ows scanner, internal small cache for sequence fragments that are neither copyable or movable Allows iteration over data source of sequences that doesn't fit into memory at once. This is primary specialisation for loading sequences from PostgreSQL relations. The Postgres side implements large scanner type that tells the lazy sequence template how to fetch blocks of sequences
`bit_casted_view<Base, Out>` Only legal for few types of alphabets and storage widths.	Special non-owning type that is used to provide zero-cost changes to pitpacking. It allows to cast bitpacked sequences to different storage widths i.e cast sequence encoded on 32-bit int to be read as sequence encoded on 64-bit or 16-bit or 8-bit units. There are special requirements for alphabet to be able to do that (discussed below).	Holds only reference to underlying compact sequence range This is adapter type that provides zero-cost conversion between different specialisations of compact sequence range. Used for example by SIMD algorithms to have specific vector lane widths
`lazy_sequence_view<A>` Type-erased, non-owning facade over every representation listed above. It is templated on the alphabet only, so exactly one type name crosses module and Postgres boundaries. The object itself is two pointers: opaque backend state and a static `backend_ops` table that holds `bp_size`, four orientation-specific `read_at` slots (read forward or backward, write forward or backward), `try_resident_packed`, `copy_packed_sequence` and `worker_shareable`. Specialization here means which callback table is installed.	`owned_sequence_adapter<R, Scanner>`, created by `make_lazy_sequence_view` for a concrete `lazy_sequence<R,Scanner>`	Complete table over an owning lazy sequence. This is the table behind every lazy sequence that pipeline code receives, where Postgres provides pointers to actual functions.
	`compact_range_adapter<R>`, created by `make_lazy_sequence_view` for `compact_sequence_range<R>`	Used as convenient adapter to convert compact sequence range to lazy sequence view without worrying about scanner type and ownership. Useful for testing.
	`facade_adapter<A>`, created by `make_pg_sequence_call_owner` on the Postgres side	This struct holds Postgres Scanner state - it is either a detoasted flat value or a piece tree root together with its read session, because sequences need to be represented as trees of slice operations

Template	Specialization, and when it is selected	Description and usage context
	`resident_adapter<A>`, created by `admitted_packed_sequence::view()`	Postgres Scanner for sequence only holds descriptor for already materialized sequence
`retained_lazy_sequence_view<A>` Has only one form. This type pairs view mentioned above with opaque lifetime pointer. It's used in Postgres code for management of sequence lifecycle during DB operations	Constructed from a view, an opaque lifetime pointer and a `noexcept` release callback, so there is nothing to specialize	Used wherever a view has to outlive the expression that produced it. The Postgres read session allocates the owner in its own memory context and registers it there, so the retain is dropped even when Postgres unwinds with `longjmp` and skips C++ destructors. This means that this type is specific version of `std::unique_ptr` that allows us to provide custom lifecycle management during scans.
`admitted_packed_sequence<A>` Supporting type to propagate sequence along with memory permit (memory permits are used in BioseqDB to manage large memory regions for sequences)	Created during materialization by `materialize_packed_sequence`. In BioseqDB there's a little sequence access planner that is responsible for deciding if we can materialize it all at once and directly or it needs to be streamed. `plan_sequence_access` picks this type when a contiguous resident region and we cannot use existing materialization (for example we access specific nucleotide that doesn't align well with storage unit boundary, but the GPU algorithm requires continous region of sequence, so we need to copy it)	Owns the permit, the aligned buffer and its descriptor together, so accounted bytes are released with the object. Its `view()` returns an ordinary view over `resident_adapter<A>`, which is how SIMD and GPU kernels obtain contiguous packed input from a sequence that is tree-backed or fragmented

1.2. Packed symbols and a common C++ interface

The template-driven design allows usage of any arbitrary alphabet for sequences, though for now I only support DNA4 and DNA5.

Sequence is encoded using bitpacking - the symbols are converted into integers values with specified fixed number of bits per symbol, then several symbols are packed into one single storage unit. Storage unit can be integer of any physical size. This is generalisation of bitpacking strategies commonly used in other software like packed-seq Groot Koerkamp and Martayan, packed-seq: Bitpacked sequence trait and implementation Rust library developed by author of SIMD minimizers paper. packed-seq Rust library offers non-alphabet-generic packing Groot Koerkamp and Martayan, packed_seq crate documentation. My design was more inspired by SeqAn approach where packing container is available, but very limited in features SeqAn Project, SeqAn Manual.

SeqAn alphabet assigns each symbol an integer called its rank. For DNA4, the four ranks are $A = 0$, $C = 1$, $G = 2$, and $T = 3$. A rank is an encoding of a symbol. Each `compact_sequence_range<R>` type needs to know certain properties of the encoding that are held

(using CRTP) in `compact_sequence_range<R>::traits` type. In my storage symbols never spill to other storage units i.e to retrieve some symbol you always need to access at most one storage unit. The symbols in the storage unit are always packed in "little-endian" order i.e the first symbol is in the lowest bits of the storage unit and the last symbol is in the highest bits. The properties in traits type are either types or constexpr static expressions/functions:

- `alphabet_type` - type of the alphabet
- `storage_unit` - type of the underlying storage unit (usually `uint8_t`, `uint16_t`, `uint32_t`, `uint64_t`)
- `alphabet_size` - number of symbols in the alphabet
- `unit_width` - width in bits of the storage unit: $\text{sizeof(storage_unit)} \times 8$
- `bits_per_char` - number of bits per symbol: $\lfloor \log_2(\text{alphabet_size}) \rfloor$
- `letters_per_unit` - number of symbols that can be packed into one storage unit: $\left\lfloor \frac{\text{unit_width}}{\text{bits_per_char}} \right\rfloor$
- `unit_used_bits` - number of bits used in the storage unit (for alphabet sizes that are not a power of two, some bits must stay unassigned): $\text{letters_per_unit} \times \text{bits_per_char}$
- `unit_leftover_bits` - number of bits leftover in the storage unit: $\text{unit_width} - \text{unit_used_bits}$
- `char_mask` - mask for the last symbol in the storage unit
- `rev_char_mask` - mask for the first symbol in the storage unit
- `used_storage_mask` - mask for the used bits in the storage unit
- Conversion functions for converting between storage unit and symbol: `letter_to_symbol`, `letter_to_char`, `letter_complement`
- Utilities for generating k-mer masks: `kmer_mask`

The figure below 1.1 shows an example of how I bitpack DNA4 sequence in storage units.

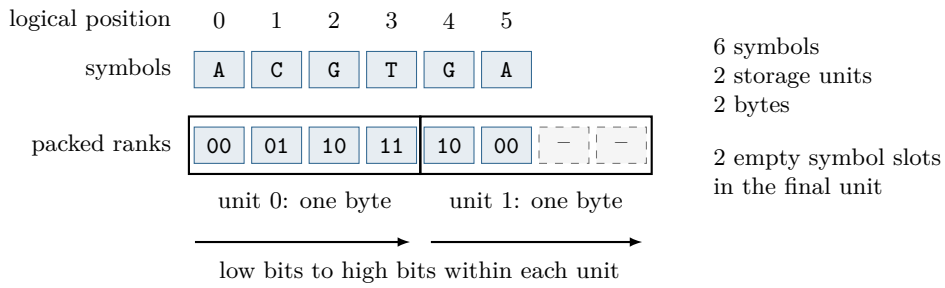


Figure 1.1: Packing `ACGTGA` with four DNA4 symbols per byte. The dashed slots are padding, which occurs at the end of the sequence as rarely sequence size is multiple of `compact_sequence_range<R>::traits::letters_per_unit` so they are not part of the sequence.

Template instance explosion

In BioseqDB I tried to allow user to specify any storage width that is power of two (8, 16, 32, 64) and alphabet (currently supported is DNA4 and DNA5). Because both of parameters are constexpr template parameters, this leads in some parts to creation of hundreds separate versions of pipeline. This slows downs compilation times and led to creation of absurdly large BMIs resulting in multi-gigabyte Clang .pcm files. Resulting binary still is optimised down to more convenient size, however I put lots of burden onto the compiler and linker. The goal of user-supplied storage widths was to observe how they affect performance and if they affect it at all. Currently in some parts of the pipeline I still cast the stored types to uint32_t, this happens because some algorithms like SIMD chaining require larger widths to store indices and scores. Extracting symbols directly from storage unit is a complex process involving lots of bitwise operations, so at this time most algorithms (SIMD/GPU) cannot operate directly on packed sequences as this would require lots of additional engineering effort. I added conversion helpers like one presented in the listing below 1.1 that convert requested ranges of sequence into non-bitpacked sequence that is easier to work with. This operation is done on demand and stored in separate memory pool region, so pipeline doesn't have to convert sequence from bitpacked to non-bitpacked format multiple times. Directly unpacking sequence inside pipeline at the latest point in time is potential opportunity for future improvements.

The following example uses actual code snippet from BioseqDB - in this case the caller owns `output`, which has at least $last - first$ bytes, and passes bounds satisfying $0 \le first \le last \le N$.

Listing 1.1: A consumer requests only the ranks it needs.

```
template<supports_compact_sequence_range R>
void read_fragment(const compact_sequence_range<R>& sequence,
                   std::size_t first, std::size_t last,
                   std::span<std::uint8_t> output) {
    // One rank per output byte, in forward order.
    sequence.template to_encoded_into<false>(output, first, last);
}
```

1.3. Slices keep logical coordinates

Postgres support array subscription operations natively, so I decided to support slicing as well in first version of BioseqDB.

A slice selects positions $[a, b)$: it includes a and excludes b. BioSeqDB uses zero-based positions, so $S[0]$ is the first symbol of the slice.

0-based indexing

This contradicts the fact that Postgres slices are 1-based, but it wasn't intentional decision. I had limited experience with Postgres array functionality and this realisation came only after most of the support was already implemented. What is more interesting that SeqKit - tool for operating on FASTA/Q files use 1-based indexing Shen, Sipos, and Zhao, SeqKit usage documentation: the subseq subcommand. This incompatibility might be something to reconsider in the future.

I provided utility functions for common operations like reversing and reverse-complementing slices as well, example available in listing 1.2.

Listing 1.2: Zero-based slices and sequence transforms in SQL.

```
WITH v(s) AS (
  VALUES ('ACGTAACCGTTA'::bioseq.seq('dna4', 32))      -- ACGTAACCGTTA
)
SELECT s[2:10]::text AS middle,                        -- GTAACCGT
       ((s[2:10])[1:5])::text AS nested,               -- TAAC
       s[-4:]::text AS suffix,                         -- GTTA
       bioseq.reverse(s[2:10])::text AS rev,           -- TGCCAATG
       bioseq.reverse_complement(s[2:10])::text AS rc -- ACGGTTAC
FROM v;
```

The bounds follow a small set of intuitive rules simiar to design of slices in Python Python Software Foundation, Python 3 Documentation: Common Sequence Operations:

- A missing lower bound means zero. A missing upper bound means the sequence length.
- A negative bound counts from the end (example: for length 12, -4 means 8) and bounds outside the sequence are clamped to its ends.
- An upper bound below the lower bound gives an empty slice, so `s[4:2]` has length zero.
- User can extract any element (symbol) from slice, which has separate SQL type `LETTER`. An out-of-range element, such as `s[12]` in this example, gives SQL `NULL`. An explicit null slice bound also gives `NULL` i.e $s[a:\text{NULL}]$ is equivalent to $s[\text{NULL}:a]$ and is also equivalent to $s[\text{NULL}:\text{NULL}]$ (all of them result in `NULL`)

The slicing can be implemented by old holding slice coordinates, without actually loading any sequences, by very simple coordinates arithmetic visible at figure below 1.2 For a slice view $V = S[a,b)$, view position j refers to $S[a+j]$. A second slice therefore combines its offsets directly:

$$V[c,d) = S[a+c, a+d).$$

The example's $S[2,10)[1,5)$ is $S[3,7)$, which reads `TAAC`. Reversal is represented by reversing start and end coordinates: For $W = \text{reverse}(S[a,b))$ we have:

$$W[j] = S[b-1-j], \qquad W[c,d) = \text{reverse}(S[b-d, b-c)).$$

Complementing operation leaves the coordinates unchanged, but alters binary flag representing if the slice is complemented or not.

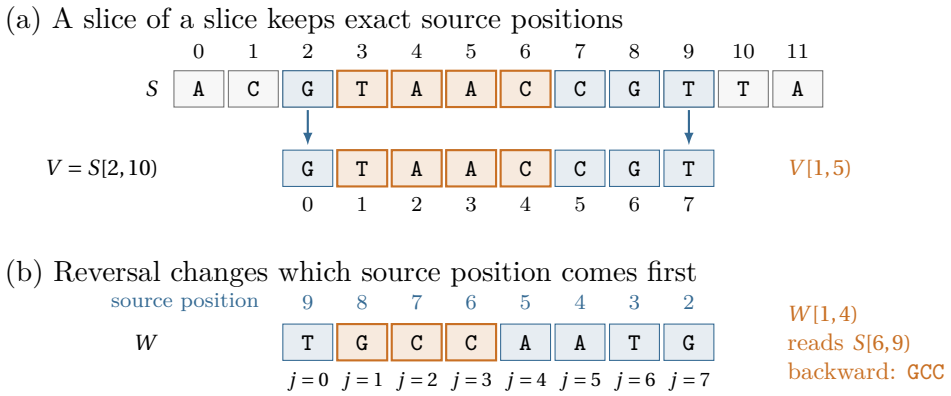


Figure 1.2: Blue cells form the first slice and orange cells show a second slice

1.4. Composing sequences without joining their payloads

The slice model I showed is very primitive. If we add ability to concatenate sequences we can represent any slice as a tree of pieces that represent operations on original sequence. In BioseqDB this tree refers to Postgres operators and datums.

The slice tree is constructed from the following elements:

- A source leaf describes interval in the source sequence.
- A literal leaf holds literal value referencing Postgres datum with additional metadata.
- A branch node that represents concatenation of two children. It holds total length, piece count (total number of leafs in subtree) and tree height. We collect total length to estimate how long the memory region in the pool should be to accomodate the materialized sequence.
- An edge that connects a node to its child. It holds reverse and complement flags, which affect the order of traversal and the interpretation of the leafs.

Figure 1.3 illustrates concatenation of three pieces including two source leaves and one literal leaf:

$$Z = S[2,6) \parallel \mathrm{reverse}(S[8,12)) \parallel \texttt{AC},$$

The result is `GTAAATTGAC`, with length $4+4+2=10$.

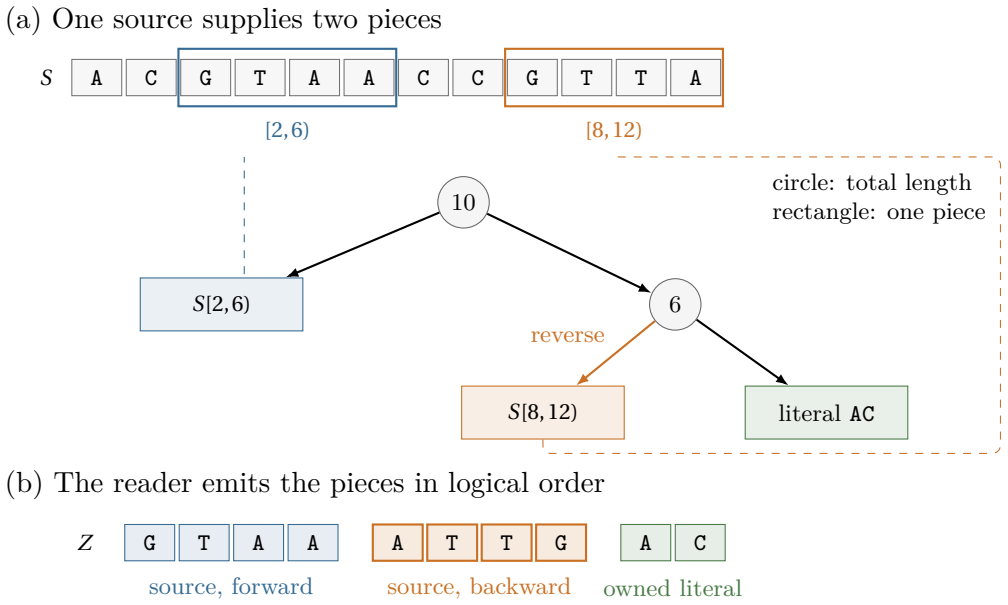


Figure 1.3: A piece tree describes how BioseqDB process slices of a sequence.

Each of the operations can easily be implemented by traversing the tree: concatenation joins roots and restores tree balance, slicing splits at the two bounds and leaves the middle part, reversing just as complementation flipps flags on the edges.

Reading actual slice content in BioseqDB is implemented using `lazy_sequence_view<A>` of table 1.1. It's built for the duration of a single call and the tree operations read only metadata from sequence locators (such as lengths). This means that we don't need to fetch large relations with sequence content and slicing is very performant. In the pipeline specific stages (like SIMD or GPU kernels) can request contiguous region of sequence and BioseqDB will materialize it into its own packed buffer first.

I implemented concatenation support for SQL text literals as well. This means that we can concatenate sequences with text directly without casting them, but this has it's own limitations:

- Slicing and reversal can operate on text without no known alphabet because they need only positions and order.
- Joining a known-alphabet sequence with a text literal validates the literal against that alphabet, so we require to go through entire literal (which shouldn't be a problem as we don't expect very long literals as this should be done via loading FASTA first into the table)
- Reverse-complement requires known alphabet with a complement rule for every symbol in that alphabet.
- Two different alphabets are not converted automatically to one another and this will result in an error.

1.5. Reading only what the algorithm requires

The storage can be provided in different forms as explained previously in the first section 1.1 In BioseqDB main implementation of Postgres-based storage is done via `lazy_sequence_`

`view<A>` and `retained_lazy_sequence_view<A>` mentioned in table 1.1. These types holds pointers to functions that implement the storage and represent the following operations:

- Report sizes of sequences
- Copy selected intervals into caller-owned memory, which in this case can be GPU-owned memory in GPU-accelerated pipeline stages
- Reading successive chunks
- Checking if the compatible interval was already materialized and can be borrowed (caching)

Internally I distinguish several access policies which are specified by the pipeline stage, they are the following choices:

1. Stream. If the algorithm accepts a stream, read the selected interval in bounded chunks and save it to reusable scratch memory (large memory pool)
2. Borrow. If the algorithm needs packed data, first check if's already in cache and safe to borrow.
3. Pack. In case if borrowing is not possible, algorithm needs to request new memory permit (this is routed through custom pool allocator). Caller is then responsible for this memory chunk.

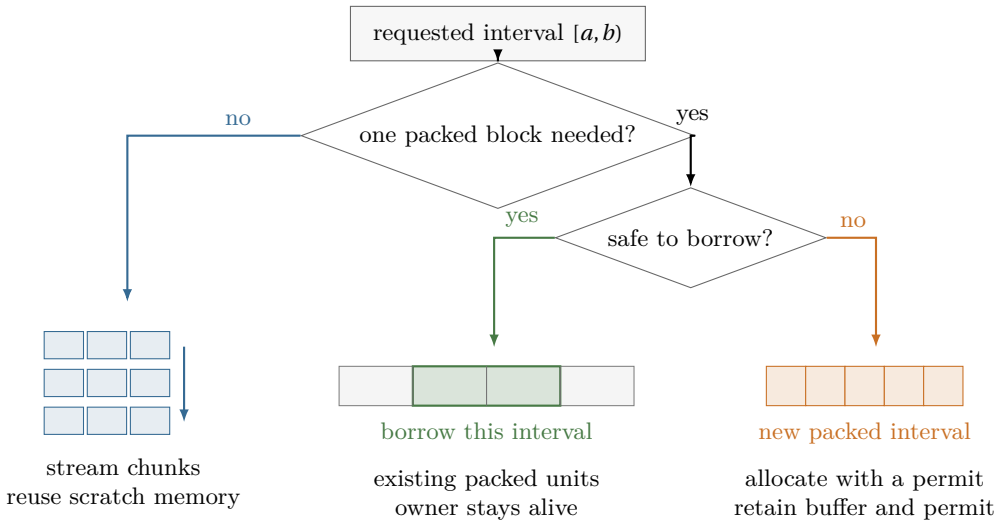


Figure 1.4: Simplified view of different access policies

Borrowing

Borrowing can fail if the region for example starts in the middle of storage unit in packed sequence, then we cannot reuse that memory chunk as it's misaligned

The view itself does not own the sequence. It is two pointers: opaque `state`, and a table of functions named `ops`. On the PostgreSQL path those functions walk a facade that lives in the read session: a piece tree plus one scanner per source row. A `retained_lazy_sequence_view`

is the same two pointers plus a release callback, so PostgreSQL can drop it even when an error skips C++ destructors.

Figure 1.5 shows how this looks when a custom scan processes a batch of table rows.

A scanner is one reader for one row's sequence, not one reader for the whole batch. It appears on the first request for symbols of that row. Its 16 slots hold copies of stored fragments. If the same row is read again in the same batch, those slots are reused. Workers never share a scanner: they only see a packed copy that the backend thread prepared first, or the backend walks the row itself.

The worker threads come from a process-wide pool. They stay alive between batches. What dies at the end of a batch is the read session: first the retained views, then the scanners, then the session's block of memory. The open table readers and the registered snapshot belong to the source and outlive the batch, so the next batch starts with empty fragment slots but does not reopen the table.

The following pseudocode states the logical read rule. A piece may refer to memory or to a stored fragment. Locating it and fetching its storage are handled by the storage reader.

Algorithm 1 Read one interval of a composed sequence

Require: view V, valid bounds $0 \le a \le b \le |V|$

Require: output space for $b - a$ ranks

1: for all pieces intersecting $V[a, b)$, in logical order do
2: | Map the intersection to that piece's source positions
3: | Read those positions, using bounded scratch space if needed
4: | Apply the piece's reverse and complement flags
5: | Append the resulting ranks to the output
6: Require exactly $b - a$ output ranks

An interval crossing two fragments must still produce the same symbols as an equivalent flat value. A storage reader may fetch complete fragments, but only the requested ranks enter the logical output. Thus the interface limits what the algorithm asks for without claiming that every source supports equally small physical reads.

1.5.1. Where a flat value is required

BioSeqDB's flat PostgreSQL value consists of a fixed header followed by aligned packed data. The header records the alphabet, unit width, length, and padding. The adapter validates the header and payload before selecting a typed compact range. Runtime checks happen at this boundary, so the inner loop can use compile-time packing rules.

An already resident, uncompressed, aligned value can be borrowed through this adapter. PostgreSQL may first have to fetch out-of-line data or decompress it using TOAST.[3] That work precedes borrowing and can include a copy.

Writing a lazy expression into durable storage is another boundary. Its temporary source references cannot be stored as raw pointers. The expression must be flattened to a value that can stand on its own. Text output similarly has to create the requested text. If the result exceeds the flat PostgreSQL value's size limit, the operation reports an error.

For repeated full reads, one packed copy can be a useful choice. Laziness preserves that option while also supporting the small-interval case. It is the consumer's access requirement, not the mere existence of a slice, that determines when to build the copy.

[3] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: TOAST.

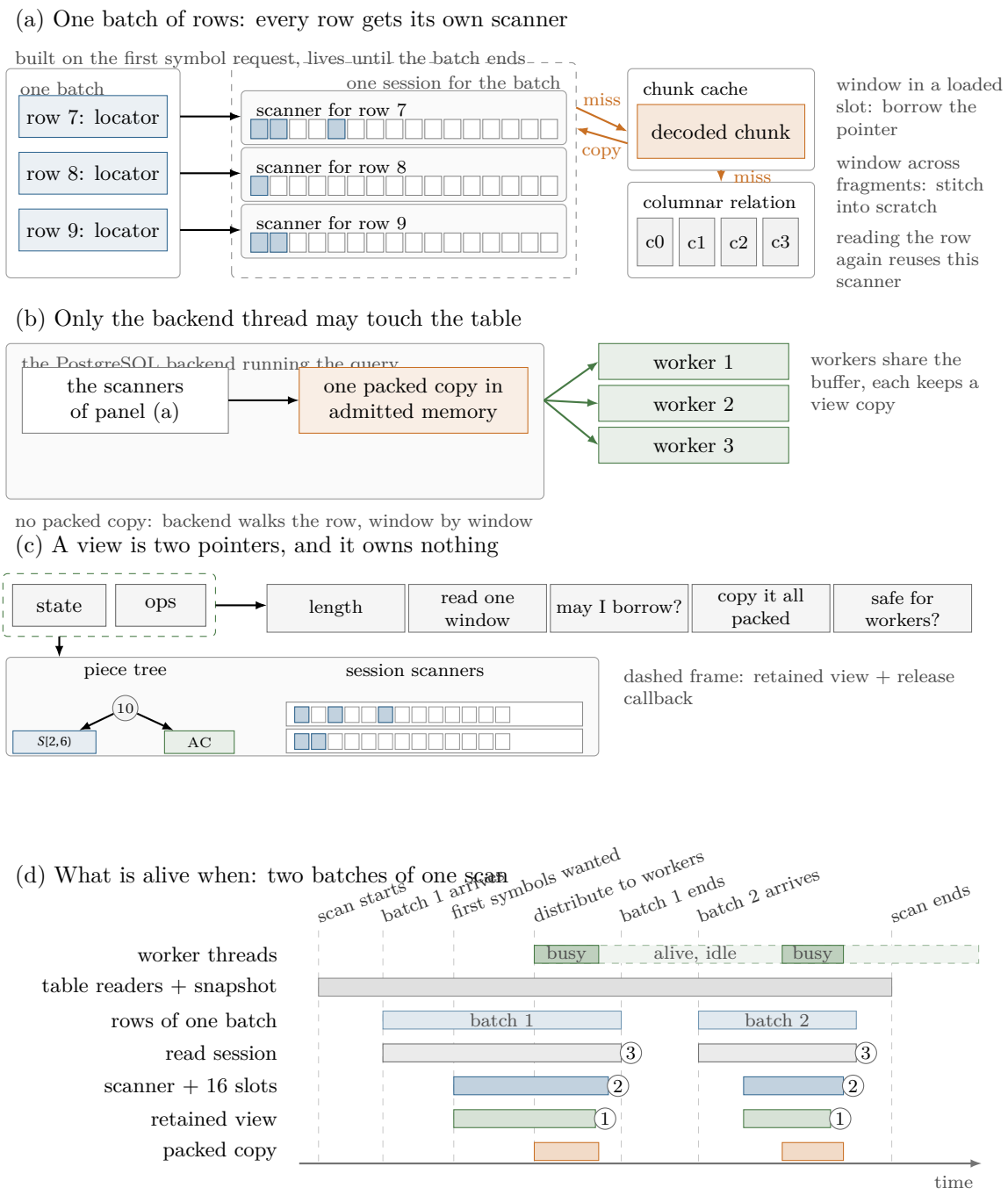


Figure 1.5: How a scan reads sequences. (a) Each row of a batch gets its own scanner with 16 fragment slots. A miss looks in the decoded chunk cache, then in the columnar relation, and copies one fragment into a slot. (b) Workers never call a scanner. The backend first builds one packed copy in admitted memory and distributes that copy. If there is no packed copy, the backend walks the row itself. (c) A view is two pointers. On the PostgreSQL path, `state` is a facade: a piece tree together with the session that holds one scanner per source row. A retained view is the same pair plus a release callback. (d) At every batch end, in this order: (1) release retained views, (2) finish each scanner so its slot copies go, (3) drop the session's memory. The worker pool and the snapshot outlive the batch. A failing query skips C++ destructors, so steps 1–3 also run from a memory context callback.

1.6. Shared storage and bounded lifetimes

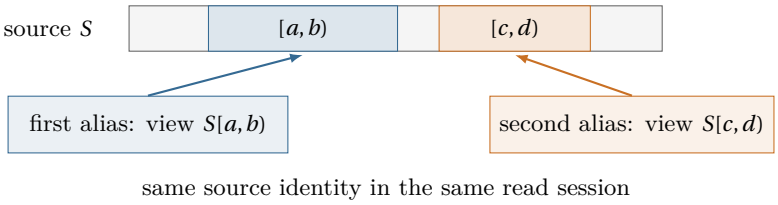
The distinction between a view and its source is useful in self-joins. A query can use two aliases of the same row to associate two different searches with the same sequence. The next chapter shows how aliases distinguish two searches. At the sequence layer, the two aliases can still refer to the same backing data.

Consider a row with a long sequence S. One consumer needs $S[a, b)$ and another needs $S[c, d)$. Creating two eager copies could load the complete sequence twice. Two lazy views instead retain the bounds and identify the source. A read session is the period during which source handles and the state needed to read them remain valid. Within one read session, BioSeqDB can share backing state or a scanner when the source identities match exactly.

This is a conditional form of reuse.

- Equal text in different rows is not enough to establish shared backing. The identity of the stored source must match.
- Sharing a scanner does not guarantee that overlapping intervals are physically read only once. That depends on the reader and its cache.
- Existing eager column caching can also share resident data. Laziness additionally permits both consumers to avoid requesting the full sequence in the first place.

(a) Two consumers can refer to one source



(b) Every read must finish while its owner is alive

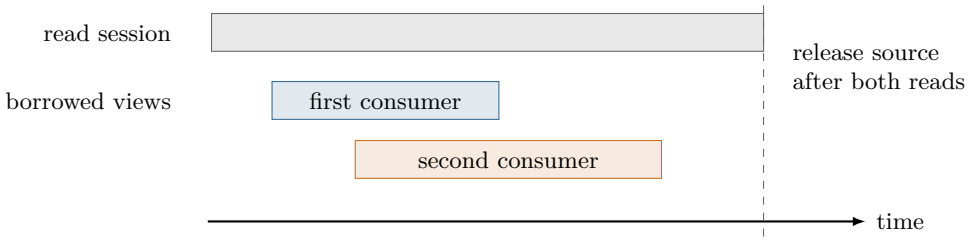


Figure 1.6: Aliases do not require duplicate sequence payloads. Sharing requires matching source identities and compatible ownership. The lower panel shows two read lifetimes fully inside the lifetime of their read session.

The non-owning lazy view cannot keep the source alive by itself. A call owner or PostgreSQL read session holds the source handles, retained values, tree memory, and temporary allocations. A consumer must finish before that owner is released. Returning an iterator or saving a view that outlives this owner would violate the rule even if no symbols had yet been read.

PostgreSQL error handling makes this an explicit engineering requirement. A database error can transfer control without unwinding C++ stack frames. Ordinary C++ destructors

alone are therefore insufficient for releasing resources across that boundary.[4] The read session registers cleanup actions that also run on the database's error path. Tree owners retain the sources and older tree memory they depend on, and release those dependencies through the same ownership scheme.

Asynchronous work needs a longer-lived owner. Before a worker or accelerator can outlive the current call, it must have stable input ownership. A packed buffer and its memory permit remain live until the task completes or the GPU signals completion. A flag saying that a source can be read by workers does not itself extend the source's lifetime.

These rules let sequence algorithms request a precise interval without also having to implement PostgreSQL storage and cleanup. They supply ranks in the required order, keep the source alive, and allocate a complete interval only when the consumer requires one. The next chapter uses this sequence interface inside a scan that can produce several alignment results from one stored row.

[4] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Using C++ for Extensibility.

Chapter 2

PostgreSQL API and custom scans

2.1. The engineering problem: a search returns hits

The sequence interfaces in the previous chapter let an alignment algorithm read a stored sequence without first copying all its bases. We now need to connect that algorithm to SQL. The main difficulty is the result: one pair of sequences can produce several alignments, and each alignment has several fields. We call one returned alignment a hit.

Consider a table `targets` with an identifier and a sequence. The SQL examples in this chapter assume BioSeqDB's table access method and a DNA5 sequence column packed into 64-bit storage units:

Listing 2.1: Search once and return the fields of each hit.

```
CREATE TABLE targets (
  id bigint PRIMARY KEY,
  seq bioseq.seq('dna5', 64)
) USING bioseq;

SELECT t.id,
       bioseq.dp_score(t.seq) AS score,
       bioseq.target_start(t.seq) AS start,
       bioseq.cigar(t.seq) AS cigar
FROM targets AS t
WHERE t.id > 20
  AND mmsearch(t.seq, $1::bioseq.seq('dna5', 64));
```

The first `mmsearch` argument is the target sequence and the second is the query. Here `$1` supplies that query sequence. The `score` alias means the dynamic-programming score of the current hit. The CIGAR string describes which positions align and where insertions and deletions occur.[1] Three requirements follow from this example:

- Compute the search once. Score, start, and CIGAR must read fields of the same hit. Calling the aligner separately for each field would repeat its most expensive work.
- Keep ordinary SQL conditions useful. The identifier test should remove unwanted targets before alignment. PostgreSQL should still see the tables, their types, and the surrounding joins.

[1] SAM/BAM Format Specification Working Group, Sequence Alignment/Map Format Specification.

- Return every selected hit as a row. A Boolean filter alone can keep or remove an input row. It cannot turn that row into two rows.

Figure 2.1 uses abstract symbols to show how one input row can produce two occurrences.

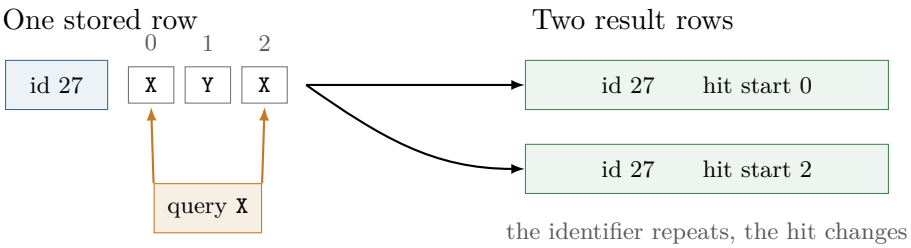


Figure 2.1: Searching `XYX` for `X` gives two occurrences. The output must preserve both even though they come from the same stored row.

For N_{in} input rows, if row i produces m_i hits, the number of output rows is

$$N_{\text{out}} = \sum_{i=1}^{N_{\text{in}}} m_i, \qquad m_i \geq 0.$$

The repeated identifier is intentional. It does not mean that the table contains duplicate records.

2.1.1. Why not pass a query as text?

An early interface idea was to pass a table name and an SQL string to a search function. The following is a design sketch, not a BioSeqDB API:

Listing 2.2: An early interface sketch passes SQL as text.

```
SELECT * FROM search_sql(
  'targets',
  'SELECT id, seq FROM reads WHERE id > 20'
);
```

This hides choices that ordinary SQL makes explicit:

- Which output is the query sequence? A function could demand a column called `query`, assume a fixed column position, or accept a separate column parameter. Renaming or reordering the inner output then becomes part of an extra API contract.
- Where do surrounding conditions apply? In a statement such as the sketch below, the outer planner cannot inspect a string as a relation. It cannot automatically insert the target identifier test into the search's input or choose a join order across the function boundary.
- Who connects the two plans? PostgreSQL can parse and optimize the inner SQL when the function submits it. The extension still has to interpret its output and arrange any rewrite that combines inner and outer conditions. Passing SQL text does not provide that integration.

Listing 2.3: An outer condition cannot automatically filter a hidden scan.

```
-- Hypothetical SQL-string interface
SELECT h.*
FROM search_sql('targets', 'SELECT id, seq FROM reads') AS h
WHERE h.target_id = 27;
-- The function must itself restrict the hidden target scan.
```

The function sees the target table through the string `'targets'`. The outer condition cannot become a restriction on that hidden table scan unless the interface explicitly passes it in or the extension adds a rewrite. In the typed query above, PostgreSQL sees `t.id > 20` and the target relation together.

2.1.2. Why not save the last result in a function?

Another idea is for a Boolean search function to save its result for later score and CIGAR calls. This gives the appearance of sharing work, but the SQL expression does not describe the dependency. PostgreSQL does not guarantee left-to-right evaluation of Boolean conditions. It may rearrange or skip parts of an expression.[2]

Listing 2.4: Hidden state does not establish an evaluation order.

```
-- Hypothetical functions with shared hidden state
SELECT id, last_score(seq)
FROM targets
WHERE save_search(seq, $1)
  AND last_score(seq) > 100;
```

The second condition cannot rely on the first having run. Declaring these functions `VOLATILE` limits some planner transformations, but does not establish a safe producer–consumer order between them. It also gives up useful assumptions about repeatable calls.[3] Nested searches, a rescan for a new outer row, and early query termination would all need explicit state management. A session variable or temporary table merely moves that responsibility elsewhere. The Boolean call still cannot emit multiple hit rows.

2.1.3. A set-returning function is a valid alternative

SQL already has a direct way to express one-to-many output. A set-returning function in a `LATERAL` join could return a record for every hit.[4] The `LATERAL` clause lets that call refer to the current target row.[5] This alternative solves both the row-count problem and the dependency between result fields:

Listing 2.5: A different API could return explicit hit records.

```
-- Alternative API sketch, not an existing BioSeqDB function
SELECT t.id, h.score, h.start, h.cigar
FROM targets AS t
CROSS JOIN LATERAL align_hits(t.seq, $1) AS h
WHERE t.id > 20;
```

[2] PostgreSQL 17, Expression Evaluation Rules.
[3] PostgreSQL 17, Function Volatility Categories.
[4] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Set Returning Functions.
[5] PostgreSQL 17, `LATERAL` Subqueries.

The reason to develop custom scans is therefore an engineering choice. We wanted the search to appear in ordinary `WHERE` and `JOIN ON` clauses, while one executor node controls batches, hit fields, and sequence lifetimes. A set-returning API would expose the hit record directly and would need its own design for batching several input rows.

2.2. From a Boolean marker to a custom scan

PostgreSQL lets an extension add a custom scan: an executor node with extension-defined methods for producing rows.[6] BioSeqDB uses this facility to make a search part of the plan tree. The node responsible for one search is its owner. It receives the input sequences, runs alignment, and produces the hit rows.

`mmsearch` is declared as an ordinary SQL function returning `boolean`. PostgreSQL can therefore resolve its arguments and accept it in a condition without a new SQL grammar. Its two-argument form uses the session's alignment settings. A third argument can supply typed options. The extra hit rows are part of BioSeqDB's search contract, which every accepted plan must preserve. During planning the call acts as a marker: it identifies the search that an owning custom scan must perform. When the plan is constructed, the owner removes that marker from the conditions that PostgreSQL would otherwise execute. An unowned non-null call reaches a function body that reports an unsupported query shape. There is no ordinary scalar alignment fallback.

2.2.1. Inspiration from PostgreSQL extensions

ParadeDB was an important motivation for this design. Its search interface keeps a search condition and a score expression in ordinary SQL, while its custom executor supplies the score of the current search result. This showed how an extension could connect a compact SQL expression to a richer execution method. BioSeqDB applies the same general idea to alignment fields and adds the requirement that one source row may produce several hits. The source references below use the public main branch; the inspected source snapshot identifies itself as version 0.25.6.

ParadeDB score expression
score_from_relation 30

In particular, ParadeDB replaces score expressions with typed placeholders, then fills them from the current search result before projection.

ParadeDB placeholder replacement
inject_placeholders 493

This is the concrete mechanism that informed the shared-field design described in Section 2.3.

The pgvector extension supplied another useful example of a concise typed operator. Its `<->` operator returns the Euclidean distance between two vectors, so SQL can order rows by that distance:

pgvector distance operator
CREATE OPERATOR <-> 174

Listing 2.6: A scalar distance with pgvector.

```
-- pgvector example: one distance for each stored vector
SELECT id, embedding <-> '[1,2,3]' AS distance
FROM items
ORDER BY embedding <-> '[1,2,3]'
LIMIT 5;
```

[6] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Writing a Custom Scan Provider.

This is an inspiration for the syntax, not an equivalent result model. One distance is a scalar value. Alignment must also preserve the identity and fields of each of several possible hits from one sequence pair.

2.2.2. Paths, plans, and live state

Three PostgreSQL objects have different jobs:

1. A path is a candidate execution strategy. It describes possible inputs, estimated work, output row count, and ordering. The planner may compare several paths before selecting one.

2. A plan records the chosen strategy. A `CustomScan` contains child plans, expression trees, and copyable descriptions of the search. It can be kept for later execution in a prepared statement.

3. Live state belongs to one execution. A `CustomScanState` holds the input cursor, retained rows, alignment buffers, and current hit. These cannot be saved as process-local pointers in a reusable plan.

The output estimate must describe hit rows, because later joins and sorts receive those rows rather than one row per stored sequence. It remains an estimate: the actual number of hits is known only after alignment.

PostgreSQL calls `PlanCustomPath` when it converts a selected custom path into a plan. It later calls `CreateCustomScanState` to create the executor state. These are separate extension callbacks.[7]

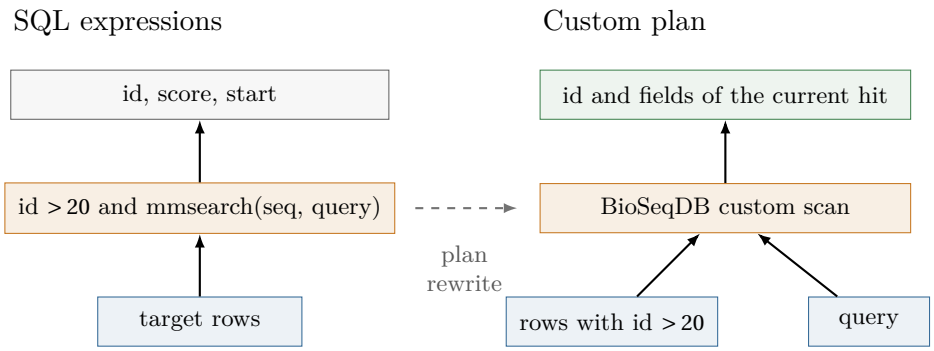


Figure 2.2: A conceptual expression tree and the work assigned by the custom plan. The search becomes the operation that produces hit rows. Ordinary identifier filtering remains before alignment. The boxes describe logical roles, rather than a literal `EXPLAIN` output.

2.2.3. What the hooks must establish

A hook is a function that PostgreSQL calls at a defined planning stage. BioSeqDB installs relation and join path hooks. A relation hook sees candidate access methods for a table. A join hook sees candidate ways to combine relations. Their task is more than finding the name `mmsearch`:

[7] PostgreSQL 17, Creating Custom Scan Paths and Creating Custom Scan Plans.

- Recognize the operation. Check the function identity, the argument types, the supported expression shape, and the search options.
- Find its inputs. Associate each operand with its table alias and column, or with a supported expression derived from them. For the target operand, this association and its query scope form the target binding used by result helpers.
- Propose an owner. Add a candidate base-scan or supported join path whose inputs can supply the search arguments. Selecting a path fixes the owner in the resulting plan; the hook does not run alignment.
- Remove ordinary execution sites. Remove the owned marker from row, vectorized, and storage filter lists. A leftover copy would call the scalar failure body after the custom scan had already done the search.

A planner support function also reports a high cost for scalar `mmsearch` evaluation.[8] Cost encourages a suitable plan, but does not enforce the search's output rules. The structural checks and the unowned-call error provide that boundary. Unsupported join, security, or expression shapes do not become correct merely because a path has a low estimated cost.

2.3. One current hit supplies every result field

Assigning the search to a custom scan solves the missing execution order. The owner selects a hit before PostgreSQL evaluates the expressions that produce its output row. Those expressions read fields already computed by the aligner. They do not call it again.

The argument in `bioseq.dp_score(t.seq)` is used to identify the search target. The score is not stored in `t.seq`. This distinction also explains why the same expression must not silently read a hit belonging to another search. For a supported transformed target, such as a slice, the corresponding helper can identify that same transformed expression.

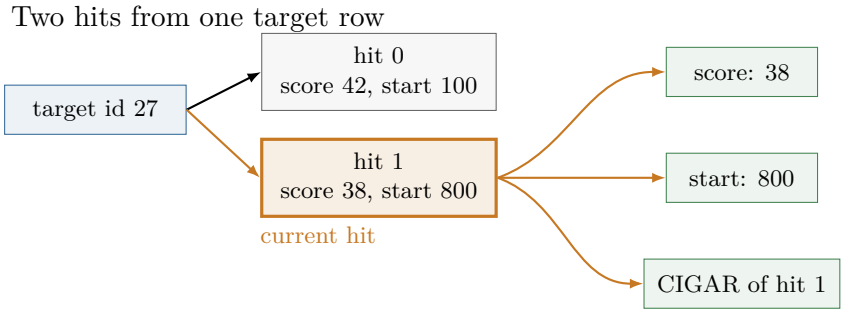


Figure 2.3: The current hit is the common source of score, position, and CIGAR. The numbers are illustrative. Changing the current hit changes all projected fields together while the target identifier stays the same.

[8] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Function Optimization Information.

2.3.1. Typed values in the expression tree

PostgreSQL evaluates output expressions through an expression tree. Before returning its first hit, the executor walks this tree and replaces the helpers belonging to the owner with typed `Const` nodes. It retains their addresses. Before projecting a hit, it writes the hit's values and null flags into these nodes and rebuilds the output-expression evaluator so that it reads the changed fields.

- Identity answers which search. The argument's target binding links a helper to the right owner. A function name alone cannot distinguish two searches in one statement.
- Type answers how to read the field. Coordinates use `BIGINT`, the DP score uses `INT4`, Boolean properties use `BOOLEAN`, and CIGAR uses `TEXT`.
- The current hit answers which value. Every field is filled from the same hit record before the output row is returned.

Although PostgreSQL calls the node a `Const`, this use is local to the executor. The value changes between output rows. The replacement happens after planning, so the planner does not fold a current hit's score into a reusable plan. No session-wide "last hit" variable is needed. ParadeDB's base scan uses the same kind of changing score node and rebuilds the projection after updating it.

ParadeDB current-result score
exec_custom_scan 1784

Algorithm 2 Project one hit without repeating alignment

1. Restore the ordinary input columns for this hit.
2. For every requested helper, write its field from this hit into the corresponding typed value node.
3. Rebuild the evaluator, then evaluate the output expressions.
4. Return the resulting SQL row. Keep the next hit for the next request.

The ordinary helper bodies are not a uniform substitute for this mechanism. Some helpers report an invalid use, while some generic numeric helpers return a special fallback value when called outside a supported owner. Such values are not computed alignment results.

2.3.2. Placement during planning is a separate problem

Joins can move expressions between plan levels. An outer join can also add a row of nulls for a missing input. PostgreSQL's `PlaceHolderVar` records where an expression must be evaluated and which outer joins can make its result null.

PostgreSQL expression placement
PlaceHolderVar 2750

BioSeqDB uses this planning mechanism where needed to keep a helper attached to the right input. ParadeDB uses this mechanism to keep score expressions at their search input while planning joins and aggregates.

ParadeDB score placement
placeholder_support 314

`PlaceHolderVar` and the changing `Const` nodes do different jobs. The first preserves expression placement while the planner builds and rewrites the tree. The second supplies a field of the current hit during execution. Neither permits unsupported join shapes to bypass the owner's validation.

2.3.3. Requested fields can require more alignment work

The owner also records which fields the SQL statement needs. For example, CIGAR output requires a base-level edit path and a formatted string. A coordinate-only query does not need that string. The requested fields are combined with the selected alignment options when the executor starts.

There is a practical limit in the current implementation: requesting CIGAR can enable further alignment work, and that path drops hits for which no usable CIGAR is produced. Therefore, adding CIGAR to a query is not guaranteed to leave its hit count unchanged. This behavior must be considered when comparing counts from queries with different output fields.

2.4. Batch work inside a row-at-a-time executor

PostgreSQL's executor asks a child node for its next row. This follows the iterator interface described in Volcano: the parent requests results one at a time.[9] A custom scan must honor that interface even when the aligner's efficient work unit is a batch of sequence pairs. BioSeqDB bridges the two sizes by retaining one batch and returning its hits one at a time.

1. Collect input. Read rows and apply supported ordinary conditions that do not depend on a hit. Retain the rows and sequence references needed for alignment.
2. Process a batch. When the batch reaches its capacity, or the input ends, submit its sequence pairs to the selected alignment pipeline.
3. Return hits. Select the next retained hit, supply its fields, and return one SQL row. Repeated requests advance through the same batch.
4. Reuse the space. After all hits have been returned, release the completed batch and collect another one if input remains.

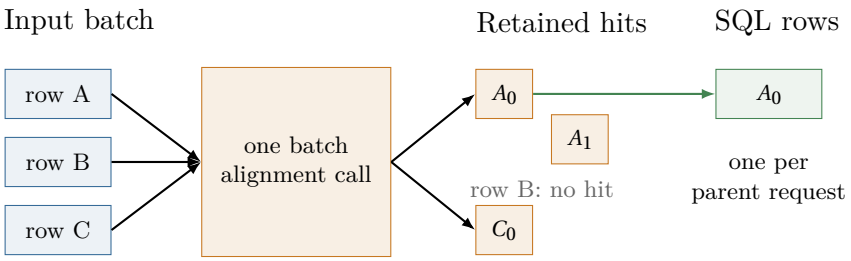


Figure 2.4: Three input rows produce three hits with a different distribution: two for A, none for B, and one for C. The parent still receives one row per request. The aligner processes batches; SQL receives individual rows.

Scalar and vectorized table access converge before alignment submission. A vectorized condition may first produce a mask of rows to keep, but surviving rows still enter the same batch mechanism. PostgreSQL parallel execution is a separate choice: the alignment owners

[9] G. Graefe, Volcano—An Extensible and Parallel Query Evaluation System, IEEE Transactions on Knowledge and Data Engineering 6(1), 1994, pp. 120–135, https://doi.org/10.1109/69.273032.

are currently marked `parallel_safe=false`. The alignment pipeline may still use its own workers or a GPU under its configuration.

2.4.1. Row stability during a search

PostgreSQL holds its current input row in temporary storage called a tuple slot. The scan may still need that row for alignment after the slot is reused. Another transaction may also update that row. The scan therefore separates its saved values from the database view that it is allowed to read:

- Keep the input values. A batch keeps ordinary fields in compact row copies in memory it owns, and keeps sequence references with their owners. A hit points to its batch entry, so several hits can return the same saved fields. Keeping a lazy reference does not require copying all sequence symbols.
- Keep the same database view. PostgreSQL uses a snapshot to decide which row versions a command can see. The sequence reader retains the scan's snapshot and checks a stored reference against the table, column, and source version visible through it. Stored source fragments are immutable. An update publishes a new version, while an older snapshot can still read its earlier version. Delaying a read does not switch it to the latest committed sequence.[10]
- Separate internal and SQL identity. The scan's internal row number locates source data; the SQL primary key identifies a logical row for the user. The batch-entry position selects saved values for a hit. None replaces the source-version and snapshot checks.
- Keep the relations available. Normal reads hold an `ACCESS SHARE` relation lock, and deferred source readers use the same mode. This prevents conflicting operations such as dropping or rewriting a relation while it is in use. Ordinary concurrent updates and deletes remain possible; the read does not lock each selected row.[11]

A rescan discards the previous batch and hit fields while preserving the scan's database view. Cleanup releases saved values before their source readers and snapshots. Reuse across later queries also needs the result-cache checks described below.

2.4.2. Filter placement and early stopping

Two kinds of condition require different inputs:

- A row condition such as `t.id > 20` depends only on stored columns. Evaluating it before alignment avoids searching discarded rows.
- A hit condition needs an alignment result. The current implementation recognizes a narrow supported form such as `bioseq.edit(t.seq) < 10` and checks the edit count of each hit. This count is the sum of mismatched, inserted, and deleted symbols in that hit's alignment; each symbol counts once. This does not imply general support for arbitrary helper expressions in `WHERE`.

[10] PostgreSQL 17, Transaction Isolation.
[11] PostgreSQL 17, Explicit Locking.

Listing 2.7: Filter input rows and then filter alignment hits.

```
SELECT t.id, bioseq.dp_score(t.seq) AS score
FROM targets AS t
WHERE t.id > 20
  AND mmsearch(t.seq, $1::bioseq.seq('dna5', 64))
  AND bioseq.edit(t.seq) < 10
LIMIT 5;
```

The identifier condition can avoid alignment for a row. The edit condition selects hits after the required alignment information exists.

`LIMIT 5` lets the parent stop asking after five rows. It does not guarantee that only five hits were computed: the current batch may already be complete. Likewise, `ORDER BY score DESC LIMIT 5` does not by itself give BioSeqDB a general top-k alignment algorithm. Producing a correct global score order can require reading all relevant hits.

2.4.3. Initialization, rescans, and cleanup

PostgreSQL defines four callbacks for the owner's execution lifetime:[12]

- `BeginCustomScan` initializes execution state once.
- `ExecCustomScan` returns the next row, or reports the end.
- `ReScanCustomScan` restarts with the current input parameters.
- `EndCustomScan` releases execution resources.

Figure 2.5 shows the normal loop and the paths that need special care.

- Initialize only execution state. A prepared plan stores descriptions of inputs, not live alignment objects. Plain `EXPLAIN` follows a description-only initialization path and submits no alignment work.
- Rescan with the new inputs. A nested-loop join may supply a different query sequence on every outer row. The owner clears retained rows and hit cursors, rebinds the execution parameters, and restarts.
- Release every owned resource. Rows, sequence references, alignment buffers, and current-hit values have bounded lifetimes. PostgreSQL errors can unwind execution without ordinary C++ stack unwinding, so cleanup must also be registered with PostgreSQL's lifetime mechanisms. It cannot rely only on C++ destructors.

2.5. Joins give sequences distinct search roles

A search between two tables needs both input rows before it can align their sequences. Assume `reads` has the same identifier and sequence columns as `targets`:

[12] PostgreSQL 17, Executing Custom Scans.

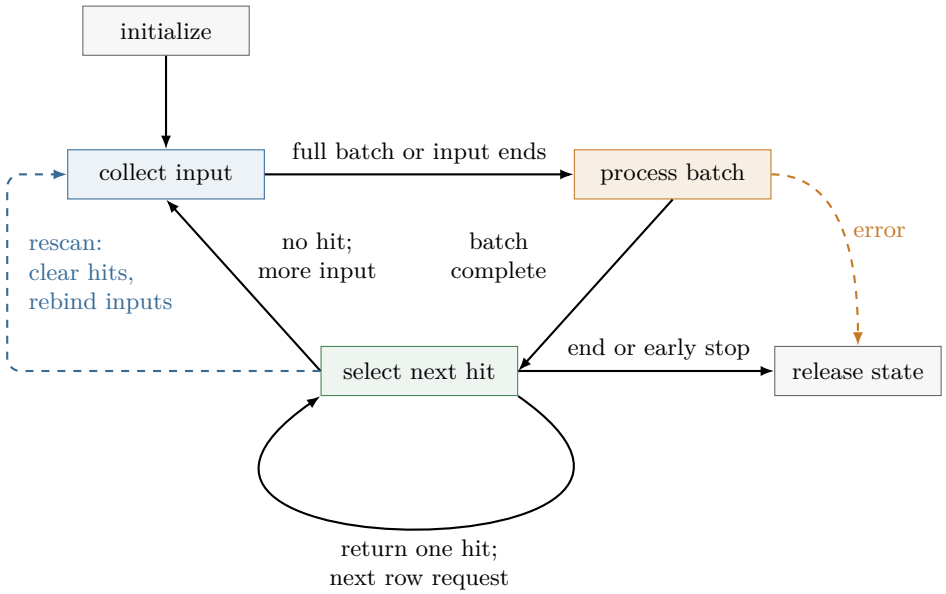


Figure 2.5: One execution owns its batches and current hit. The ordinary loop returns all hits of a batch before collecting another. A batch with no hits continues to the next input batch or ends the scan. A normal end means that no input rows or retained hits remain. A rescan starts with fresh inputs. Cleanup is also required after early termination or an error.

Listing 2.8: Search sequence pairs from two tables.

```
SELECT t.id AS target_id, q.id AS query_id,
       bioseq.dp_score(t.seq) AS score,
       bioseq.cigar(t.seq) AS cigar
FROM targets AS t
JOIN reads AS q ON mmsearch(t.seq, q.seq)
WHERE t.id > 20 AND q.id <= 100;
```

For a supported inner-join shape, `BioseqBatchJoin` owns the search. It retains all query input rows and stable sequence references, and streams the target child. It constructs batches of candidate pairs and returns each accepted hit with the ordinary columns of both rows. Retaining the query input is a real cost: the bounded alignment batch does not mean that every part of this join uses only one batch's worth of memory.

There are several possible plan placements:

- Search before a join. A base `BioseqScan` can search a target against a constant query and pass its hit rows into an ordinary SQL join.
- Search with an outer parameter. A nested-loop plan can supply a query sequence from its current outer row. Rescanning the owner then rebinds that query sequence.
- Search at the join. A supported `BioseqBatchJoin` receives target and query child plans and owns their sequence pairing.
- Evaluate a sequence expression below the owner. A `BioseqSliceProject` can make a supported slice or transformation available with a stable source identity.

These placements preserve the same rule: a hit's fields must come from the owner of its search. Ordinary join conditions still have to be evaluated at the level required by their inputs and SQL null semantics. The specialized join owner does not support every outer-join, security, or volatile-expression shape.

2.5.1. Two searches of the same stored sequence

The syntax has a cost. A helper takes a target expression, rather than an explicit hit-record argument. If both searches below used `t.seq`, the helper would not say which query supplied its score:

Listing 2.9: Two searches with one ambiguous target binding.

```
-- Unsupported: two ordinary searches share one target binding
SELECT t.id, bioseq.dp_score(t.seq)
FROM targets AS t
WHERE mmsearch(t.seq, $1::bioseq.seq('dna5', 64))
  AND mmsearch(t.seq, $2::bioseq.seq('dna5', 64));
```

Independent hit outputs need distinct, unambiguous target bindings. A self-join gives the same stored row two such roles. Since `id` is a primary key, the equality below pairs each row with itself:

Listing 2.10: Give each independent search its own target binding.

```
SELECT a.id,
       bioseq.dp_score(a.seq) AS score_a,
       bioseq.dp_score(b.seq) AS score_b
FROM targets AS a
JOIN targets AS b ON a.id = b.id
WHERE mmsearch(a.seq, $1::bioseq.seq('dna5', 64))
  AND mmsearch(b.seq, $2::bioseq.seq('dna5', 64));
```

Let a_i be hit i of the search bound to alias `a`, and let b_j be hit j of the search bound to alias `b`. Figure 2.6 shows their combinations for one target row.

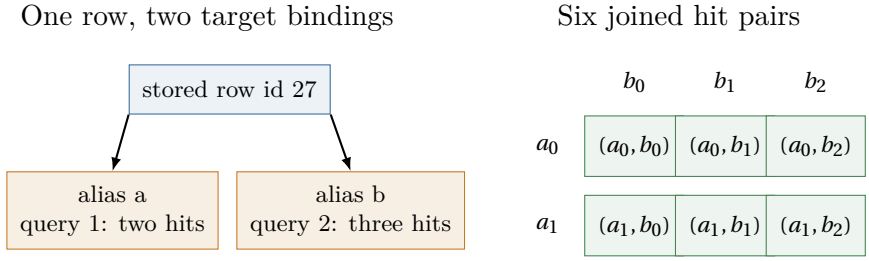


Figure 2.6: The aliases bind independent searches even when their input is the same stored row. Two hits from the first search and three from the second produce six combinations under the identifier join. Additional hit conditions could select a subset of this matrix.

This can also be stated using tuple relational calculus. Let T be the target relation and let $\mathcal{H}(s, q)$ be the relation of hit occurrences for sequence s and query q. Ignoring SQL nulls

in this small model, the self-join requests

$$\{(r.id, h_a, h_b) \mid r \in T \ \wedge \ h_a \in \mathcal{H}(r.seq, q_1) \ \wedge \ h_b \in \mathcal{H}(r.seq, q_2)\}.$$

Two independent hit variables are needed even though both refer to one stored sequence. Ordinary SQL self-joins are already useful for comparing different rows and expressing several roles. BioSeqDB adds this particular reason to use them: naming separate alignment results. The expression above also explains the 2×3 row count in Figure 2.6.

Logical aliases do not require two private copies of every sequence byte. The lazy source and storage reuse mechanisms in the previous chapter can reuse eligible underlying regions when both aliases refer to the same stored value. That reuse depends on the source, configuration, and lifetime. It is not a promise that every self-join reads storage only once. Each owner must still keep its own query, current hit, and projected fields.

The hooks use target bindings to detect unsupported stacked searches and ambiguous helpers. The current implementation also has a narrow exception: an extra constant search beside a parameterized primary search can act as an existence filter. It only decides whether primary hit rows survive and does not expose a second score or CIGAR. It therefore does not replace independent bindings when both searches' hits are required.

2.5.2. Understanding the selected plan

`EXPLAIN` shows where the search sits in the plan. A `BioseqScan` receives its target from one relation, while a `BioseqBatchJoin` receives both sides of a supported search join. Adding `ANALYZE` executes the statement and reports its row counts:

Listing 2.11: Show the search node and its output row count.

```
EXPLAIN (ANALYZE, VERBOSE)
SELECT t.id, bioseq.dp_score(t.seq) AS score
FROM targets AS t
WHERE t.id > 20
  AND mmsearch(t.seq, $1::bioseq.seq('dna5', 64));
```

The custom scan's output count is the number of emitted hits. It can exceed the count of stored input rows because one target can produce several hits. Without `ANALYZE`, `EXPLAIN` describes the selected plan without running alignment.

2.6. Sharing completed query results

An alignment query may run for hours while returning a small result. If two sessions submit the same request, running the alignment twice wastes work. The separate `bioseq_cache` extension coordinates these requests and stores completed results. This differs from the two caches used when reading sequences: a lazy read session can reuse fetched fragments, and the column cache reuses decompressed storage chunks within a backend. The result cache can serve another execution without repeating alignment.

The extension provides two forms of reuse:

- Exact results. An identical eligible request can wait for a running producer or read its completed result, including repeated rows and their stored order.
- Reusable fragments. A provider can describe which input rows a completed computation covers. A later request can read suitable results and compute only the missing part.

The generic cache implements both forms. The current BioSeqDB adapter provides exact result reuse; it does not yet provide the coverage and row identity callbacks needed for reusable fragments. The range example below explains the generic mechanism and the contract such an adapter must meet.

2.6.1. One producer, waiting readers

An entry is one attempt to compute a request over compatible data versions. The producer owns that attempt. Other executions either wait for it or read a completed entry. To seal the result is to finish writing it and close it to further rows. Figure 2.7 shows the successful path and the two ways in which an attempt stops being reusable.

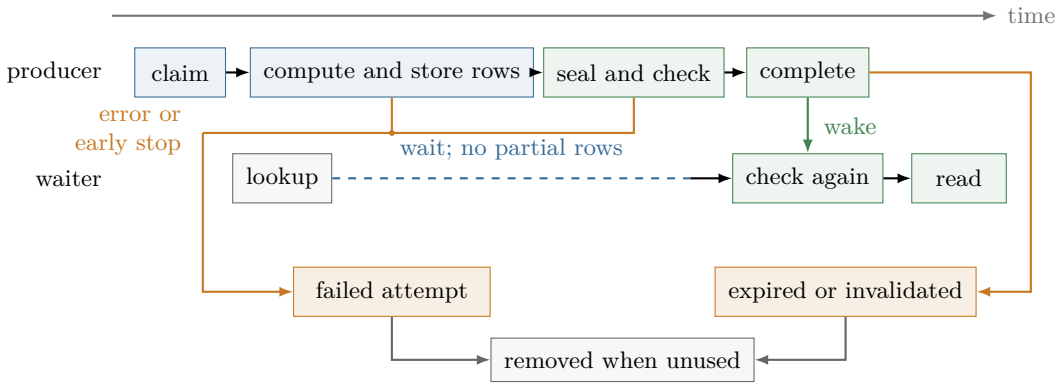


Figure 2.7: Two executions share one computation. The producer can return its own rows while building the entry. A waiter reads only after completion and validation. Failed, expired, or invalidated entries are removed after their users release them.

The lifecycle has four main rules:

- Claim briefly, compute without the metadata lock. A short shared-state update chooses one producer for a matching request and data version. The producer does not hold this lock throughout the alignment. Waiters sleep and periodically check cancellation, producer liveness, and data validity.

- Publish only complete work. Reaching the child's end of output allows the producer to seal the stored result, check dependencies, and mark the entry complete. An early cursor close cannot publish the rows collected so far as a complete result.
- Validate before reading. A wake-up means that the entry changed. The reader checks it again and keeps a reference that prevents removal while it reads. The retention time starts at completion.
- Release on every exit. Canceling a waiter releases that waiter's reference and leaves the producer running. A failed or canceled producer wakes its waiters, which can run the retained ordinary plan. A later request can start a new attempt. Waiting is declined when executions could end up waiting for one another.

Matching SQL text alone is insufficient. Exact identity includes the analyzed query, parameter values, result types, relevant execution settings, and the database and security context. Data validity is checked separately against the query's snapshot and recorded dependency versions. For example, an identical query after a committed change to its dependencies must not return an invalidated result. Permissions still undergo normal PostgreSQL checks. Unsupported or invalid cache entries leave the ordinary query plan available. Section 3.7 describes result storage and its memory and file limits.

2.6.2. Describe coverage, then compute the difference

Suppose the expensive operation and its settings remain fixed, while the user changes a metadata filter. A reusable entry stores both its results and the predicate describing the input rows it processed. A cached source is a completed entry selected for reuse. This is the idea of semantic caching: describe the cached region, read the useful part, and compute a remainder.[13] Coverage records completed work even when some input rows produced no hits. The absence of a saved hit alone cannot prove that its input was searched.

For example, select input rows with $40 \le id < 100$. Completed sources cover $0 \le id < 60$ and $50 \le id < 80$. We read $[40, 60)$ from the first source, $[60, 80)$ from the second, and compute $[80, 100)$. These ranges select metadata row identifiers, not positions within a biological sequence.

We can state the rule in tuple relational calculus. Fix one compatible database snapshot, search query, alignment settings, and requested match fields that affect which hits are emitted. Define:

- $T(t)$: t is an input row whose identity is retained;
- $A(t, h)$: the operation emits hit occurrence h for row t; h includes a distinct identity even when two hits have equal displayed values;
- $p(t)$: the new SQL predicate P evaluates to `TRUE` on t;
- $c_i(t)$: cached source i passed validation and its recorded predicate P_i evaluates to `TRUE` on t.

[13] Dar, Franklin, Jónsson, Srivastava, and Tan, Semantic Data Caching and Replacement, VLDB 1996, pp. 330–341.

Order the selected sources from 1 to m. Source i supplies branch B_i; the relation D contains the input rows that still need computation:

$$B_i = \{(t,h) \mid T(t) \wedge A(t,h) \wedge p(t) \wedge c_i(t) \wedge \forall j < i : \neg c_j(t)\}, \tag{2.1}$$

$$D = \{t \mid T(t) \wedge p(t) \wedge \forall i \in \{1,\ldots,m\} : \neg c_i(t)\}. \tag{2.2}$$

Each requested hit is returned from the first valid source covering its input, or computed from D. Thus

$$\{(t,h) \mid T(t) \wedge A(t,h) \wedge p(t)\} = B_1 \uplus \cdots \uplus B_m \uplus \{(t,h) \mid t \in D \wedge A(t,h)\}. \tag{2.3}$$

Here $\uplus$ joins disjoint sets of occurrences. The operation is evaluated only on missing inputs D; A is a description of the result, not an instruction to compute every possible hit first. Removing the hidden identities afterwards preserves repeated SQL rows. SQL `UNION ALL` has the required concatenation behavior; `UNION` could remove legitimate repeated rows.[14]

2.6.3. An overlapping range example

Figure 2.8 shows the same example, including the overlap between the two sources.

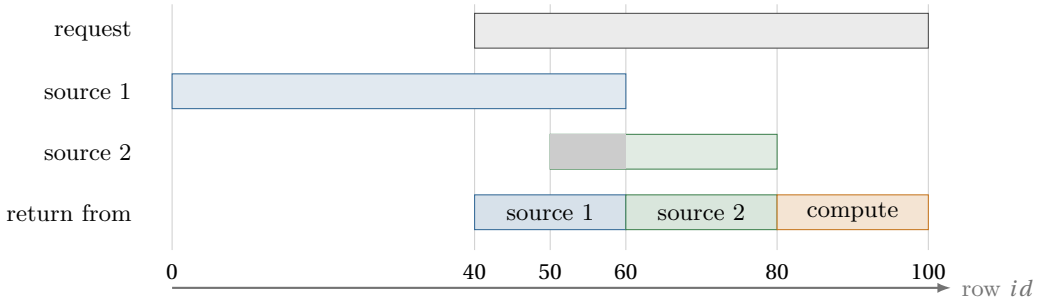


Figure 2.8: The new request uses source 1 for $[40,60)$, source 2 for $[60,80)$, and computes $[80,100)$. The gray overlap in source 2 is omitted because source 1 already supplies those occurrences. Cached rows outside the new request are also omitted.

With both sources valid, the missing inputs have this equivalent SQL description:

Listing 2.12: Input rows on which the expensive operation is still needed.

```
SELECT id, seq
FROM sequences
WHERE id >= 40 AND id < 100
  AND (id >=  0 AND id < 60) IS NOT TRUE
  AND (id >= 50 AND id < 80) IS NOT TRUE;
```

The cache builds PostgreSQL expression nodes for this condition; it does not construct and parse a new SQL string. A provider must place the condition before the expensive operation. The listing describes the input selection and does not introduce a new public search function.

Three details make this transformation correct:

- Handle SQL NULLs. A `WHERE` clause accepts only `TRUE`. For a nullable predicate, `NOT P` also becomes unknown when P is unknown. `P IS NOT TRUE` includes both false and unknown, so it keeps every input excluded from the earlier cache.[15]

[14] PostgreSQL 17, Combining Queries.

[15] PostgreSQL 17, Comparison Functions and Operators, Table 9.2.

- Restore invalid coverage. All source checks finish before output begins. If source 1 fails validation, c_1 is false: source 2 supplies $[50, 80)$ and D becomes $[40, 50) \cup [80, 100)$. An internal Boolean parameter enables each exclusion only for an accepted source.
- Require enough information. A reusable provider must keep input and hit identities, prove that results have compatible meaning, and supply columns needed to test the new predicate. Missing metadata may be fetched from the original rows under the validated snapshot. A result containing coordinates alone cannot supply an uncomputed CIGAR.

Coverage is taken at a complete expensive-operation boundary. Joins, aggregation, and ordering above that boundary run again on the reconstructed rows. If an upper `LIMIT` stops the boundary early, its partial rows cannot become a complete coverage entry, even when the final limited query has a complete exact result. These rules allow reuse to save computation while keeping PostgreSQL responsible for the surrounding query.

Chapter 3

Physical sequence storage in PostgreSQL

3.1. Store sequences together with their metadata

A sequence search usually needs more than sequence symbols. A researcher may search samples collected on a particular date, return their identifiers, and join the hits to gene annotations. The database must keep these relationships while allowing a search to read a small part of a long sequence.

Existing systems show why these relationships matter. BioWorkbench imports variant and gene annotations into SQLite and joins them with SQL.[1] The IBM Functional Genomics Platform uses a DB2 column store and mapping tables to connect genomes, genes, proteins, and annotations.[2] Database-backed sequence search has also been studied with DynamoDB records that pair each sequence with an organism name.[3] These examples motivate keeping biological data queryable. Here the engineering question is how to store the sequence itself so that database execution can request its symbols as needed.

Consider a table `reads(id, sample_id, seq)`. The following query returns identifiers and does not need sequence symbols:

Listing 3.1: A metadata query should not read sequence payloads.

```
SELECT id
FROM reads
WHERE sample_id BETWEEN 500 AND 600;
```

An alignment query on the same rows does need symbols, but it may request several short intervals rather than the complete sequence. The sequence interfaces in Section 1.5 already express those requests. Storage must support them without first copying a whole chromosome into one value.

PostgreSQL already stores large values out of line through TOAST. For some uncompressed external values, a substring can fetch only the relevant pieces.[4] BioSeqDB additionally needs packed biological alphabets and a fragment reader usable by the alignment algorithms. A sequence's packed payload can also exceed the ordinary one-gigabyte limit for a PostgreSQL variable-length value. BioSeqDB therefore separates three objects:

[1] Mondelli et al., "BioWorkbench: a high-performance framework for managing and analyzing bioinformatics experiments."

[2] Seabolt et al., IBM Functional Genomics Platform, A Cloud-Based Platform for Studying Microbial Life at Scale.

[3] Flach et al., "Analysis of Biological Sequence Search Performance in NoSQL Database."

[4] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: TOAST.

- The parent row holds ordinary metadata and a small sequence locator. A locator identifies where the packed sequence is stored.
- Fragment rows hold fixed-size pieces of packed sequence data in internal PostgreSQL relations. A reader fetches fragment symbols only when they are needed.
- Persistent indexes keep lookup information derived from those symbols. Their identity and storage are described in the next chapter.

⚠ Adapted from Citus and Hydra Columnar

BioSeqDB's columnar layer adapts code from both Citus Columnar and Hydra Columnar.[a] Hydra extended the Citus code with features including vectorized SQL execution and a cache of decompressed column buffers. BioSeqDB modernized the adapted C code into C++26 modules and substantially changed the storage, reads, and updates. Sequence locators, fragment readers, and sequence indexes are BioSeqDB-specific additions.

[a] Citus Data contributors, Citus Columnar storage implementation; Hydra contributors, Hydra Columnar: PostgreSQL columnar storage extension.

The linked Hydra columnar documentation was unavailable when consulted for this work. Understanding the chunk operations therefore required tracing the available source code to reconstruct their write, read, and cache lifetimes. The repository remains a public reference for the original implementation.[5]

The table access method connects this layout to PostgreSQL's table operations.[6] Citus illustrates the broader value of building database features through PostgreSQL's extension interfaces, including hooks and transaction callbacks.[7] The following sections explain the particular choices made in BioSeqDB.

3.2. Read the needed columns and row groups

The parent table uses a columnar layout because sequence workflows often scan many rows while using only a few metadata columns. Keeping sequence symbols outside the parent row already avoids fetching them for a metadata query. Columnar storage adds further ways to save work:

- Read fewer columns. Values from the same column are stored together. A scan of identifiers and sample numbers can leave unrelated metadata and sequence locators unread.
- Compress similar values together. A buffer containing one column's values uses a common byte encoding and often contains repeated patterns. This is ordinary column compression, separate from packing biological symbols into a few bits each.
- Process values in batches. In vectorized SQL execution, a supported operator works on an array of column values. A supported comparison can produce a mask of rows that

[5] Hydra contributors, Hydra Columnar: PostgreSQL columnar storage extension.

[6] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Table Access Method Interface Definition.

[7] Cubukcu et al., "Citus: Distributed PostgreSQL for Data-Intensive Applications."

pass the condition. This reduces repeated per-row operator calls and allows a loop over many values of one type. A batch does not imply that every operation uses hardware SIMD.

- Reuse decompressed buffers. A column cache can keep the output of decompression for later reads. This avoids decompressing the same bytes again, although reconstructing SQL values can still require work. BioSeqDB's optional cache and its lifetime are described in Section 3.6.1.

Hydra's batch execution and column cache were important reasons to use its extension of the Citus design. Its public reader checks the decompressed column cache before decompressing a buffer. Hydra column-cache lookup and decompression columnar_reader.c 1916 These SQL operations are separate from BioSeqDB's SIMD minimizer, chaining, and alignment algorithms. The two layers can reduce different parts of a query's cost. Column storage also favors bulk appends and analytical scans; it is not a universal improvement for frequent small updates.

3.2.1. What analytical benchmarks motivate

ClickBench supplies a useful example of the potential benefit on a large table. Figure 3.1 uses historical PostgreSQL, Citus, and Hydra Columnar submissions on the same named machine configuration.[8] All contain results for the same 43 queries over the roughly 100-million-row web-analytics dataset. For each query, take the shorter of its second and third runs, then sum those 43 times. We call this the sum of hot-query times. Loading time is excluded. The figure also shows the submissions' reported storage sizes.

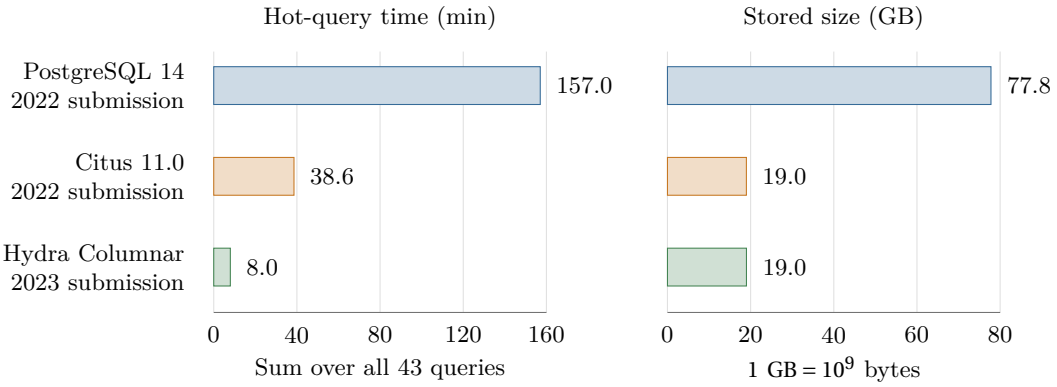


Figure 3.1: Historical ClickBench workload comparison, calculated from the repository snapshot of 20 September 2023. Each submission uses one `c6a.4xlarge` machine with 16 vCPUs, 32 GiB RAM, and 500 GB of GP2 storage. PostgreSQL and Citus results are dated 1 July 2022, and Hydra results 19 September 2023. The times use the shorter of runs two and three for every query. Versions, settings, and execution paths differ. These are external SQL workload results, not BioSeqDB alignment measurements.

Both columnar submissions store about 19 GB, compared with PostgreSQL's 77.8 GB. Hydra has a much shorter sum of query times than Citus despite their similar stored sizes. This motivates considering execution and reuse as well as compression. It does not measure

[8] ClickHouse contributors, ClickBench: analytical database benchmark.

how much of the difference comes from vectorization: parallelism, caching, and PostgreSQL settings also differ. Hydra itself attributes its performance to these mechanisms together.[9]

Hydra's benchmark notes likewise separate SELECT execution time from data loading. They distinguish release tests on `c6a.4xlarge` from continuous tests on four-core runners.[10]

3.2.2. Group rows into stripes and chunks

The columnar layout has three levels:

- A stripe is a batch of appended rows. The default maximum is 150,000 rows.
- A chunk group is a smaller row interval within a stripe. The default maximum is 10,000 rows. Every column uses the same group boundaries.
- A column chunk contains one column's values for one chunk group. It has a presence bitmap and a value buffer. A presence bit says whether that row contains a non-NULL value.

Stripe and chunk-group sizes are table options. Smaller groups give finer read selection, while larger groups give the compressor more values to work with. This is the grouping model described by Citus Columnar.[11]

3.2.3. Two independent ways to avoid reads

Projection selects the columns required by the query. The sample query needs `id` for its output and `sample_id` for its condition. It does not need `seq`.

Min/max pruning selects chunk groups. For an ordinary ordered column, the writer records the smallest and largest non-NULL values in each chunk. A condition can reject a group if none of its values could satisfy the condition. For example, a group with sample identifiers from 1 to 100 cannot satisfy `BETWEEN 500 AND 600`.

Figure 3.2 shows both decisions. A retained group still needs filtering. Each retained row must be tested against the condition: bounds of 450 and 700 do not say which rows fall between 500 and 600.

[9] Hydra contributors, Hydra Columnar: PostgreSQL columnar storage extension.
[10] Hydra contributors, Hydra Columnar benchmarks.
[11] Davis, Citus 10 brings columnar compression to Postgres.

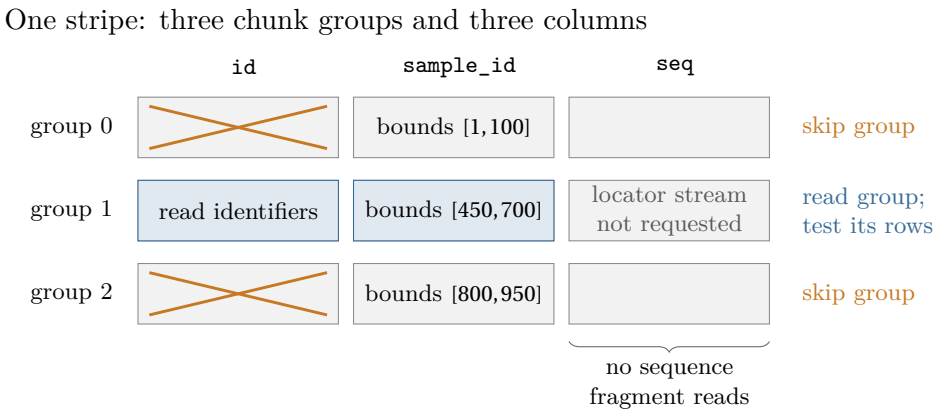


Figure 3.2: Column selection and group pruning for the metadata query. Blue cells are read. Orange crosses mark groups whose scalar bounds exclude every row. Unrequested sequence locators remain unread. The drawing shows logical chunks, not their byte order on disk.

Sequence columns are excluded from this min/max test. A locator's byte order does not describe sequence similarity. A separate scalar length, date, or sample identifier can supply useful bounds. When a supported comparison or usable bound is missing, the reader keeps the group.

3.2.4. A column chunk separates missing values from values

Figure 3.3 uses six rows to show the two buffers in one column chunk. The presence bitmap has one bit per row. Only present values enter the value buffer, with the byte alignment required by their PostgreSQL types. The reader uses the bitmap to put values back into their row positions.

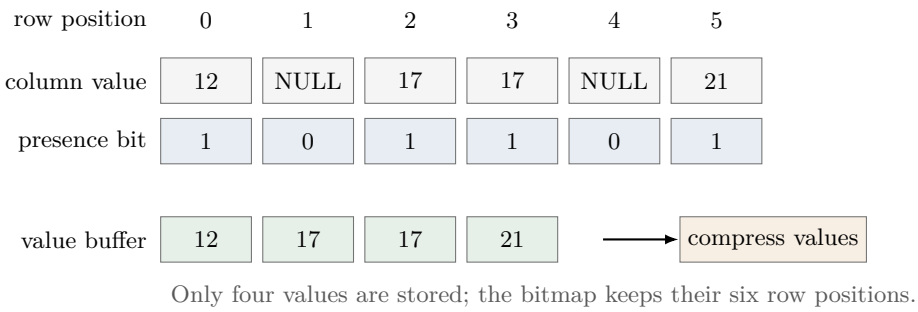


Figure 3.3: Presence and values have different jobs. Generic column compression acts on the serialized value buffer. Sequence rank packing, introduced earlier, is a separate encoding step.

The value buffer can use PostgreSQL PGLZ, LZ4, or Zstandard, depending on the build and table options. Metadata records the compression method used and the decoded length. Successful LZ4 and Zstandard output is not always smaller than its input, so compression does not guarantee a space saving for every chunk. No particular compression ratio is assumed here.

Abadi et al. show that the choice of compression method depends on both the data and the query workload.[12] For compressed columns, BioSeqDB reconstructs SQL values from decoded buffers, obtained by decompression or cache reuse, so both stored size and decoding work matter.

The physical order differs from the logical grid. Within a stripe, one column's presence buffers are written first, followed by that column's value buffers. The next column follows. Stored offsets locate each buffer without requiring the reader to decode earlier columns. A column added after a stripe was written can be supplied from its constant PostgreSQL default or as NULL.

3.3. Store byte streams in PostgreSQL relation pages

A compressed buffer need not fit in one database page. BioSeqDB gives column writers a continuous logical byte space and maps it onto page payloads. Block 0 contains the metapage, block 1 is reserved, and stream data begin in block 2. Each data page keeps its PostgreSQL page header.

Let BLCKSZ be the PostgreSQL page size in bytes, H the page-header size, and $P = \text{BLCKSZ} - H$ the remaining payload capacity. Measure $x \geq 0$ from the first stream byte in block 2. Then

$$\text{block}(x) = 2 + \lfloor x/P \rfloor, \qquad \text{position}(x) = H + (x \bmod P).$$

This choice of origin makes the stream's first byte $x = 0$. Page headers do not consume positions in the logical byte stream. Figure 3.4 shows a request that crosses a page boundary.

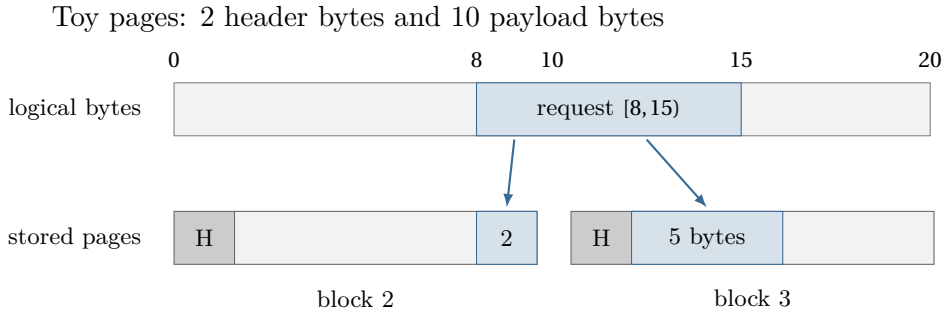


Figure 3.4: Logical offsets hide page boundaries. The example uses deliberately small pages to make the split visible. Actual page sizes and header sizes come from the PostgreSQL build.

This mapping follows the Citus column-store design. The public source gives the corresponding page and logical-offset operations. Citus columnar page mapping columnar_storage.c 118 PostgreSQL documents the common page header separately from the layout an access method puts after it.[13]

3.3.1. Reserve space, write bytes, publish metadata

Concurrent writers must not allocate the same row numbers or byte interval. The metapage records the next stripe number, row number, and free byte offset. The write path separates allocation from visibility. It has three responsibilities:

[12] Abadi, Madden, and Ferreira, "Integrating compression and execution in column-oriented database systems."

[13] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Database Page Layout.

- Reserve. Under the relation extension lock, reserve non-overlapping identifiers and byte space.
- Write. Copy the stream bytes into relation pages. Page changes use PostgreSQL buffer locks and generic write-ahead logging (WAL).
- Record completed data. Store rows, offsets, lengths, and chunk metadata in the catalogs. The bytes and catalog changes belong to the same transaction. Other transactions can read a completed stripe when its committed metadata is visible under their snapshots.

A reserved byte range alone is not a readable stripe. If a write does not complete, the range does not authorize readers to interpret unfinished bytes as table rows. Generic WAL supplies page-change recovery for logged relations;[14] the storage code still has to publish the right catalog state and release its locks correctly.

3.3.2. Why use relations instead of private payload files?

The primary column and fragment data use PostgreSQL relations. A read obtains a shared database buffer. A page pin keeps that buffer assigned to the page while it is in use; a buffer lock protects its contents during the copy. After copying the requested bytes, the reader releases the pin and lock. The copied column buffer belongs to the reader. A sequence view therefore does not keep every database page it has touched pinned for the rest of the query.

This choice reuses PostgreSQL's shared buffer manager, relation identity, locking, and WAL path. A private-file implementation would need separate rules connecting its writes, deletion, and recovery to database transactions. These benefits do not establish every transaction property of a custom access method automatically. They identify the machinery used for these particular bytes. Derived index images can use a different read mechanism, described in the next chapter.

3.4. A locator addresses packed sequence fragments

When sequence symbols are stored outside the parent row, the row keeps a 56-byte sequence value: a 16-byte sequence header followed by a 40-byte locator. The header retains the alphabet and packed storage width. The locator supplies the information needed to find and validate the source:

- the length bucket and shard that hold the sequence,
- its first fragment row and number of fragments,
- its logical length in symbols, and
- its source epoch, an identifier for this immutable stored sequence version.

The current locator format is version 3. Readers check the version, non-zero epoch, fragment count, and final padding before accepting it. They also check the full registered source identity, which includes the parent table and column. A matching fragment row number by itself is insufficient: another relation or source version could use the same number.

[14] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Generic WAL Records.

3.4.1. Symbols, packed units, fragments, and pages

The units have different meanings. A packed storage unit of B bytes holds L symbols according to the alphabet encoding from Section 1.2. A fragment reserves 1,024 of these units. Thus a full fragment has

$$\text{payload bytes} = 1024B, \qquad \text{symbol capacity} = 1024L.$$

For DNA4 and 32-bit units, $B = 4$ and $L = 16$: one fragment has 4,096 payload bytes and room for 16,384 symbols. With 64-bit units, it has 8,192 payload bytes and room for 32,768 symbols. A fragment is consequently not a synonym for a PostgreSQL page, even when their sizes happen to be similar.

Consider a 40,000-symbol DNA4 sequence stored with 32-bit units. It needs three fragment rows. The last row uses only 7,232 of its symbol positions, but its packed payload allocation still has the full fragment size. The locator's length prevents a read from returning that unused capacity. This is separate from unused symbol positions in the final packed unit. If the first fragment row number is r_0, fragment i is at $r_0 + i$. Consecutive row numbers avoid a separate stored address for every fragment. The locator needs only the first row number and the fragment count.

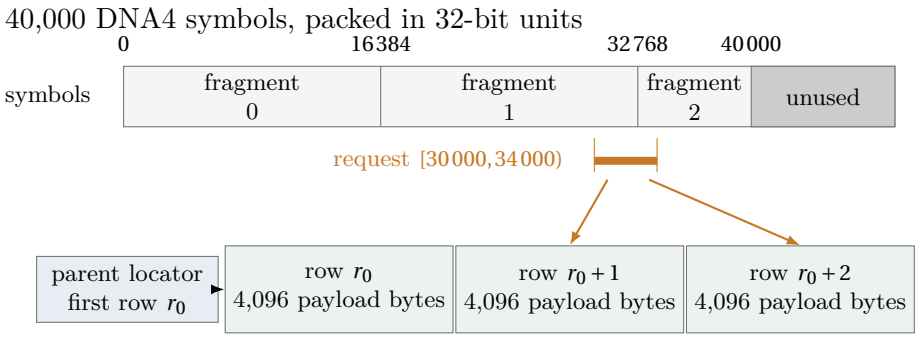


Figure 3.5: A symbol interval becomes fragment-row requests. Equal-width fragments reserve equal packed space. Only the used prefix of the final fragment belongs to the sequence. Underlying column chunks and database pages can contain additional data, so two requested fragments do not imply exactly two physical reads.

Let n be the sequence length and F the number of symbols per fragment. For a requested interval $[a, b)$ with $0 \le a < b \le n$, the required fragment numbers run from

$$\lfloor a/F \rfloor \quad \text{through} \quad \lfloor (b-1)/F \rfloor .$$

The subtraction by one keeps a request ending exactly at a boundary from reading the following fragment. This is the physical counterpart of the half-open slice convention introduced earlier.

3.4.2. Length buckets separate short and long values

A length bucket groups sequences by their logical number of symbols. A shard is an internal fragment relation within one bucket. Both belong to a particular sequence column. Length buckets are local storage groups, not partitions sent to different database servers. For known-length input, a long reference sequence does not fill the active shard used for short reads.

The current layout has 35 buckets. The first covers lengths through 256, the next through 512, and the next through 1,024. The doubling ranges continue through 2^{41}. The final bucket covers larger lengths. Relations are created when needed. A table does not eagerly create every possible bucket and shard.

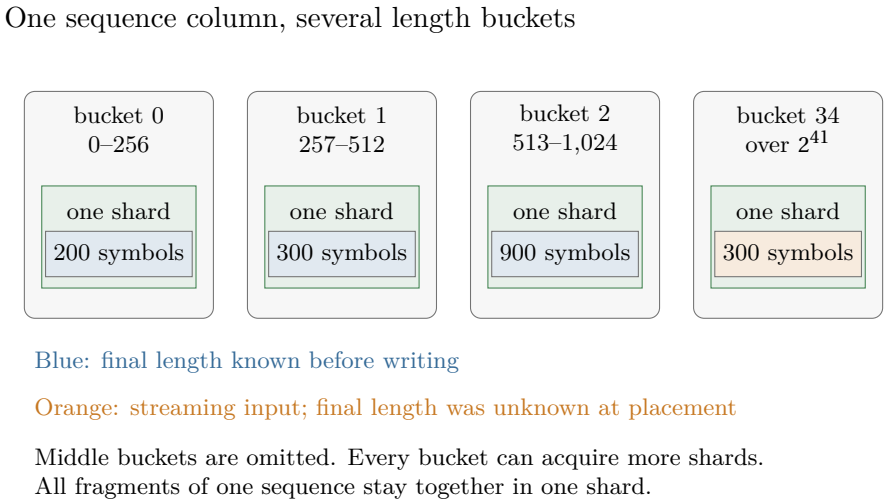


Figure 3.6: Length-based placement and its streaming exception. Known-length inputs choose their length range. Streaming inputs currently use bucket 34, even when the completed sequence is short.

Known-length insertion can choose a shard before writing:

1. Choose the length bucket and calculate the space for its full fragment rows, including padding and row headers.
2. Check the active shard's recorded and pending use against its configured size limit.
3. If the sequence fits but the active shard lacks space, open a new shard. If the sequence alone exceeds the limit, reject it.
4. Reserve consecutive fragment rows in the selected shard, write them, and retain their first row number in the parent locator.

The default configured shard size is 1 TiB. This is an allocation limit based on fragment-row sizes, not a promise about compressed disk size. Streaming input currently has a narrower check: it verifies the growing sequence's size, but does not first reserve space for its unknown total size in the active shard. It also remains in the highest bucket after the final length becomes known. Thus the limit is not a strict total-size guarantee for a shard receiving streaming inputs.

Transaction-scoped advisory locks coordinate allocation within a parent column and bucket, and writes within a selected shard. The compact lock key only coordinates writers. Locator validation uses the complete registered identity, so a lock-key collision can cause extra waiting without making unrelated sequence addresses interchangeable.

3.5. Keep old readers stable when a row changes

A lazy reader may fetch symbols long after it first receives a parent row. If another transaction changes the sequence meanwhile, that later fetch must still return the version belonging to

the reader's database snapshot. PostgreSQL's multiversion concurrency control (MVCC) lets snapshots select different committed row versions.[15] BioSeqDB connects its source epochs and deletion masks to that visibility decision.

3.5.1. Deletion changes visibility before it changes storage

Compressed column streams and stored sequence fragments are not edited in place to remove one row. A deletion mask records which internal row numbers are deleted. A mask bit is separate from a column's presence bit: the first removes a row from a scan, while the second describes a NULL value in a row that exists.

For a parent row with sequence data, deletion performs two related changes:

- It invalidates the source epoch for the old row version in the source catalog.
- It marks that internal row number as deleted in the row-mask metadata.

Both catalog changes follow the transaction's visibility. They do not overwrite the old fragment bytes. An older registered snapshot can still see the earlier source catalog entry and the earlier deletion mask. Its later lazy read therefore reaches the same immutable source.

An update uses this removal step and then appends a replacement row. The replacement has new sequence storage and a new source epoch. Its SQL primary key can remain unchanged even though its internal row number changes. Figure 3.7 shows the distinction.

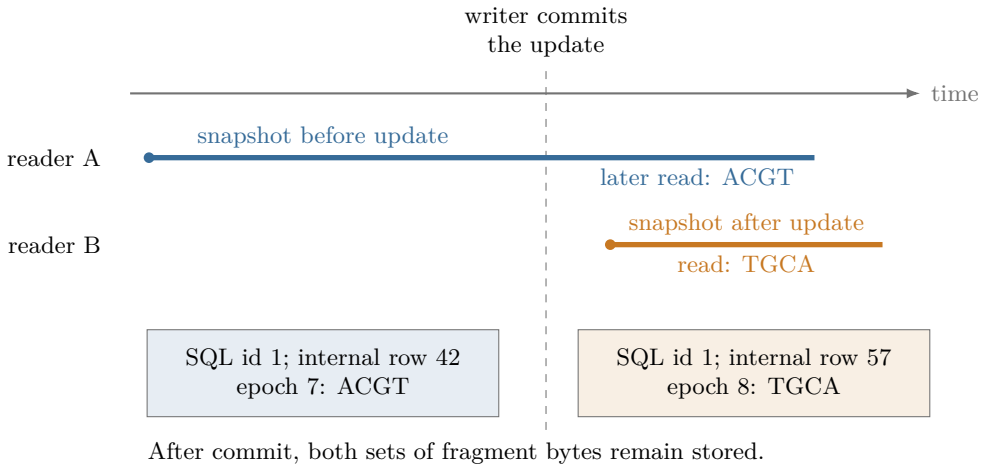


Figure 3.7: Two snapshots can read different versions of one logical row. The identifiers are illustrative. Reader A keeps its earlier snapshot throughout its read session. A plain command started later can use reader B's newer view. The old bytes are retained, not modified into the new sequence.

Listing 3.2 gives a two-session example of this visible behavior.

The sequence reader registers the scan snapshot. Keeping its C++ handle alive does not permit reads after the owning transaction ends. This is the same read-session rule described in Section 1.6.

[15] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Introduction to Concurrency Control.

Listing 3.2: A repeatable-read transaction keeps the old sequence version. Initially, row id 1 begins with ACGTACGT.

```
-- Session A
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT bioseq.to_text(seq[0:8]) FROM reads WHERE id = 1;
-- ACGTACGT

-- Session B, while A remains open
BEGIN;
UPDATE reads
SET seq = repeat('TGCA', 4096)::bioseq.seq('dna4', 32)
WHERE id = 1;
COMMIT;

-- Session A
SELECT bioseq.to_text(seq[0:8]) FROM reads WHERE id = 1;
-- ACGTACGT
COMMIT;
SELECT bioseq.to_text(seq[0:8]) FROM reads WHERE id = 1;
-- TGCATGCA
```

3.5.2. Locks protect different parts of the operation

PostgreSQL distinguishes relation locks, advisory locks, and short locks inside its buffer manager.[16] BioSeqDB uses them for different purposes:

- Relation locks keep storage available. Ordinary readers and deferred source readers use `ACCESS SHARE`. It conflicts with operations that require `ACCESS EXCLUSIVE`, such as dropping the relation. It permits ordinary concurrent updates and deletes.
- Writer locks coordinate mutations. Parent update and delete operations take a transaction advisory lock for the parent table's storage identity. Fragment allocation and writing use their own advisory-lock keys.
- Buffer locks protect a page copy or change. These short locks are released after the page operation. They are not held for the duration of an alignment.

The combination supports stable snapshot reads without locking every row returned by a search. It is a description of the implemented read and write paths, not a claim that every PostgreSQL row-locking feature has identical behavior in the custom table access method.

3.5.3. Removing bytes is a separate maintenance operation

Deleting one parent row does not immediately erase its fragment rows or its persistent index payload. This preserves old readers, but also means that logical deletion alone does not reclaim all occupied space. A reader using a newer snapshot must reject a source epoch invalidated in that snapshot, even while its bytes remain present.

Registered fragment relations reject direct update, delete, and truncate operations. Ordinary vacuum does not compact their live rows into new positions, because a locator depends

[16] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Explicit Locking.

on consecutive internal row numbers. An unused physical tail can be truncated when the required lock is available.

Parent-table maintenance owns the larger cleanup boundary. Dropping or truncating the parent removes its registered fragment relations. A full table rewrite reconstructs visible sequence values in a new storage family and then removes the old family. The current full-rewrite path can assemble a complete sequence while copying it, so the bounded fragment-read guarantee must not be extended to that maintenance operation.

Cleanup can also find orphaned fragment relations or stale registry rows. This is relation-level cleanup, not a background collector that packs individual deleted sequences together. Derived index generations have separate validity and cleanup rules, discussed in the next chapter.

3.6. Load fragments when a consumer needs them

Reading a parent table and reading sequence symbols are separate steps:

1. Read visible stripe and chunk metadata under the scan snapshot.
2. Select the required columns and reject chunk groups contradicted by supported scalar bounds.
3. Read the selected presence and stored value buffers, decode a group, and apply row masks and remaining conditions.
4. Bind the surviving row's locator to its source, validating the source identity, epoch, and fragment geometry under the scan snapshot.
5. When symbols are requested, fetch the fragment rows covering that interval and check their packed representation.

Chernishev et al. study late materialization, which uses row positions to postpone tuple reconstruction.[17] In BioSeqDB's lazy read path, a tuple carries a sequence locator, and fragments are fetched when an operation needs their symbols. Sequence contents can therefore remain unread after PostgreSQL has formed a tuple containing the locator.

The column reader loads stored buffers for all selected groups of a stripe before decoding groups in turn. Thus avoiding a whole-sequence allocation does not mean that the complete scan retains only one compressed chunk. Likewise, a fragment request can cause its containing column chunk to be read and decoded. The logical request and the physical I/O unit differ.

The table reader can return ordinary tuple slots or column batches containing projected values, NULL masks, a mask of surviving rows, and internal row numbers. Supported scalar conditions can update a column batch's mask. Surviving rows can then be extracted into ordinary tuple slots. Alignment separately collects its input rows into the batches described in Section 2.4. These are different work units. Paths that pass source locators through or evaluate sequence expressions retain scalar fetch where the vector conditions are disabled. Batching itself does not imply a hardware SIMD instruction for every expression.

How a scanner copies a fragment into its own slots, how long those slots live, and how that copy sits next to the column chunk cache is described in Section 1.5.

[17] Chernishev et al., "A Comprehensive Study of Late Materialization Strategies for a Disk-Based Column-Store."

3.6.1. Decoded column chunks have a different cache

The optional column decompression cache stores decoded compressed chunks inside one PostgreSQL backend. Its key contains relation, stripe, chunk group, and column identifiers. It is disabled by default; the configured capacity when enabled defaults to 200 MiB.

The cache counts reads when selecting entries to remove. It protects one current chunk group per relation from removal. A cache hit returns a borrowed buffer, which must not outlive its cache entry. Uncompressed chunks do not need this decompression cache. Reuse can save repeated decoding during a scan or rescan before scan cleanup clears the cache. It does not share a decoded buffer between sessions.

Table 3.1: Stored bytes and reusable memory have different owners.

Object	Retained data	Scope
PostgreSQL buffer	Relation page bytes	Shared database buffer manager; short pins protect each page operation
Fragment slot	Copy of one packed fragment	Sequence scanner and its read-session owner
Column cache	Decoded column value buffer	One backend, subject to its configured byte limit
Query-result cache	Completed SQL result rows	Separate extension shared across backends

3.7. Physical storage of shared query results

The separate `bioseq_cache` extension stores completed SQL results, not source fragments. Its producer and waiter lifecycle, exact matching, and generic range-reuse contract are described in Section 2.6. The physical store groups result rows into columnar blocks with a schema description, NULL information, value offsets, and checksums. Variable-length values are flattened before storage; process-local pointers cannot become reusable result bytes.

Completed blocks can live in shared memory or cache-owned private files. Before storing a block, the cache reserves capacity. For shared memory, it checks the configured byte ceiling and the minimum amount of system memory that must be left free. Before writing a file block, it also checks the file capacity limit and the minimum amount of disk space that must be left free. If neither tier can accept another block, the original query continues without publishing a reusable result. Active readers keep references that prevent their blocks from being removed.

This file tier is derived data with its own cleanup rules. Cached results are cleared across a server restart. It should not be confused with the PostgreSQL relations holding authoritative sequence data or with persistent sequence indexes.

3.8. From stable sources to persistent indexes

The storage layer lets two queries use the same table in different ways. A metadata query reads selected column chunks and can avoid sequence access entirely. An alignment query keeps a validated locator and asks for packed fragments under the same snapshot as its input

row. Updating or deleting the row changes what newer snapshots may read while preserving the old source for an existing reader.

The next chapter builds reusable lookup structures on these sources. It uses the same distinction between a location, a source version, and its lifetime. An index can still occupy bytes after its source is invalidated; being present in storage is not sufficient permission to use it.

Chapter 4

Persistent sequence indexes

4.1. Keep the lookup work between queries

The sequence store can fetch a small interval without decoding a whole chromosome. Seed lookup has a different question: at which positions does a query minimizer occur in the target? Reading every target symbol and rebuilding its minimizer table for each query would repeat much of the same work. A persistent index saves that table between executions.

The useful object is a mapping from a seed key to its occurrences. A posting is one stored occurrence, including a position and any source or strand information needed to interpret it. A posting list contains all stored occurrences for one key. The key may be a minimizer hash, as in BioSeqDB and minimap2, or a spaced word, as in the CPU MMseqs2 prefilter. The seed-selection rule determines which occurrences the index stores.

Reuse of reference indexes already appears in biological workflows. For example, BioWorkbench's RASflow workflow checks for an existing Bowtie2 index before preparing it again.[1] MMseqs2's interactive search system also saves precomputed search structures to reduce setup for repeated requests.[2] BioSeqDB has to connect this reuse to the database's changing rows and to the search operators from the previous chapters.

Four questions guide the design:

- What is stored? Minimizer positions answer seed lookups; symbol counts answer a different question about sequence composition.
- How is it grouped? Grouping by target interval makes construction local. Grouping by hash makes later reads selective.
- When is it usable? The index must describe the same source version and minimizer rules as the query expects.
- Who keeps its bytes alive? A reader needs valid storage and buffers until its last lookup, including when another transaction changes the source.

An index is derived data: removing it does not remove the sequence. This also distinguishes it from the completed-query cache in Section 2.6. A minimizer index can serve many different query sequences; a cached result describes completed executions.

[1] Mondelli et al., "BioWorkbench: a high-performance framework for managing and analyzing bioinformatics experiments."

[2] Milot Mirdita, Steinegger, and Söding, "MMseqs2 desktop and local web server app for fast, interactive sequence searches."

4.2. What existing sequence indexes store

4.2.1. Minimap2: buckets, singletons, and position lists

Minimap2 selects minimizers from the reference and groups their occurrences by hash.[3] Its index construction first uses the lowest b hash bits to choose one of 2^b buckets. Within each bucket, sorting places equal hashes together. A bucket-local hash table then records where their positions can be found. The construction issue and the accompanying format explanation give a useful view of this layout.[4]

There are two cases:

- A singleton occurs once. Its position is stored directly in the hash-table value.
- A repeated key has several occurrences. Its value stores an offset and a count into the bucket's separate position array.

Each position record includes the reference sequence identifier, position, and strand. A marker bit in the stored key distinguishes the two cases. Figure 4.1 uses small integer hashes to show the partition and lookup; it omits the marker's bit encoding. Its positions are zero-based last-symbol coordinates.

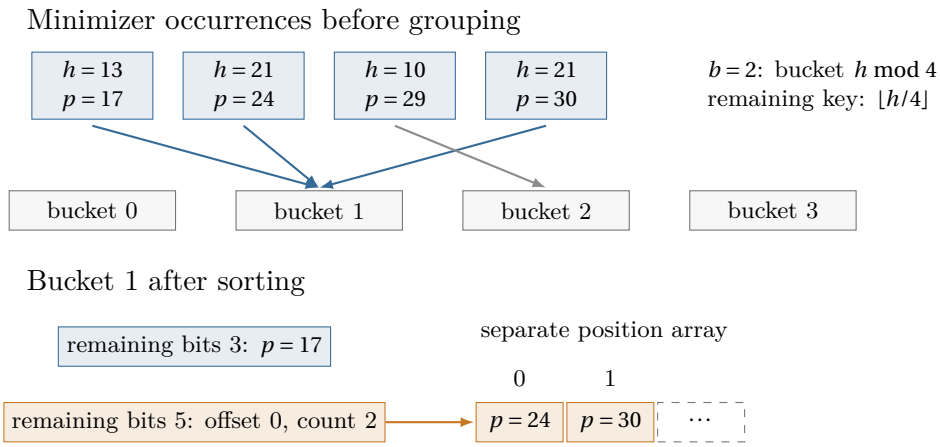


Figure 4.1: A simplified minimap2 bucket. Hashes 13 and 21 have the same lowest two bits, but different remaining keys. The two occurrences of hash 21 become adjacent in the bucket's position array. Concept based on the minimap2 implementation and format discussion.

The `.mmi` writer saves parameters, reference names and lengths, bucket position arrays, hash entries, and optionally packed reference symbols. The loader allocates the bucket structures and reconstructs one complete index part. A large reference can be split into several parts, so "complete" here does not require every part to be resident at once.

Minimizer bucket construction
`worker_post 226`

Writing and loading an index part
`mm_idx_dump, mm_idx_load 475`

[3] Li, "Minimap2: pairwise alignment for nucleotide sequences"; Li, minimap2.

[4] minimap2 contributors, How to interpret the mmi index file? Issue 820; Lanius, parse_mmi.py: Python parser and lookup functions for minimap2 indexes.

This is a useful arrangement for a mapper whose caller chooses a reference file. BioSeqDB keeps the same useful key-to-occurrences idea, but adds selective database reads and source-version checks. These are concrete differences in responsibility, not a measured speed comparison.

4.2.2. MMseqs2: words point back to target sequences

The CPU MMseqs2 prefilter uses an inverted index: a word points to the target sequences and positions containing it. Its 2017 description uses spaced k-mers, words selected through a fixed pattern that skips some positions. Query processing also generates similar words according to protein substitution scores, instead of requiring every matched word to be identical.[5]

The lookup has three steps:

1. Take a spaced word from the query and generate similar words.
2. Read their posting lists, whose entries are pairs (target identifier, target position).
3. Find successive matches to one target on the same diagonal, defined as target position minus query position. These matches suggest an ungapped candidate region for further scoring.

The current ordinary-sequence builder counts entries for each encoded word, forms cumulative offsets, and fills one contiguous entry array. Looking up code c then reads entries in $[offset[c], offset[c+1])$. Offsets count entries, not bytes. The offset table reserves a slot for every possible encoded word, including absent words. This makes a list easy to find, but the table's size grows with the alphabet and word length. The implementation keeps the first position per distinct word code in each target sequence; it does not promise every repeated occurrence within that sequence. BioSeqDB's minimizer indexes instead retain distinct indexed minimizer occurrences, including occurrences at different positions in one target. These positions can support different alignments from the same stored row.

Count and fill the MMseqs2 index
IndexBuilder::fillDatabase 56

Read a word's entry range
getDBSeqList 178

Keep distinct word codes within one target
addSequence 350

MMseqs2's later tools demonstrate why the distinction matters. Linclust selects a limited number of low-hash words per sequence and sorts a table containing the word, sequence identifier, and position. Equal words bring candidate sequences together for clustering.[6] This whole-sequence sampling differs from choosing a minimizer in each sliding window. The interactive server precomputes its CPU index and other lookup structures, then uses memory mapping so repeated searches can benefit from the operating system's page cache.[7]

The same software family also serves taxonomy searches, where sequence hits must remain connected to taxonomic identifiers and reference metadata.[8] Its GPU search follows another route: it scans sequence residues with gapless dynamic programming, then performs selected

[5] Steinegger and Söding, "MMseqs2: sensitive protein sequence searching for the analysis of massive data sets"; Steinegger and Söding, "MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets."

[6] Steinegger and Söding, "Clustering huge protein sequence sets in linear time."

[7] Milot Mirdita, Steinegger, and Söding, "MMseqs2 desktop and local web server app for fast, interactive sequence searches."

[8] M. Mirdita et al., "Fast and sensitive taxonomic assignment to metagenomic contigs."

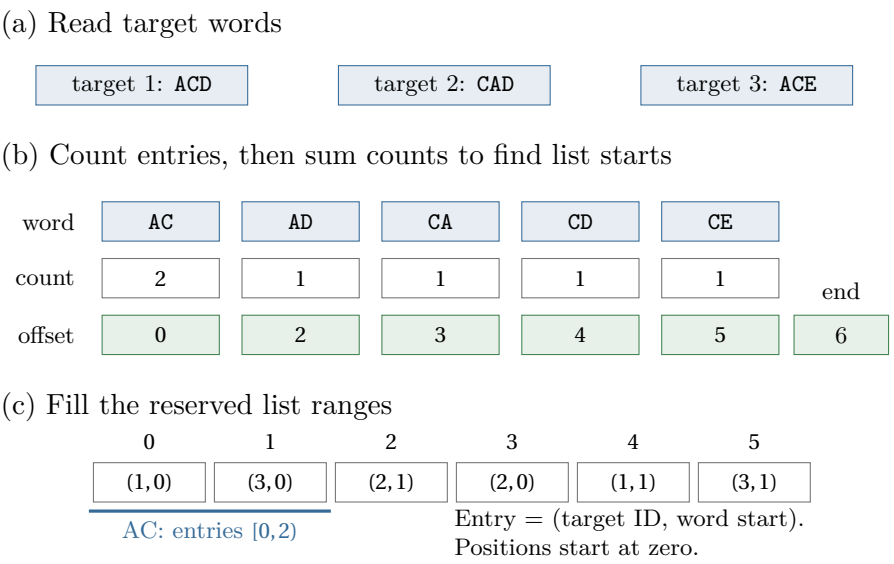


Figure 4.2: A small example of MMseqs2's count-and-fill construction, using contiguous two-letter words for clarity. The real CPU search uses its configured spaced-word pattern and larger words. Counts reserve adjacent list ranges, and a second pass fills them. AC occurs in targets 1 and 3. Empty code slots are omitted from the drawing.

gapped alignments. It can keep database batches on the GPU or stream them as needed.[9] Thus a persistent word index, a sampled clustering table, and resident sequence batches solve different problems. BioSeqDB's persistent minimizer indexes serve its seed-and-chain alignment path.

4.3. The same minimizers in two orders

BioSeqDB supports minimizers per fragment (MPF) and minimizers per sequence (MPS). MPF groups entries by source interval; MPS groups them by hash across one logical sequence. Both preserve positions needed to form anchors. Figure 4.3 shows the same six occurrences in the two organizations. Here a minimizer position is its last-symbol coordinate, using zero-based positions.

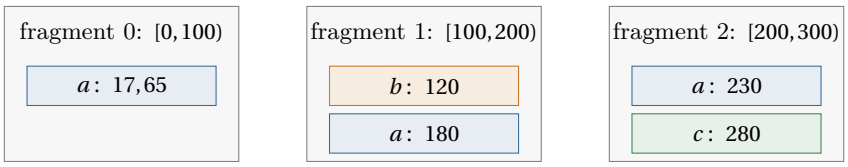
4.3.1. MPF follows source intervals

An MPF lookup has a hash table or ordered hash controls and a separate array of positions. A control records the hash and the location and length of its posting list. Local construction avoids sorting the complete logical sequence at once. A target query can read or assemble the necessary fragment objects, then use their positions to construct anchors.

The ordinary MPF builder uses fixed 32,768-symbol cores. A core is the interval whose occurrences a construction task owns. A generation is a separately identified index build, which becomes usable only after completion and publication. The optional grouped generation builder groups positions according to physical source fragment capacity, while collecting them through cores of at most 32,768 symbols. These construction units should not be confused with a PostgreSQL page or a compressed column chunk. Both builders need symbols

[9] Kallenborn et al., "GPU-accelerated homology search with MMseqs2."

(a) MPF: keep each source interval together



(b) MPS: put equal hashes together across the sequence

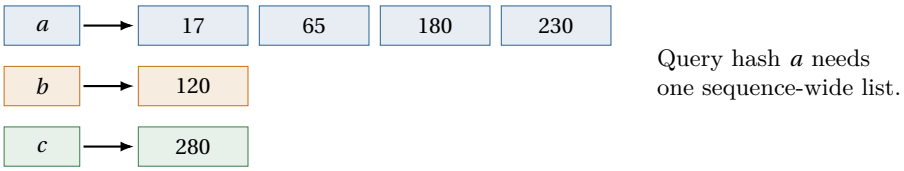


Figure 4.3: Changing the grouping changes which stored units a query needs. Only the minimizer hashes a, b, c are shown. All positions are shown in global sequence coordinates to make the two panels comparable; MPF storage can encode them relative to a fragment and rebase them when read. The 100-symbol intervals are illustrative.

around their boundaries so that a minimizer near an edge is chosen from a complete window. Section 4.4 explains this overlap.

The grouped MPF layout stores hashes, list offsets, counts, and positions in separate arrays, also called a structure of arrays (SoA). This gives the reader explicit ranges to check and allows a prepared lookup to use arrays without recreating a C++ hash table. An additional read-only index file (Section 4.6.2) contains a sequence-wide hash-to-positions section built during publication. It uses membership bits and compact list records, explained in Section 4.6.2. The lookup setting chooses this section or the fragment controls in the same file. Building the section moves the merge of equal hashes across fragments into index construction. Grouped layouts and their readers are optional, rather than implied merely by selecting the MPF index name.

4.3.2. MPS follows hash ranges

MPS combines the sequence's minimizers into hash order. Its directory maps hash ranges to stored data. A query can search this directory before loading positions. The current selective layout separates three streams:

- the directory identifies the hash ranges and their controls;
- controls give hashes, exact occurrence counts, and posting locations;
- postings give the sequence positions and strands.

All indexed occurrences remain stored. A query's occurrence threshold controls which posting lists are read; it does not remove them permanently during construction.

MPS spends more work on sorting and merging so later queries can skip unrelated hash ranges. MPF keeps its base representation closer to source intervals. The current default active minimizer index is MPF. MPS is an explicit choice for workloads that justify a sequence-wide build. The registered persistent minimizer profile uses $k = 15, w = 10$; the selective MPS

stream route additionally requires the supported DNA4 and hash-layout identity. Not every minimizer setting for a lookup built during the query has a corresponding persistent reader.

4.3.3. A smaller index answers symbol counts

Not every sequence index stores seed positions. The `symbol_counts` index keeps a histogram for one sequence: one 64-bit counter for each encoded symbol rank. For DNA4, the counters are A, C, G, T. The sequence `AACGTA` therefore gives $(3, 1, 1, 1)$, occupying 32 counter bytes before row and value metadata.

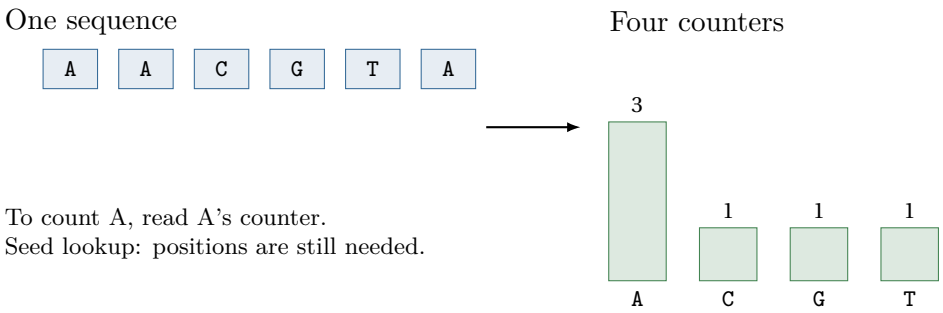


Figure 4.4: A histogram preserves sequence composition but loses position and order. It can answer a count directly, but cannot locate an alignment seed.

Listing 4.1: Request a symbol count for each stored sequence.

```
SELECT id, bioseq.symbol_count(seq, 'A') AS a_count
FROM reads;
```

For a supported scan projection on a typed sequence column and a constant letter, the executor can read the current row's saved histogram and supply the selected counter. If the eligible histogram is unavailable, it counts the lazy sequence's symbols. This index avoids a full sequence read for such projections. It is not an implemented rule for discarding alignment candidates, and it does not preserve enough information to answer position queries. The histogram is stored in an internal relation keyed by the parent's internal row number.

4.4. Build a large index with bounded work buffers

A simple MPS builder could collect every $(hash, position, strand)$ record and sort the complete vector. Its memory use would then grow with the number of target minimizers. BioSeqDB instead sorts smaller runs: ordered groups of records that can be merged without holding their complete contents in memory.

The source scan must first handle boundaries correctly. A halo is the extra context read on either side of a core. For the current grouped builder, a halo of $k + w - 2$ symbols covers the span needed around a minimizer window. An occurrence belongs to the core containing its last-symbol coordinate. Thus overlap supplies context without giving the same occurrence to two owners.

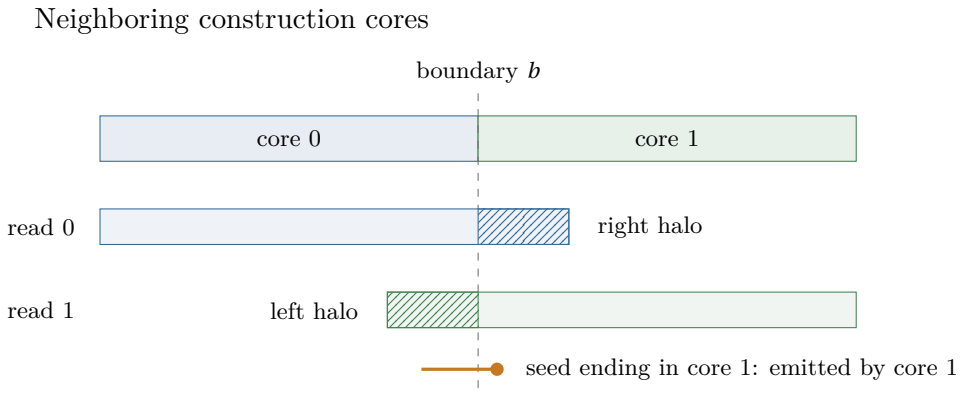


Figure 4.5: The readers overlap, but their output ownership does not. The orange seed crosses the boundary; its last position belongs to core 1. The drawing shows only the internal halos, with lengths exaggerated.

MPS turns the resulting global records into a sequence-wide order:

1. Fill one work buffer, sort it by hash, then position and strand, and remove duplicate records.
2. Write the sorted run to PostgreSQL temporary-file storage, then reuse the work buffer.
3. Merge a bounded number of runs at a time. Reading a small prefix from each input is enough to choose the next output record.
4. Continue merging until one ordered stream remains. Remove duplicates across runs as well.
5. Read that stream to count each hash's occurrences, then replay it to encode controls and positions with the correct counts.

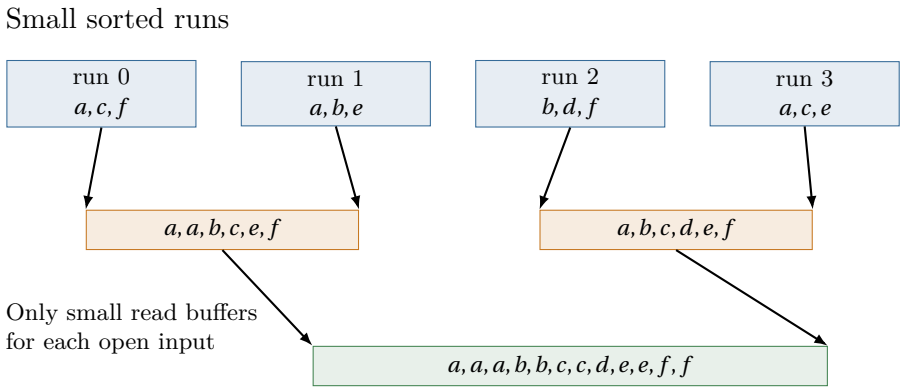


Figure 4.6: External merging bounds the number of simultaneously open input runs. Equal letters represent equal hashes with different positions, so their occurrences are preserved. Only duplicate full records are removed.

The spill implementation uses PostgreSQL's `BufFile` temporary-file interface, rather than giving each run an unmanaged permanent file. The number of merge inputs is limited independently of the total run count. One unusually frequent hash can also have a posting list larger than a work buffer. Its positions are streamed through bounded buffers, so one long list does not require one equally large vector. Temporary disk use can still grow with the input and the merge passes. A memory bound does not imply constant disk space or constant build time.

Construction writes the complete payload before its final navigation metadata becomes usable. In the sequence-wide stream layout, directory entries may only refer to finished controls and postings. Publication of the new generation is a separate final step, described in Section 4.6.1.

4.5. Read counts before large position lists

Suppose the query contains hashes 12, 35, and 78. Reading every target posting would waste work on keys that cannot produce an anchor for this query. It can also waste work on a highly repeated key that the occurrence filter will reject. The selective MPS format, called Format 6, separates the information needed to make these two decisions.

The reader performs four steps:

1. Read and check the complete directory, then find hash ranges intersecting the distinct query hashes.
2. Read the selected controls to find each key's exact global occurrence count.
3. Apply the occurrence threshold before requesting positions.
4. Read accepted posting ranges and form the lookup needed by the alignment batch.

"Global" here means over the indexed target sequence. Looking at one loaded piece alone could underestimate repetition and wrongly admit a frequent key. The stored compact counts therefore have additional exact-count information when their small field cannot hold the value.

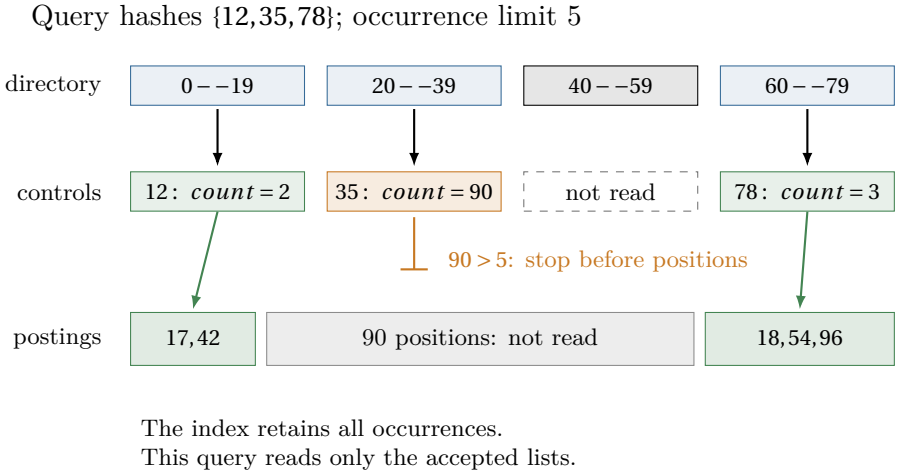


Figure 4.7: Two decisions save different reads. All directory entries are read; blue ranges select the controls. Counts reject a frequent hash before its long list is loaded. Positions and thresholds are illustrative.

Adjacent accepted ranges can be read together through a bounded I/O buffer. The PostgreSQL representation stores payloads in heap rows, so a requested range may fetch a complete row containing some additional bytes. Selective logical ranges therefore reduce unnecessary reads without making the physical read granularity equal to one posting.

The selective MPS reader prepares a lookup for the complete stored target identified by its locator. This reader does not support transformed targets or arbitrary SQL slices. Other index readers can clip and rebase positions for supported intervals. The conditions for using each reader remain separate from the lazy sequence interface itself.

4.6. A saved index needs a source version

The same table and SQL primary key can survive an update that changes a sequence. An index must therefore identify more than its owner by name. The versioned index metadata binds the payload to a source epoch and to the rules that produced its minimizers. Invalidation changes the old epoch's catalog state. A replacement source receives a new epoch.

The important checks have simple reasons:

- Source identity. The relation, column, source locator, epoch, and generation must name the intended source version.
- Seed meaning. The alphabet, k, w, hash scheme, and scheme version must agree with the query. Equal parameter names alone do not make different hash schemes compatible.
- Position meaning. Width and coordinate encoding must represent the target without truncation, including its strand bits.
- Array meaning. The reader must recognize the format and every addressed range. A device reader also needs a compatible array interface.

Lengths, offsets, and checksums help detect damaged or misaddressed data. Checks happen before the reader forms a span or allocates a destination from an untrusted stored count. A checksum is a corruption check; it does not replace source-version or algorithm checks. Ordinary serialized minimizer formats record layout revisions and array sizes, but do not store a source epoch or hash-scheme identifier. Generation readers also check the stored source version and minimizer hash scheme.

4.6.1. Publish complete generations

A small descriptor is the record that identifies a generation and its payload. Building a new generation does not require changing the old payload in place:

1. Allocate a new generation and write its payload.
2. Check the complete layout and its source and algorithm identities.
3. Replace the catalog descriptor transactionally, after the payload is ready.

The descriptor update uses PostgreSQL's tuple visibility. An older snapshot can continue to see its older descriptor; a later eligible snapshot can see the replacement. Failure before publication must not make a partial build the current index.

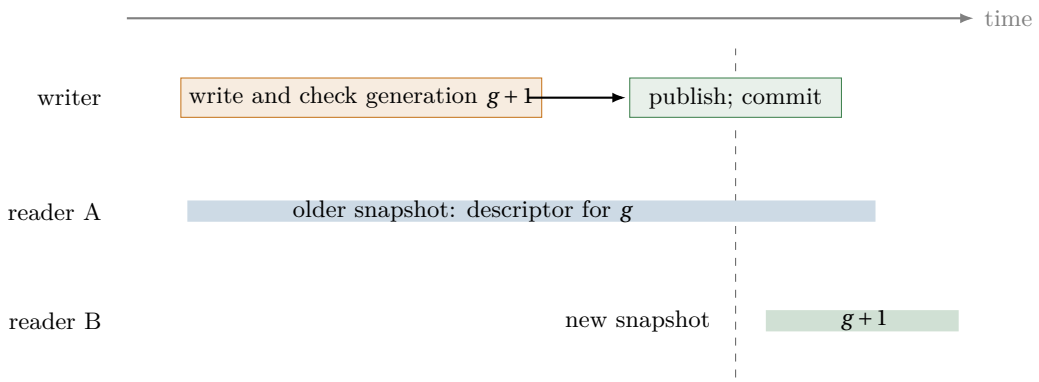


Figure 4.8: Versioned publication with two snapshots. Reader A resolves the old descriptor and retains its read resources. Reader B starts after commit and can resolve the new one. Physical reclamation is a separate operation.

A snapshot that sees a parent sequence's deletion must also reject its old source epoch. These changes belong to the same transaction. Updating a sequence similarly invalidates the old source for newer snapshots and appends a replacement. Neither operation means that every fragment and index byte is immediately erased. An old reader may still need the former version, and the stored objects need their own cleanup policy. The current MPS build path does not automatically delete replaced payloads; reclaiming them requires separate cleanup.

4.6.2. From stored index rows to a CPU reader file

The optional grouped MPF path keeps two stored copies of its index data. It first writes index rows in PostgreSQL, then creates a file for CPU reads. The motivation is repeated searches over a large target. Saving minimizers avoids computing them again, but a reader may still need to sort positions and rebuild a lookup structure. Grouped arrays make the saved layout usable directly. The additional file gives CPU workers stable array ranges without extracting heap tuples or copying the full payload into each reader. To understand why, distinguish three objects:

- The sequence. The original packed symbols remain in the sequence-storage relations from the previous chapter.
- The index relation. Separate heap rows hold the computed minimizer controls and positions, split into byte chunks, with generation metadata. This is already an index, not another copy of the sequence.
- The reader file. This file contains the index payload bytes in consecutive ranges, their directory and metadata, and an additional hash-to-positions section. We call this fixed layout a reader image. Once published, the image is read-only.

The conversion copies the encoded index payload out of the rows. It does not copy PostgreSQL pages or recompute the minimizers. Heap pages contain page headers, row headers and other fields around each stored value.[10] Mapping the heap file would still leave

[10] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Database Page Layout.

those fields between payload chunks. It would not expose one contiguous index array. The reader image removes that obstacle once during index construction.

Construction happens once per generation. The current relation-backed builder takes these steps:

1. Build the index rows. Compute the minimizers and save a new generation's payload, directory and metadata in the index relation.

2. Write the reader file. Reread and check that generation's rows in index order. Append their payload bytes to a private construction file. Add the directory, metadata and hash-to-positions section, then finish and check the file header.

3. Finish the file before publishing the index. Make the file read-only, synchronize it, give it a new final name without replacing an existing file, and remove the temporary name. Then update the database descriptor and active-generation record in the transaction. Other transactions can select the new generation after commit.

The index relation remains after this export. The extra copy costs disk space and a complete payload reread during construction. The hash-to-positions section also adds construction work and bytes. In the current writer this section is always built; the lookup setting chooses how a later query reads the image. Figure 4.9 shows generation g and its completed file F_g.

Direct lookup uses a hash as a bit address. The extra section combines equal hashes from all fragments of one target sequence. It has three kinds of data:

- Membership bits. Bit $B[h]$ is one when hash h has a position list stored in this section. The hash is the bit's address, so it need not be stored again as a key beside that list.

- List records. Records follow increasing hash order. The number of set bits before h, called its rank, gives its record number. Saved counts at each 256-hash boundary make this calculation local: add the saved count to the number of set bits before h in that group. A record gives the list's start, total count, and number of strand-zero positions. Strand-zero positions precede strand-one positions.

- Positions. The list start is an entry number in a shared position array, not a file-byte offset. Positions include a strand bit. When a list has one 32-bit position, the record stores it directly.

In Figure 4.9, hash 5 has one earlier set bit. Its rank is therefore 1. Record 1 gives start 2 and count 2, so the reader uses posting entries $[2,4)$, which contain positions 40 and 230. This is direct addressing followed by a short bit count, with no hash-table collision search or reconstruction of a target-wide C++ map.

An unset bit means no list is retained in this extra section. It can mean either that the hash is absent or that its sequence-wide count exceeds the saved cap, currently 65,534. The original fragment payload remains stored. A query can impose a lower positive occurrence limit, but cannot exceed the saved cap on this direct route. The $k = 15$ profile used below has 2^{30} possible hash values. Its membership bitmap occupies 128 MiB and its saved counts occupy 16 MiB, before list records and positions. This fixed space cost favors large targets. It is a file-size cost, not a requirement to keep every mapped page in RAM at once.

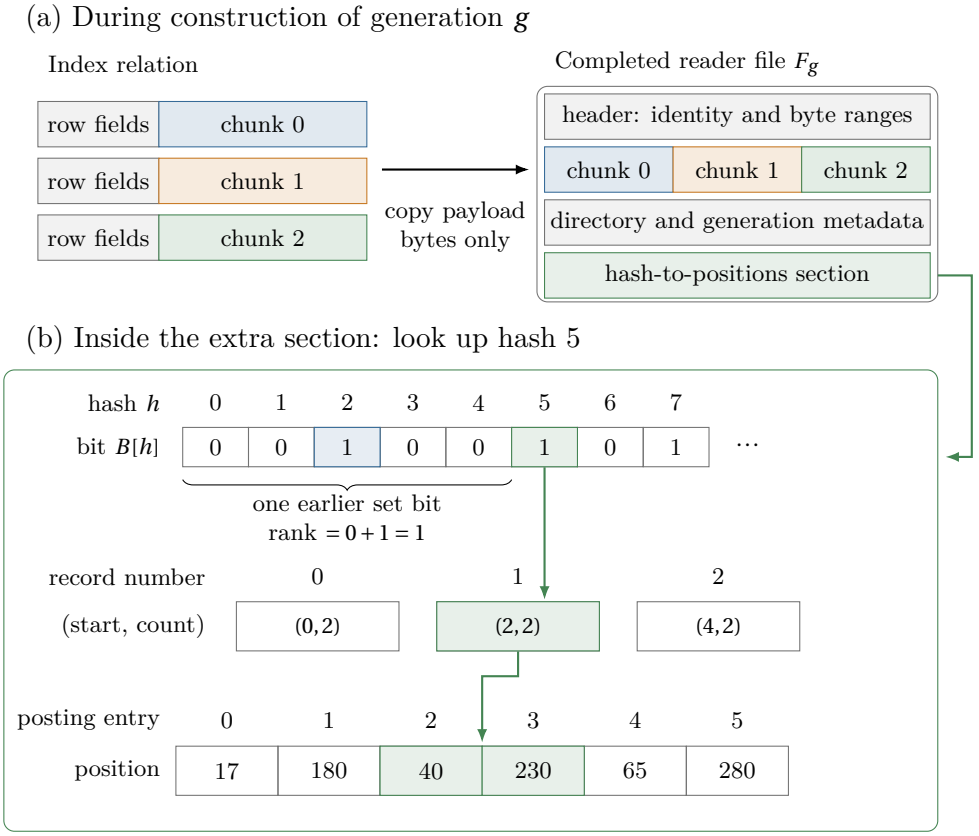


Figure 4.9: Building the reader image and using its direct lookup. The upper panel copies chunk payloads and adds a lookup across the target's fragments. The lower panel uses illustrative hashes 2, 5 and 7, each with two positions. Only the bitmap's first eight bits are shown. Its first 256-hash group has saved count zero. Hash 5 selects bit 5, whose rank selects record 1; that record selects posting entries 2 and 3. Green marks this lookup. Positions are decoded sequence coordinates; all strands are zero, and the record's strand count is omitted here.

Queries open the file rather than exporting it again. The CPU reader first validates the source and reads the generation descriptor under the query snapshot. It opens the matching file read-only, checks its identity and size, and maps it with `mmap`. Mapping gives the process an address range through which it can read the file. After header and range checks, the reader uses views of the mapped index arrays. It still allocates metadata and query work buffers, but does not copy the complete index payload into a new array. One prepared reader can serve several query sequences in a batch. A later prepared reader can reopen the same file.

Algorithm 3 separates the two operations. Here s is the complete, unchanged stored target sequence, R_g denotes its index rows for generation g, and σ is the query snapshot. The named operations are pseudocode, not an API. File and database errors stop the operation; registered cleanup releases partly opened resources. The build runs within the caller's transaction.

Algorithm 3 Build an MPF reader image once; open it for later queries.

```
1:  procedure BuildMappedMPF(s)
2:      g ← a new generation number
3:      R_g ← BuildAndStoreMPF(s, g)
4:      F ← CreatePrivateFile
5:      for all payload chunks c of R_g, in stored index order do
6:          CheckChunk(c)
7:          AppendPayload(F, c)
8:      AddMetadataAndDirectLookup(F, R_g)
9:      F_g ← CheckAndPublishReadOnlyFile(F, g)
10:     UpdateDescriptorAndActiveGeneration(s, g, F_g)
11:     return g  the caller commits the transaction

12: procedure OpenMappedMPF(s, σ)
13:     D ← ReadAndCheckVisibleDescriptor(s, σ)
14:     F_g ← OpenAndCheckFileReadOnly(D)  check identity and size
15:     M ← MapFileReadOnly(F_g)
16:     CheckHeaderAndRanges(M, D)
17:     return a reader using M while the read session owns the mapping and file
```

Which SQL queries use the image? The public function `bioseq.update_index` builds the file when grouped MPF construction is enabled. Later SQL can reach the mapped CPU reader through an eligible `BioseqBatchJoin`. This custom execution node handles an ordinary inner join whose `mmsearch` call compares a stored target column with a query column. The first argument names the target; the second supplies the queries. In the mapped MPF case, the executor:

1. Keeps the query rows. These can include lazy sequence values; retaining a row need not load all its symbols.

2. Takes one target row. Its stored locator identifies the target whose compatible index generation must be opened.

3. Opens the existing image. The reader validates and maps the file built earlier by `update_index`. Search does not repeat that construction.

4. Searches the retained queries. Query minimizer hashes find target positions in the mapped arrays. Those positions supply anchors for chaining and alignment. The same reader serves all query groups for this target.

5. Moves to the next target. The target's reader is released, while its file stays available for later SQL searches.

A query input with one row is allowed. Two aliases of the same table can also supply the two inputs. Thus "batch" describes how the executor reuses target preparation, not a separate SQL interface or a requirement to search two different tables.

The mapped branch currently requires a complete, unchanged stored target, a compatible CPU profile and generation, and `index_read_requirement = 2`: require a compatible stored index. An ordinary single-table search such as `WHERE mmsearch(r.seq, 'ACGT...')` does not use this mapped reader. Requiring an index does not turn that scan into the two-input join; an unsupported required read fails. Even the `BioseqBatchJoin` name in `EXPLAIN` is insufficient, because a separate variant for transformed operands has the same name and rejects this requirement. The example below uses plain columns and returns identifiers, leaving the stored target in the form needed by the mapped reader.

Both approaches still read the input tables. Figure 4.10 compares two ways to prepare the target lookup inside a CPU batch join. In both, `queries` supplies query rows and `refs` supplies target rows. The difference is the source of the target's minimizer lookup:

- Ordinary MPF: read and decode the stored fragment indexes, then assemble a private lookup for this join, containing the hashes needed by the retained queries. Reuse that lookup across the target's query groups. A cache of stored byte chunks can help the reads, but this join still assembles its own lookup.

- Grouped MPF: open the saved image and use views of its arrays. This avoids assembling the private target lookup above.

Both still fetch sequence symbols needed for base alignment from sequence storage. The mapped image supplies the seed index. If no compatible stored index is used, a third choice is to compute a transient lookup from the target sequence during the query.

The figure assumes that the two indexes were built beforehand: ordinary construction with grouped generation disabled on the left, and grouped construction enabled on the right. The right-hand query must also select the supported mapped-reader path. Changing the construction setting alone does not switch the join's reader.

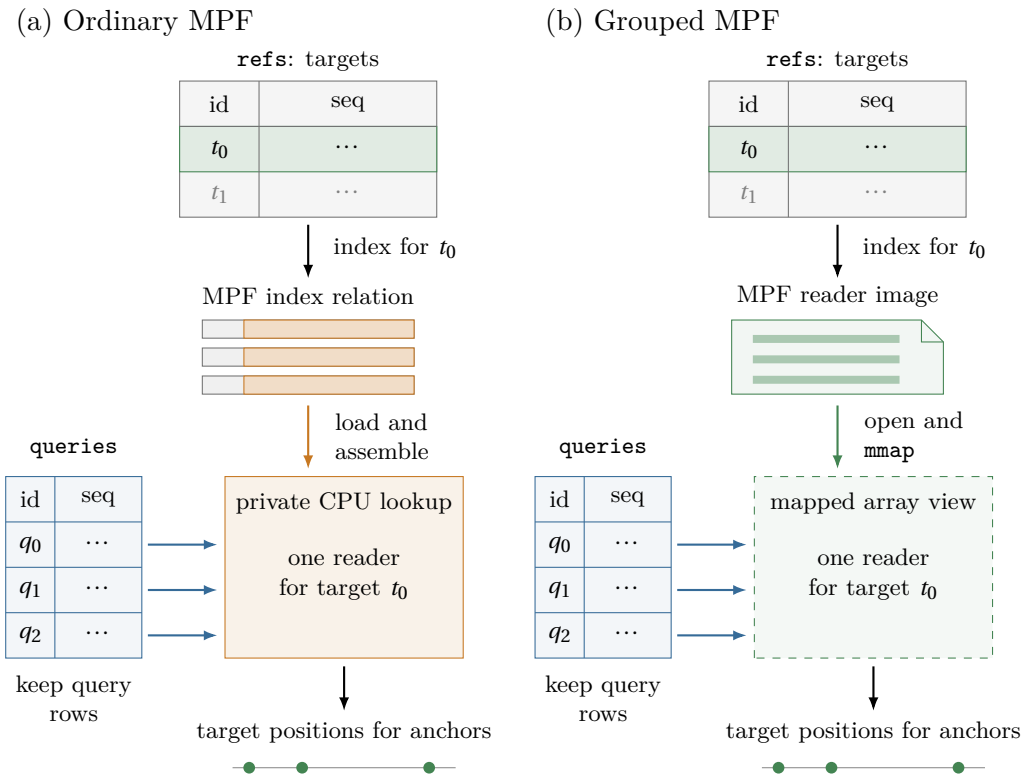


Figure 4.10: The same SQL inputs, with different target-index preparation. Target t_0 is selected while t_1 waits. The blue arrows represent query hash lookups through one reused reader. On the left, stored fragment indexes become a private CPU lookup. On the right, the dashed view refers to the file's existing arrays; it is not another complete payload copy. Both indexes were built before the query. Generation checks and later sequence-symbol reads are omitted.

Construction settings and read settings have different jobs. A GUC is a PostgreSQL configuration setting changed with `SET`. Three settings control the construction choices used here:

- `bioseq.aligner.grouped_soa_generation.enabled`
  Default: `off`. Makes `update_index` build a grouped generation. For MPF this writes both relation rows and the reader image. For MPS it writes the separate relation streams. Turning the setting on does not itself build an index, convert an older layout, or select a query reader. Turning it off later does not remove a completed generation.

- `bioseq.index.async_persistence.enabled`
  Default: `on`. Permits eligible background persistence work. It does not make an explicit `update_index` call asynchronous.

- `bioseq.index.recompute_async_persistence.enabled`
  Default: `off`. Additionally permits a query that recomputes a lookup to request persistence for later queries. It needs the preceding setting too. Turning it off does not stop a query from building a transient lookup to answer the current search.

The current deferred implementation rebuilds eligible ordinary MPS indexes. It stages source identifiers and parameters, then attempts to submit work when the transaction com-

mits. Abort discards the request. A background worker reopens the source and checks its policy before rebuilding it. Queue availability and these checks can prevent persistence, so enabling both settings does not promise that every recomputed lookup will be saved. This mechanism does not build the grouped MPF image. Explicit grouped construction rejects either asynchronous-persistence setting being on; both must therefore be off in the example.

Listing 4.2 shows the public calls. It assumes committed DNA4 sequences with 32-bit packed units in a new permanent `refs(id,seq)` table using the Bioseq table access method, and query sequences with the same alphabet and packing in `queries(id,seq)`. Load `refs` with `bioseq.index.insert_maintenance.enabled = off`. This setting disables insertion-time index maintenance, including symbol counts; it avoids creating an ordinary MPF relation during this load. It does not remove one that already exists. Grouped construction is not an automatic conversion of that older layout. Other settings not shown retain their ordinary defaults.

The remaining settings select compatible algorithms and execution:

- `bioseq.aligner.minimizer.algorithm = 'two_stack_nthash'`
  Selects the SIMD implementation with the minimap2 integer hash listed in Table ??. Construction and search must agree on that implementation. The accompanying `kmer_size` and `window_size` settings set the query's k and w. The current grouped builder uses the fixed $k = 15, w = 10$ profile, so the query must match it.

- `bioseq.aligner.chaining.algorithm = 'simd_minimap2'`
  Selects CPU chaining. Setting `bioseq.aligner.base_alignment.algorithm` to `'block_aligner'` selects base alignment. These choices govern later alignment stages; they do not change the stored minimizer arrays.

- `bioseq.aligner.vulkan.enabled = off`
  Marks Vulkan as unavailable to the aligner. Together with the CPU algorithms and the other default settings, this gives a CPU example. It does not rewrite any GPU algorithm choices previously made in the session.

- `max_parallel_workers_per_gather = 0`
  Prevents PostgreSQL parallel query workers.[11] The current batch path needs an eligible serial query input. BioSeqDB can still use its own CPU workers and SIMD instructions.

- `bioseq.index.sequence_read.enabled = on`
  Allows stored reads for permissive searches. The per-call requirement 2 is stronger: it forces stored reading and reports a missing compatible index as an error. It is an `aligner_options` argument, not a GUC.

- `bioseq.aligner.fragment_minimizer.lookup_route`
  Selects the grouped MPF lookup method. The default `control_block_scan` reads fragment controls. The example selects `direct_compact_hash_lookup`, which uses the bitmap and records in Figure 4.9. Both methods map the same image; changing between them does not require rebuilding it.

[11] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Resource Consumption.

Listing 4.2: Explicitly build and use the mapped MPF path.

```
BEGIN;
SET max_parallel_workers_per_gather = 0;
SET bioseq.aligner.vulkan.enabled = off;
SET bioseq.aligner.minimizer.algorithm = 'two_stack_nthash';
SET bioseq.aligner.minimizer.kmer_size = 15;
SET bioseq.aligner.minimizer.window_size = 10;
SET bioseq.aligner.chaining.algorithm = 'simd_minimap2';
SET bioseq.aligner.base_alignment.algorithm = 'block_aligner';
SET bioseq.index.async_persistence.enabled = off;
SET bioseq.index.recompute_async_persistence.enabled = off;
SET bioseq.aligner.grouped_soa_generation.enabled = on;

SELECT bioseq.update_index('refs'::regclass,
                           'minimizers_per_fragment');
COMMIT;

SET bioseq.index.sequence_read.enabled = on;
SET bioseq.aligner.fragment_minimizer.lookup_route =
  'direct_compact_hash_lookup';
SELECT bioseq.set_active_read_index('refs'::regclass,
                                    'minimizers_per_fragment');
SELECT r.id, q.id
FROM refs AS r JOIN queries AS q
  ON mmsearch(r.seq, q.seq,
              aligner_options(index_read_requirement => 2));
```

Plain `SET` survives a successful `COMMIT`, so the later query keeps the chosen algorithms and parameters. `SET LOCAL`, used in the MPS example below, instead restores the previous session value when the transaction ends.[12] The function `set_active_read_index` records which index the column's searches should use; it is a catalog change, not a GUC or an index build.

Grouped MPS reads its relation-backed streams. The grouped MPF GPU path reads the complete relation payload into device arrays. Neither uses this CPU reader file.

The mapped path is implemented and callable through public SQL, but remains an optional, experimental path: construction is disabled by default, supported query shapes and profiles are restricted, and automatic removal of retired images and complete recovery support remain unfinished.

RAM is an alternative, not the opposite of a mapped file. An index kept in RAM can be fast while it remains available. BioSeqDB's ordinary index caches already retain prepared CPU objects. However, a new backend may have to read and reconstruct its own copy, and an index may be larger than the memory available for that copy.

A mapped file still uses RAM: the operating system keeps file pages in its page cache. Read-only mappings in different processes can use the same cached file pages. Pages can be loaded as needed and reclaimed under memory pressure without making each backend retain the whole index in a private array. This is the performance reason for the extra representation: avoid repeated tuple extraction and full-payload copies at query preparation. It is not a guarantee of faster search. A cold query can wait for page reads, and a broad

[12] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: SET.

lookup can touch much of the file. Building the image also costs an extra write, checks and synchronization. A warm in-memory index remains a useful comparison.

Releasing a reader and deleting its file are different steps. The file has the following lifecycle:

- During construction: a private temporary name identifies the unfinished file. Publication installs a distinct final name.
- On transaction abort: registered file actions attempt to remove the transaction's unfinished or newly published files after checking their identity. On commit, the completed file is kept.
- When a reader finishes: its PostgreSQL resource owner unmaps the address range and closes the file descriptor. The file remains available for another reader. Error cleanup releases these resources too.
- After replacement or source deletion: visibility and source checks decide whether a reader may use the old generation. The current query and rebuild paths do not automatically reclaim its retired image.

A missing or damaged image causes the required mapped read to fail. The query does not recreate it from the relation. An explicit rebuild produces a new generation and image. The file is derived data outside ordinary relation WAL. Synchronizing it does not establish a recovery protocol or ensure that a backup contains a matching file and database descriptor. Its relation-backed generation also uses unlogged payload relations. Crash recovery, backup and removal of orphaned files therefore need explicit support rather than an assumption that PostgreSQL manages the image like a heap relation.

4.7. Choose reuse without changing the search meaning

Update policy decides when construction happens. BioSeqDB has two policies:

Table 4.1: Maintenance policy for one registered index name.

Policy	When work runs	Useful case
`UPDATE_ON_INSERT`	Build the new row's index while inserting it; other rows are not rebuilt	Small additions that should be indexed immediately
`UPDATE_MANUALLY`	Skip that index's insertion-time build; `update_index` requests an explicit rebuild	Load rows first and schedule index construction separately

For example, ordinary MPF maintenance can index one new reference without rebuilding the table's older references. The following uses the storage chapter's DNA4 column with 32-bit packed units. It assumes ordinary defaults: insertion maintenance enabled and grouped generation construction disabled.

Listing 4.3: Maintain MPF when one new sequence row is inserted.

```
SELECT bioseq.set_index_policy(
  'reads'::regclass, 'minimizers_per_fragment', 'UPDATE_ON_INSERT');

INSERT INTO reads (id, sample_id, seq)
VALUES (101, 7, repeat('ACGT', 2000)::bioseq.seq('dna4', 32));
```

An update appends a replacement row, so the same insertion policy applies to its new sequence. Changing a policy does not build missing indexes for older rows, clear existing payloads, or select the index that later searches use. The SQL policy call applies the chosen index name to all current sequence columns in the given table.

Read policy answers a separate question: a query can allow stored reads, require a compatible stored index, or require a lookup built for the current query. We call the last choice a transient lookup.

The important choices can be stated without listing every format setting:

- Ordinary MPF. MPF is the default minimizer index and uses insertion maintenance by default. Its relation adapters read serialized objects and prepare a runtime lookup.
- Grouped MPF. Optional generation construction and its prepared CPU reader use the explicitly described arrays and immutable image. The grouped setting is disabled by default. This CPU reader requires the complete stored target without slicing or transformation, and a policy that requires a compatible stored index. Load and commit source rows before enabling this construction mode.
- MPS. Maintenance is manual by default, though ordinary MPS also supports opting into `UPDATE_ON_INSERT`. The selective generation form uses an explicit rebuild and the separate streams described above.
- Symbol counts. A small per-sequence histogram is maintained on insertion by default and can supply supported count projections.

For a bulk load, an explicit policy keeps a selected index's construction outside the insertion transaction:

Listing 4.4: Choose manual MPS maintenance before loading source rows.

```
SELECT bioseq.set_index_policy('reference_sequences'::regclass,
  'minimizers_per_sequence', 'UPDATE_MANUALLY');
```

This call changes MPS only. MPF and symbol-count maintenance can still run under their own policies. Manual maintenance does not remove the work needed for a later rebuild; it controls when that work runs.

After loading and committing `reference_sequences`, the next example requests an MPS generation build. The construction settings have the roles explained in Section 4.6.2: grouped arrays are enabled and both background-persistence permissions are disabled. This example starts without an older ordinary MPS index; changing its maintenance policy would not remove one. Here `bioseq.aligner.minimizer.algorithm` is set to `'scalar'`, selecting the scalar CPU minimizer implementation for DNA4 sequences. The build uses the registered $k = 15, w = 10$ profile. `SET LOCAL` restores the prior session settings at transaction end; later searches must still select a compatible minimizer implementation and parameters.

Listing 4.5: Build a sequence-wide minimizer index after loading the source.

```
BEGIN;
SET LOCAL bioseq.index.async_persistence.enabled = off;
SET LOCAL bioseq.index.recompute_async_persistence.enabled = off;
SET LOCAL bioseq.aligner.grouped_soa_generation.enabled = on;
SET LOCAL bioseq.aligner.minimizer.algorithm = 'scalar';

SELECT bioseq.update_index('reference_sequences'::regclass,
                           'minimizers_per_sequence');
COMMIT;
```

If a query requires compatible persistence, an absent or incompatible object is an error. Silently rebuilding it would violate that operational request. A permissive query can instead construct a transient lookup. Requiring a transient lookup bypasses stored-index reuse even when a stored index exists. The result's seed semantics still depend on the selected algorithm and its parameters, not on whether the same lookup was loaded or rebuilt.

The deferred ordinary MPS work described in Section 4.6.2 also respects PostgreSQL ownership. A background worker uses its own transaction and memory; it does not borrow the submitting backend's snapshot, relation pointer, or memory context.

4.7.1. Keep useful bytes while their owners remain valid

There are several kinds of reuse, with different lifetimes:

- Serialized chunks. Ordinary relation readers can use a bounded backend-local cache of stored byte chunks. A read handle may still copy a cached chunk into its own buffer.

- Prepared lookups. Ordinary MPF and MPS paths can retain assembled CPU objects under keys describing the source, index, algorithm, and selected interval. Eviction removes the cache's reference; an active reader can still own the object.

- Mapped pages. The operating system caches touched pages of an immutable image. The grouped generation route does not populate the ordinary serialized-chunk cache.

- Device arrays. A compatible GPU lookup can remain resident across work in a batch. Its allocation and memory allowance remain held while any active handle needs it.

The ordinary chunk cache has both an entry cap and a byte cap: currently 256 entries and 2 GiB. The assembled CPU cache has caps of 32 entries and 4 GiB. These cap retained cache entries, not the entire backend's peak memory. Other live buffers, active references to evicted objects, and device owners can overlap. Memory permits account for significant owned allocations while they remain live. A mapped file's virtual length and the operating system's page-cache residency are separate from that accounting.

The chunk cache marks entries written by the current transaction so abort cleanup can discard them. Relation and index invalidation also remove affected cache entries. Reuse requires the matching source and index identity. Releasing a lookup or a mapping is not the same as deleting the stored index: the next eligible query can reopen its persistent representation.

4.8. What the database integration adds

Like minimap2 and MMseqs2, BioSeqDB reuses seed indexes. It connects this reuse to PostgreSQL sequence values through fragment or hash grouping, exact counts before position reads, bounded construction, and source-version checks. The costs are MPS merging, generation metadata, and old payloads awaiting safe cleanup. Unlogged payloads and mapped images need recovery or regeneration. Reuse still requires a compatible source version, representation, and reader lifetime.

Chapter 5

BioSeqDB architecture

5.1. Connecting sequence processing to the database

Consider a search between stored reference sequences and query sequences. Both tables below have an identifier `id` and a sequence column `seq`. One statement supplies comparisons to the alignment system:

Listing 5.1: A sequence join connects SQL input to the processing core.

```
SELECT r.id AS target_id, q.id AS query_id
FROM refs AS r JOIN queries AS q
  ON mmsearch(r.seq, q.seq);
```

PostgreSQL supplies the rows and owns their visibility. BioSeqDB selects compatible implementations, obtains sequence and index views, schedules the comparisons, and returns matches for SQL projection. This chapter explains how those responsibilities fit together.

A batch groups input comparisons for processing. A wave is a subset admitted together under the memory policy. A task is a unit of work submitted to the task scheduler and can process one or more comparisons. These terms describe how work is grouped, not different alignment algorithms.

The preceding chapters describe the work performed by a sequence search. Minimizers produce sparse seed keys (Section ??), chaining groups matching positions (Section ??), and final alignment compares the selected intervals at base level (Section ??). These stages must also fit into a database execution. The sequence store determines how symbols can be read, persistent indexes retain lookup work, and the custom scan turns matches into SQL rows.

BioSeqDB connects these components through a reusable C++ core and PostgreSQL-specific adapters. The core expresses biological operations over logical sequences and index views. The adapters connect those operations to database resources. This division gives three kinds of code different duties:

- Algorithms consume symbols, positions and scores. Their interfaces do not require a PostgreSQL tuple or relation.
- Adapters construct suitable views and keep their backing resources valid. They also convert returned results into database values.
- PostgreSQL execution controls planning, snapshots, locks and the lifetime of a statement. These mechanisms remain native to PostgreSQL.

The build reflects this division. A shared object-library target owns the PostgreSQL-free modules, and the extension consumes their compiled objects and module interfaces. PostgreSQL-dependent modules are compiled separately. The common code is therefore shared with non-server consumers without recompiling it under PostgreSQL-specific definitions. This is a source and compilation boundary. It does not by itself define a separately distributed library or remove PostgreSQL discovery from the project's build configuration.

TODO

Thing below doesn make sense and I dont know where to place it

Two applications illustrate why composition matters. Masai describes seed search over conceptual trees but chooses concrete index structures for each role and combines the search with banded bit-vector verification.[1] DREAM-Stellar groups query segments by reference region and uses bounded batches with shared sequence ownership before local verification.[2]

5.2. Contracts describe the operations a stage needs

The implementation is built in C++26 mode. It uses templates, concepts and named modules to express these boundaries, although concepts and modules entered the language in C++20. A concept states which operations a type must provide. A template constrained by that concept can accept different representations without requiring inheritance from one common base class.

The following reduced API excerpt shows the contiguous minimizer-lookup contract. It keeps the current operation and return type, while shortening the template parameter names. A lookup receives a hash and returns a borrowed span of encoded positions.

Listing 5.2: The contiguous lookup API constrains the operation visible to a consumer.

```
template<class I>
concept contiguous_minimizer_lookup_index =
    requires(const I& index, std::uint64_t hash) {
        { index.lookup(hash) }
            -> std::same_as<std::span<const std::uint64_t>>;
    };
```

This is one supported lookup shape. Other consumers can receive segmented position spans or canonical anchors prepared for a query. The consumer needs the meaning and lifetime of those positions, rather than the PostgreSQL relation from which they came. In particular, the total occurrence count remains authoritative when a selective reader omits the postings of a high-frequency seed. The storage choice must preserve the pruning decision.

Index construction uses a separate contract. It describes typed chunk reads and writes, enumeration and temporary sorted-run storage. Concrete PostgreSQL types satisfy the contracts for per-fragment chunks, a single per-sequence value, and table-wide chunks. Compile-time assertions check the required interfaces. Separating these storage operations from the query's lookup operation lets each side use the representation suited to its work. The persistence mechanisms are developed in Section 4.1.

[1] Siragusa, Weese, and Reinert, Fast and sensitive read mapping with approximate seeds and multiple backtracking.
[2] Aasna et al., "DREAM-Stellar: parallel and space efficient exact local alignment."

Sequence access follows the same principle. A scanner supplies size and fragment-access operations. Its PostgreSQL implementation reads the fragments described in Section 3.1, while an in-memory implementation exposes caller-owned packed storage. Algorithms normally use the logical views introduced in Section 1.1, so they do not need to repeat either scanner's storage logic. A smaller example appears in column comparison: a result-sink concept asks for result allocation and writable value and null masks. The PostgreSQL sink supplies those buffers using database allocation rules. Character comparison that requires PostgreSQL collation remains in the PostgreSQL adapter.

These are selective abstractions. A minimizer lookup needs occurrence positions, but it does not need an abstract join operator. A sequence reader needs valid source access, but it does not implement an alternative MVCC system. Keeping the database mechanisms in PostgreSQL preserves their existing ownership and visibility rules. The common contracts instead expose the operations needed by the biological stages and their test callers.

5.3. Ownership completes the interface

A successful concept check proves that an expression has a usable type. It does not prove that a returned span remains valid or that its source may be read from another thread. Figure 5.1 separates the operation interface from the resources that make an execution valid.

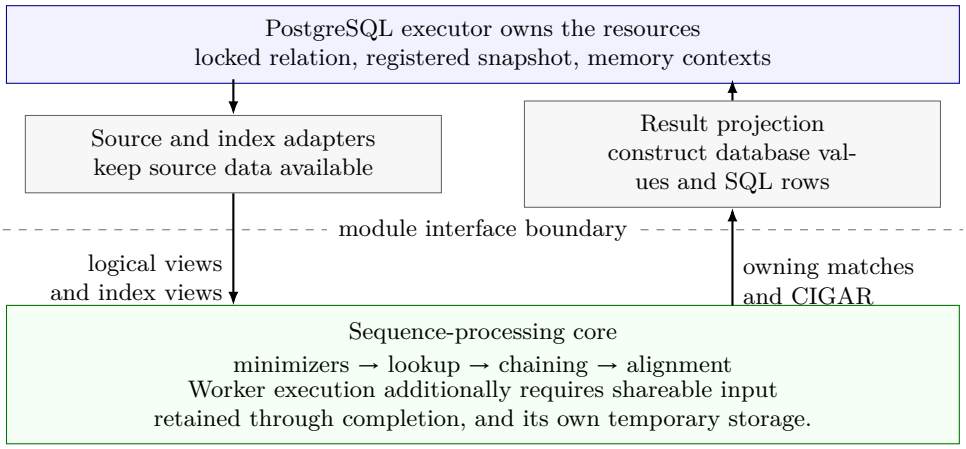


Figure 5.1: Data crosses a narrow interface while the PostgreSQL executor retains database ownership. Arrows show runtime data flow. Static module dependencies point from the adapters to the core contracts. A logical view may still request a PostgreSQL read on the owning backend thread. Worker execution requires an additional check of input sharing and lifetime.

The custom scan from Section 2.1 owns its locked relation and registered snapshot. Read adapters borrow them within the statement's lifetime. A lazy sequence view can therefore be PostgreSQL-free as a C++ interface while still requesting data through a PostgreSQL-backed source. The source rejects calls from the wrong thread before entering PostgreSQL error-handling machinery. The sharing rules in Section 1.6 apply here: worker eligibility and source lifetime are separate requirements. Depending on the route, preparation may copy data into admitted resident storage or retain an immutable mapped index until its readers finish.

Memory also has several owners. The core context separates temporary alignment storage from longer-lived pipeline storage. Numerical worker tasks install a context for their execut-

ing thread, while database buffers remain under PostgreSQL memory contexts. A memory permit accounts for a reservation but does not own the allocation itself. The bytes must be released before that reservation becomes available to another task. Returned matches and their CIGAR operations use owning C++ vectors, which the PostgreSQL adapter can consume without retaining a temporary alignment arena.

Error handling follows the same division. PostgreSQL failures are translated at checked calls so that surrounding C++ owners can unwind normally. Database resource cleanup also has to remain valid on PostgreSQL's own error path. PostgreSQL interrupt checks occur at supported adapter boundaries, including between rows, rather than inside every native numerical loop. Thus a generic function signature alone makes no promise about cancellation latency.

The remaining shared assumptions are concrete. Both sides must agree on alphabet ranks, coordinate conventions and index encodings. A prepared index must describe the source version and minimizer rules expected by the query. The build must contain the selected implementation. These agreements make the stages composable, while the adapters keep each execution within the database's resource and visibility rules.

5.4. Runtime choices and compiled implementations

An alignment request names biological work, but it does not identify one machine-code function. BioSeqDB must also account for the sequence alphabet, packed representation, available indexes, and execution hardware. These decisions select an already compiled implementation and connect the algorithms described earlier to the storage and execution layers.

The logical flow remains sketching, seed lookup, chaining, region selection, and optional base alignment. Sections ??, ??, and ?? explain those algorithms. Here the concern is how the system selects their implementations and supplies compatible inputs.

5.4.1. Turn SQL settings into a request

PostgreSQL configuration parameters, usually called GUCs, provide session defaults. The three settings in Listing 5.3 select minimizer extraction, chaining and base alignment, respectively. Their defaults are `scalar`, `scalar_minimap2` and `block_aligner`. The available CPU minimizer choices are described in Section ?? and Table ??.

A constant `aligner_options` record overrides individual settings for one search. Unspecified or NULL fields inherit the session values. The following example instead selects all three algorithm families through named GUC values: scalar sketching, scalar chaining, and KSW2 base alignment. Both `refs` and `queries` are stored sequence tables with an `id` column and a `seq` column of type `bioseq.seq('dna4',32)`.

Listing 5.3: Select algorithm families before planning a sequence join.

```
BEGIN;
SET LOCAL bioseq.aligner.minimizer.algorithm = 'scalar';
SET LOCAL bioseq.aligner.chaining.algorithm = 'scalar_minimap2';
SET LOCAL bioseq.aligner.base_alignment.algorithm = 'ksw2';
EXPLAIN
SELECT r.id, q.id
FROM refs AS r JOIN queries AS q ON mmsearch(r.seq, q.seq);
COMMIT;
```

An ordinary `BioseqScan` stores the defaults and overrides in its plan. A `BioseqBatchJoin` reads the defaults again when execution starts and reapplies the constant overrides. Changes to the corresponding alignment GUCs invalidate cached plans. The options reference in Section ?? gives the complete mapping. The request also fixes the selected chaining policy: PostgreSQL SIMD chaining uses `bounded_batch_approximate`, whereas the C++ interface additionally permits `exact_visited`. They are distinct policies, as described in Section ??. Likewise, the internal batched KSW2 implementation is not an additional PostgreSQL base-aligner setting.

Required results also determine which stages run. A coordinate-only request can avoid base alignment. A CIGAR or exact edit-distance request requires additional alignment work. Divergence and mapping quality have their own requirements. Thus selecting an aligner does not imply that every search executes it. A prebuilt query sketch also lets execution enter after sketching.

5.4.2. Separate algorithm selection from hardware selection

Figure 5.2 separates the decisions that constrain execution. A requested combination must have an implementation and satisfy its input contract. The system does not treat every combination of algorithm names as interchangeable or supported.

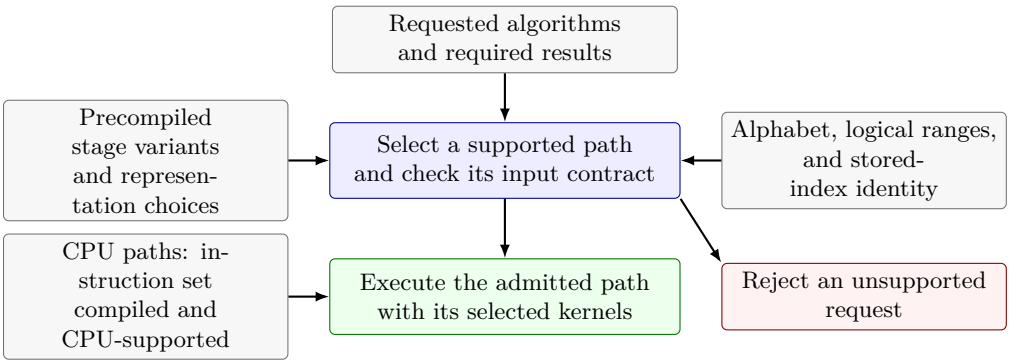


Figure 5.2: Algorithm selection, input compatibility, and CPU target selection have different responsibilities. The diagram shows their dependencies, not a universal order of planner callbacks. GPU execution has additional admission requirements. A runtime failure can still occur after admission.

For CPU kernels, Highway selects an instruction-set target that is both compiled into the extension and supported by the running processor. BioSeqDB binds the resulting kernel table once at the alignment-batch entry and passes it to the selected implementation. It does not repeat CPU feature detection for each seed or choose a different biological algorithm because another instruction set is available. PostgreSQL vectorized tuple processing is a separate execution choice from the SIMD kernels inside an aligner.

Here a specialization is a compiled variant of a C++ template. Runtime selectors choose among these variants. The dispatch chain fixes a representation bucket and algorithm traits before calling a typed pipeline. Representation-width choices depend on the selected k-mer bucket, avoiding a full independent product of both axes. When an outer caller has already selected the representation, the inner pipeline retains that choice. When sketches are supplied, it also removes the unnecessary sketch-algorithm dispatch dimension.

Not every numerical parameter needs its own specialization. Compile profiles control which specialized forms are instantiated. Other values use a compiled generic implementation

that reads the requested k and w at runtime. This specialization fallback preserves the biological parameters. It is different from the explicit short-pair adjustment of untouched defaults, which can choose smaller k and w before execution. An indexed path prepares query sketches with the parameters required by the stored lookup.

5.4.3. Require compatible indexes and execution routes

An index is usable only when its identity matches the selected operation. The relevant information includes the source generation, alphabet, minimizer scheme, producer identity, and k, w values. Matching the latter two numbers alone is insufficient. In particular, the resident and indexed continuation routes within the two-stack family cannot be mixed merely because they share an algorithm name. The indexed query uses the selection route required by its index. Section 4.7 describes the separate policies for allowing or requiring persisted reuse.

Admission also limits acceleration:

- CPU SIMD. Selected kernels check their supported alphabet, representation, parameter range, and vector shape. Bounded ntHash requires DNA4 with $k = 15, w = 10$. SIMD chaining requires a vector target and valid limits for its selected policy. A rejected request is not retried with another chaining policy or a scalar implementation.
- Biological Vulkan stages. PostgreSQL requires the complete supported resident combination, including Vulkan sketching, seed lookup, chaining, fused execution, and permission to build GPU indexes when needed. It requires usable hardware and a batch containing at least two jobs. Unsupported mixtures and device failures produce errors instead of entering the general CPU pipeline.
- GPU output. The persisted grouped-SoA route currently supports coordinates only. Transient resident execution also has a bounded CIGAR route, which retains host packing and result construction. Admission to a GPU route therefore does not mean that every substage runs on the device.

These rules have specific exceptions and should not be reduced to a blanket ban on fallback. The reference SIMD minimizer explicitly uses scalar rescan for alphabets without its two-bit representation. A Vulkan rank prefilter can accompany CPU biological stages, as explained in Section ??. A permissive index policy can rebuild an absent or inapplicable lookup. Finally, a rejected parallel work split can run serially with the same algorithm. Each alternative belongs to a particular layer and preserves that layer's stated contract.

5.5. Schedule independent work within a backend

The ownership boundary in Section 5.3 also determines where parallel execution is possible. Two queries can read the same immutable target, but they must not overwrite the same alignment scratch space. A database-backed reader may expose the right sequence interface while still requiring access from its creating PostgreSQL thread. The scheduler therefore needs more than a collection of independent mathematical problems. It needs inputs that remain readable, separate output locations, and enough memory for the work that overlaps.

BioSeqDB expresses this work through oneAPI Threading Building Blocks (oneTBB). A task is a unit of executable work, such as processing a range of independent queries. A worker thread is an operating-system thread that executes such tasks. An arena is oneTBB's

scheduling domain with a limit on how many threads may participate simultaneously. Tasks describe available parallelism without assigning a permanent thread to each operation. This is the distinction emphasized by oneTBB's task-oriented and data-parallel interface.[3]

5.5.1. PostgreSQL processes and oneTBB threads

PostgreSQL executes a client session in a backend process. Its original server process, traditionally called the postmaster, starts backends and remains responsible for accepting connections. It does not execute each client's alignment tasks.[4] Inside an executing backend, BioSeqDB has one process-local shared scheduling interface. It owns a oneTBB arena used by alignment and other supported work. The name "global pool" refers to this process-local scope.

The extension also registers a separate PostgreSQL background supervisor when it is preloaded. The supervisor maintains shared scheduling configuration and service state. It can perform background work such as servicing index-persistence requests. It does not own one central CPU arena to which every client sends its sequence pairs. Figure 5.3 separates these roles.

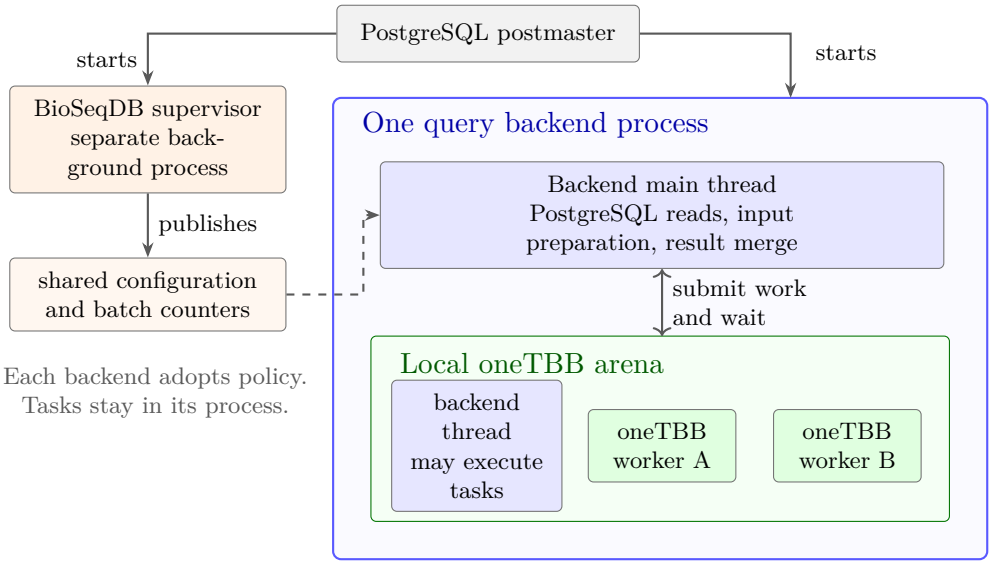


Figure 5.3: The supervisor and the query backend are different PostgreSQL processes. The arena belongs to the query backend. Its worker threads share that process's address space, and the backend main thread may participate in arena computation. Dashed communication carries shared policy, not a cross-process queue of alignment tasks. The three arena participants are illustrative.

Initialization follows the same boundary. During preload, BioSeqDB registers PostgreSQL hooks and background-worker metadata. It delays arena initialization until execution in a backend or the background process. At a scheduling boundary, the backend adopts the current configuration generation and configures its own arena. A policy restart can advance that generation and recreate the caller's arena. Other backends adopt it at a later synchronization point. This is not an immediate stop and restart of every query in the database.

[3] UXL Foundation Contributors, oneTBB Benefits.
[4] PostgreSQL 17, Architectural Fundamentals.

oneTBB's arena limit includes the application thread when it participates. An arena limit of four therefore does not promise four extra workers in addition to the backend main thread. The scheduling interface's range operations enter the arena through `task_arena::execute`, which permits the caller to execute work. The configured concurrency is also distinct from a live census of threads that happen to be busy.[5]

PostgreSQL's parallel-query workers are different resources. A `Gather` plan obtains additional PostgreSQL processes, subject to PostgreSQL's worker-process limits.[6] A oneTBB thread inside a backend does not become one of those processes and does not consume a PostgreSQL parallel-worker slot. Nevertheless, both forms of concurrency use the same machine's processors and memory. BioSeqDB clamps its arena width to one when running inside a PostgreSQL parallel worker unless composition is explicitly enabled. This is a defensive policy for such callers. It does not make the alignment custom scan parallel-safe, nor does it allocate a common CPU quota across all client sessions. Several ordinary backends can still compete for the same cores.

5.5.2. Choose the unit of parallel work

The scheduling interface uses oneTBB range operations, task groups, and explicit task submission. The biological pipeline's stages remain the operations described in the earlier algorithm chapters. They are not assigned to a row of permanent threads, with one thread sketching and another chaining. Instead, the implementation exposes independent work where its data and algorithm dependencies permit it.

At dispatch level, eligible in-memory sequence pairs can run independently. Each item writes to its own result collection. The backend combines those collections after the range has completed. A join waits for submitted work to finish before its results are combined. A second useful shape is several queries against one shared target. The backend first validates the query and target identities and prepares the supported target reader. When the required resident target can be admitted, queries borrow the same immutable target state and retained query sketches. They still own separate alignment scratch and results.

Figure 5.4 shows why the number of tasks need not equal the number of threads. Query costs vary with the number of anchors and the extension work they induce. oneTBB can subdivide ranges and schedule ready tasks as participants become available. Its scheduler favors recently created local tasks and can steal older ready work from another participant's deque. This balances available work while seeking locality, without guaranteeing a particular task order.[7]

Within one query, sufficiently large groups of independent alignment regions can also run in parallel. Large-target recomputation can expose several admitted segments. These choices depend on the selected algorithm and on whether the relevant sequence views can safely cross threads. A PostgreSQL reader does not become shareable because its C++ interface matches a core concept. Such work must retain a supported resident view or use the backend's serial reading path, as described in Section 5.3.

The scheduler provides three modes:

- Single-thread: run dispatch and coarse pipeline regions on the caller.
- Dispatch-parallel: admit eligible pairs from a batch.

[5] UXL Foundation Contributors, oneTBB Specification: task_arena.

[6] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Resource Consumption.

[7] UXL Foundation Contributors, How Task Scheduler Works.

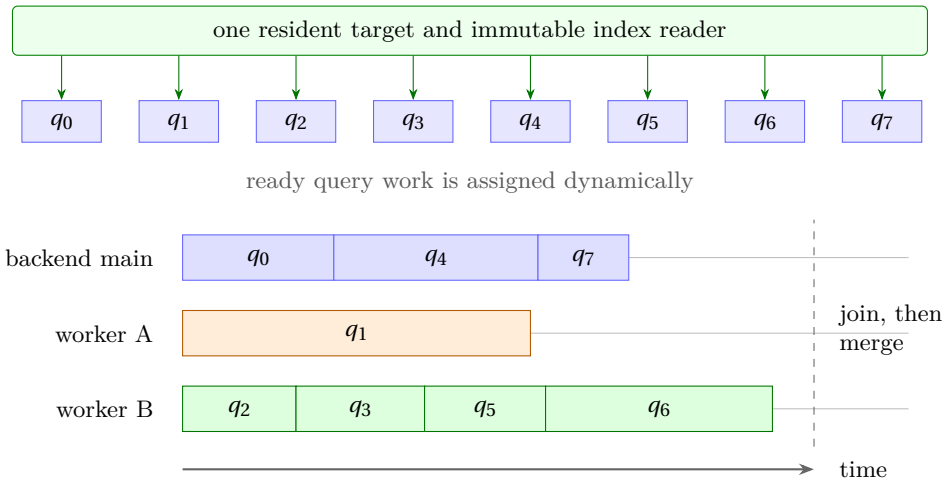


Figure 5.4: Eight independent queries can share three arena participants. Each query has its own result slot, regardless of which thread executes it. Colors distinguish participants. Block lengths and assignments illustrate uneven costs, not measurements or a guaranteed oneTBB schedule. A range task may also contain several queries. Shared target bytes remain owned until all their consumers finish.

- Task-parallel: also allow supported query, region, segment and index-fragment work.

These are runtime scheduling policies, not another dimension of biological algorithm selection. Running a supported region serially does not by itself choose a different minimizer, chainer, or aligner.

Coarse parallel regions share a process-wide guard. Once one such region has acquired it, another coarse region executes serially instead of opening another level of parallel work. For example, a parallel query batch does not also multiply its active workers by independently parallelizing every query's alignment regions. The guard applies to these coarse regions, rather than prohibiting all nested oneTBB operations. Workload thresholds also avoid submitting very small ranges whose scheduling overhead could exceed their useful work.

Each participating thread needs its own installed computation context for temporary pipeline storage. For context-aware pipeline ranges, the pool supplies contexts scoped to the parallel region and reuses an already installed context when possible. The results outlive temporary scratch through their own owners. Consequently, work stealing changes which thread executes ready work, but does not grant a task access to another thread's mutable alignment scratch storage.

5.5.3. Admit memory before increasing overlap

A CPU limit alone cannot bound memory use. Four simultaneous target-index builds can require roughly four times the temporary storage of one build, even if each task uses only one processor. BioSeqDB therefore combines the arena's concurrency limit with a process-local memory-admission controller. Admission means obtaining a reservation, represented by a move-only permit, before starting work whose retained allocation is covered by that permit.

The controller uses the common memory policy's view of the process footprint and available host memory. It also applies an optional index-memory cap and accounts for bytes already promised to live permits. Reservations are updated atomically, so simultaneous tasks in the same process cannot each treat the same remaining reservation capacity as their own.

A rejected request returns without waiting for another task to free memory. The caller can reduce the admitted wave, use a supported serial or streaming path, or report that the selected route cannot proceed.

A wave is the group of work items whose input and temporary storage may overlap. It is smaller than the logical batch when the whole batch would retain too much state. Preparation belongs inside this bound. Waiting until after every sequence has been copied would leave the copies outside the intended memory limit. Conversely, a lazy PostgreSQL source need not be flattened merely because its locator belongs to a batch. The selected reader and compute path determine which bytes must become resident.

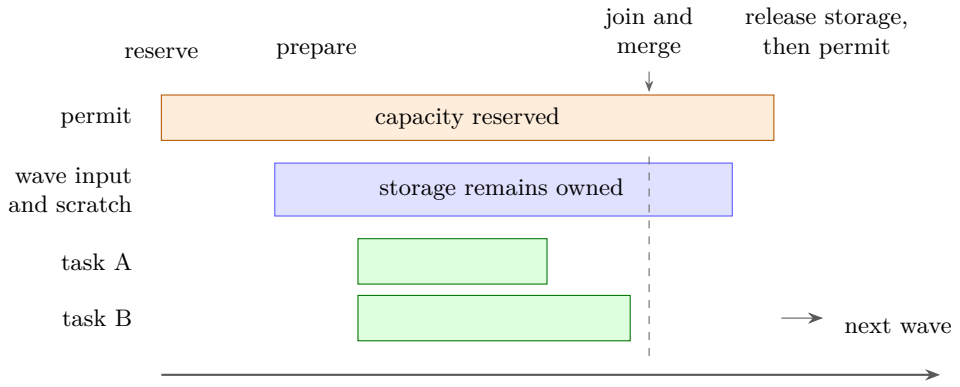


Figure 5.5: An admitted wave retains its input and temporary storage through the join. Completing one task is insufficient to release bytes another task still reads. In the ordinary permit-retaining path shown here, covered storage is released before its reservation becomes available to subsequent work. Output that must survive the wave has a separate owner.

Figure 5.5 distinguishes reservation from allocation. A permit records admitted capacity. It does not own the bytes or prove that the operating system has reserved physical pages. The ordinary release order is therefore to destroy the covered allocation and then return its permit. Paths that reconcile an allocation with the measured process footprint can adjust reservation accounting, but still need an owner for the bytes and their readers.

Algorithm 4 summarizes the CPU scheduling boundary. It deliberately leaves biological implementation selection outside the loop. An unsupported requested implementation is handled by its admission policy, rather than silently replaced because fewer CPU tasks can run.

These decisions bound controlled overlap within one process. They do not provide a cluster-wide memory reservation. Another backend can allocate after a host-memory sample, and unavailable host probes cannot establish how much physical memory remains. Persistent results and other database state also have their own lifetimes. The useful architectural guarantee is the explicit ownership and admission of controlled work, not a claim that any number of sessions or any input can never exhaust memory.

5.5.4. Completion, cancellation, and progress

Submission does not transfer PostgreSQL's API to a worker. Relation access, PostgreSQL allocation, and interrupt checks stay on the backend main thread. The worker runs the permitted computation and returns results or a failure to that thread. For explicitly tracked asynchronous tasks, the owner registers cleanup resources before submission. A bounded error record carries worker failure, and completion is published before joining can release

Algorithm 4 CPU work under backend ownership and bounded admission

```
Require: supported CPU request and a bounded batch
Backend adopts its scheduling configuration
for each next row or query wave do
    Validate source and index identities on the backend
    Obtain admission for required resident input and scratch
    if mode, work size, ownership, and admission allow parallel work then
        Prepare retained, worker-shareable input
        Run independent items in the local arena into separate result slots
        Join the submitted range
    else
        Execute the supported serial or streaming path
    Check PostgreSQL interrupts at backend-controlled boundaries
    Merge completed results into their continuing owners
    Release the wave's input and temporary storage
    Return remaining admission
```

the borrowed storage. The CustomScan lifetime rules in Section 2.4 supply the enclosing statement cleanup. They need not be implemented again by every numeric algorithm.

This boundary limits cancellation responsiveness. The ordinary parallel range completes before the backend resumes its per-row interrupt checks. The native aligner does not continuously call PostgreSQL's cancellation API while computing an alignment. The explicitly tracked task wrapper can observe a cancellation request before invoking its callback, but that is not evidence of interruption at every point inside an already running kernel. A join protects ownership even when the caller no longer needs a result. It does not imply that already executing CPU work stops instantly.

The scheduler must also retain enough runnable work to make progress. oneTBB is intended primarily for computation that does not spend long periods blocked on IO or mutexes.[8] Nonblocking memory admission and the coarse-region guard avoid important forms of nested waiting and resource multiplication. They do not prove general deadlock freedom. BioSeqDB's input path, for example, can overlap parsing with backend writes through a bounded queue, so a claim that every pool task is nonblocking would be incorrect. Its producer and consumer require their own close-and-join protocol. The common arena supplies execution resources, while each operation retains responsibility for a valid dependency graph and bounded ownership.

5.6. Load sequence files through a reusable parser

Public sequence archives distribute FASTA or FASTQ text, often compressed with gzip. BioSeqDB's input layer turns that text into compact sequence chunks and raw headers. A PostgreSQL sink then turns the headers into typed columns and stores the chunks. Separating these jobs lets the same parser serve database loading and ordinary C++ consumers without importing PostgreSQL into the file-reading code.

[8] UXL Foundation Contributors, When Task-Based Programming Is Inappropriate.

5.6.1. Separate byte sources from record framing

A byte source fills a caller-owned buffer and reports bytes read, end of input, or an error. The parser is specialized for the source type, so refill calls need no virtual dispatch. Plain files can use a memory mapping or sequential reads. The SQL loading path uses zlib for gzip input. Biological format and compression are selected independently: the filename suffix chooses FASTA or FASTQ, while gzip magic bytes identify compression. The loader does not infer the biological grammar from an arbitrary first record.

The parser design was inspired by Helicase's vectorized parsing and compact DNA output, and by StringZilla's vectorized byte-search operations.[9] BioSeqDB implements its own kernels through Highway. These libraries are not runtime dependencies of the input path. In FASTA, the header begins with `>`, followed by a sequence body. For DNA4 packed into 64-bit units, the optimized body loop searches for the next `>` while encoding symbols. Boundary detection and packing thus share an input pass. Other alphabet and unit combinations use the same record states with their corresponding encoder and line scanner.

FASTQ adds a `+` separator and quality bytes. Quality lines can contain `@`, `+`, or `>`, so those characters cannot independently identify the next record. The parser instead counts quality bytes against the raw sequence length after removing line endings. This count is distinct from the number of symbols retained by the compact encoder.

5.6.2. Deliver chunks while a record remains open

The parser invokes sink functions, called callbacks, when a record begins, when packed chunks are available, and when the record ends. It keeps its current state and an unfinished storage word across refills. Complete words can be delivered before the record ends. The final chunk supplies the remaining word and its valid symbol count. Figure 5.6 shows why an input-buffer boundary need not be a record boundary.

The SQL sink runs on the PostgreSQL backend thread. Its default path consumes callbacks directly. Optional overlap moves owned events through a FIFO queue of fixed capacity: a worker parses, and the backend performs database writes. A full queue blocks the producer. Closing it wakes blocked participants, and the worker is joined before a consumer error is propagated.

The reusable batch API has a different contract. It owns complete batches and can use direct mapped views, parallel parser shards, or a parallel gzip source. Those options do not describe SQL streaming, which refills a slab and uses the single-threaded zlib source for gzip. In particular, a batch parser's throughput is not a measurement of database loading.

5.6.3. Route header fields into SQL columns

Headers can be preserved verbatim or parsed using a named convention, such as GenBank, RefSeq, or UniProt. Their conventions distinguish identifiers from annotations carried in the same definition line.[10] Automatic selection samples a bounded decompressed prefix once per file. The loader also resolves field-to-column routes once, using destination names and any explicit mappings. Per-record parsing then passes field slices to the PostgreSQL sink, which applies the destination types. A header dialect describes metadata structure. It is not a strict validator for the sequence alphabet.

[9] Martayan et al., "Helicase: Vectorized parsing and bitpacking of genomic sequences"; Vardanian, StringZilla: Fast SIMD, SWAR, and GPGPU String Processing.

[10] National Center for Biotechnology Information, FASTA Format for Nucleotide Sequences; UniProt Consortium, FASTA headers.

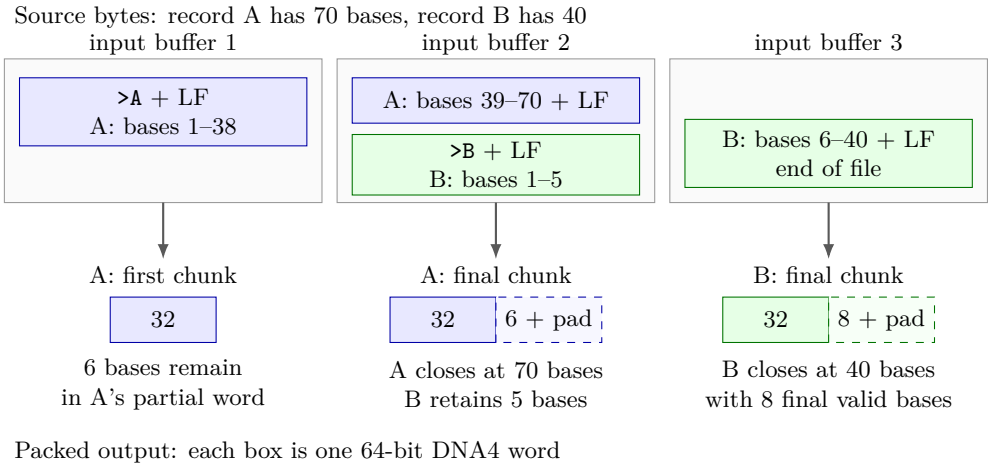


Figure 5.6: Two single-line FASTA records parsed with illustrative 41-byte input buffers. LF denotes a line-feed byte. Header and LF bytes occupy input space but are not sequence symbols. The first refill ends inside A: 32 bases are delivered, while six remain in its partial word. The next buffer finishes A and starts B. Source and packed widths are schematic. Word labels count valid bases, with padding in dashed words. Chunk views are borrowed for their callback and copied by the backend before reuse.

For example, a gzipped FASTA file may contain the header `>dna_new [organism=Homo sapiens] [chromosome=17] [moltype=mRNA]`. The following table gives those fields their relational types:

Listing 5.4: Load a FASTA file with GenBank-style header fields.

```
CREATE TABLE sequences (
  id text,
  organism text,
  chromosome integer,
  moltype text,
  seq bioseq.seq('dna4', 64)
) USING bioseq;

SELECT bioseq.load('sequences'::regclass,
                   '/data/example.fasta.gz');
```

Here automatic routing produces identifier `dna_new`, organism `Homo sapiens`, integer chromosome `17`, and molecule type `mRNA`. The return value is the number of loaded records. Compression changes the byte source, while the SQL interface remains the same.

The sequence column selects the compact alphabet and storage-unit width before parsing starts. Once a record's fragments are ready, the sink writes the parent row and publishes its sequence locator. PostgreSQL controls transaction visibility and rollback. The sink evaluates defaults and `NOT NULL`/`CHECK` constraints. It does not run ordinary `INSERT` triggers or maintain ordinary secondary indexes. Normal trigger-based foreign-key checks and index-based unique or exclusion checks therefore do not run.

The memory guarantee concerns streamed packed bodies. Headers and FASTQ quality storage can grow with record length, and paths that retain unfinished physical lines remain subject to the input-buffer limit. Alphabet handling is permissive, and the declared strict-alphabet option is not enforced by the parser. FASTQ length checks also have an exception for

excess quality bytes on an unterminated final line. These limits preclude treating successful loading as complete input validation.

5.7. Remote sequences and local persistence

An installation built with external-provider support exposes metadata, complete nucleotide records and sequence chunks through relations in the `bioseq_external` schema. These relations obtain their rows from remote services. An application can select a versioned accession and insert its sequence into a local BioSeqDB table before repeated analysis.

Metadata discovery and sequence retrieval use distinct requests. Generated HTTPS clients perform bounded metadata requests, and provider adapters normalize their responses. For NCBI, the catalog reads Datasets assembly reports.[11] A nucleotide accession such as `NC_012920.1` instead requests plain-text FASTA from the EFetch nucleotide endpoint.[12] Assembly accessions beginning with `GCF_` or `GCA_` select a Datasets ZIP package containing sequence files. An assembly accession and a constituent sequence accession therefore identify different retrieval units.

The remote relation returns an unbound sequence literal: sequence characters without a selected storage alphabet or packed layout. In Listing 5.5, assignment to the destination `bioseq.seq('dna5',64)` column validates and encodes those characters for the declared alphabet and storage width.

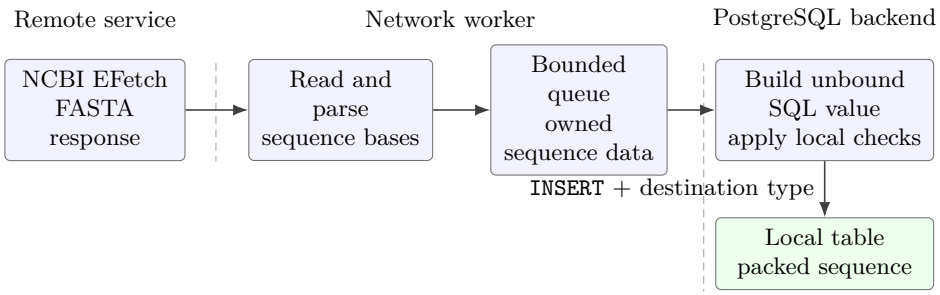


Figure 5.7: An exact NCBI sequence request followed by local insertion. Remote delivery and persistent sequence encoding have separate owners.

Listing 5.5: Importing one versioned NCBI sequence into local storage.

```
CREATE TABLE ncbi_sequences (
    accession text,
    sequence bioseq.seq('dna5', 64),
    retrieved_at timestamptz
) USING bioseq;

INSERT INTO ncbi_sequences
SELECT accession, sequence, retrieved_at
FROM bioseq_external.sequences
WHERE provider = 'ncbi' AND accession = 'NC_012920.1';

SELECT accession, bioseq.len(sequence) AS bases
FROM ncbi_sequences;
```

[11] National Center for Biotechnology Information, NCBI Datasets v2 REST API documentation.
[12] Sayers, "The E-utilities In-Depth: Parameters, Syntax and More."

Ordinary SQL qualifications remain local checks even when equality predicates also constrain the remote request. Array expansion is capped at 64 query combinations, with a warning when combinations are omitted. A provider union may return partial results if one provider fails, and metadata scans do not automatically follow every page. Explicit accessions and a single provider make the import's requested scope clear. Selecting fewer columns from a sequence relation does not currently avoid sequence retrieval.

For the NCBI streaming route, a worker passes owned sequence data through a queue bounded by both bytes and item count. Chunk delivery limits temporary buffering, whereas a complete-record value still requires record-sized memory. Archive retrieval buffers a size-limited ZIP before decoding. The backend checks cancellation between bounded queue waits. Transient FASTA requests can be retried only before the first row is emitted, preventing replay of visible chunks.

Remote responses are not a durable cache, and request rates are not coordinated across backends. E-utilities usage requirements therefore remain relevant to the caller.[13] A transaction controls the local INSERT, but does not make separate HTTP requests one database snapshot. Retaining the accession version and retrieval time records which import supplied the stored sequence. Subsequent queries of the local table need no remote request.

5.8. Generation and validation

The shared computational core needs several interfaces: typed C++ calls, PostgreSQL functions, configuration settings, and benchmark adapters. Keeping these interfaces consistent is part of the architecture. Generation removes some repeated declarations, while assigning compilation work to modules limits its repetition.

5.8.1. Deriving the PostgreSQL boundary

A PostgreSQL routine repeats one contract in several places. Installation SQL declares its argument types, result and volatility. A C entry point must expose the corresponding symbol, decode those arguments and encode that result. BioSeqDB derives these artifacts from annotated C++ declarations. The following existing annotation fixture illustrates the API. Its declaration supplies the name and C++ types, while attributes supply SQL metadata:

Listing 5.6: A typed declaration with SQL metadata.

```
[[bioseq_sql::function({
    .volatility = bioseq::sql::volatility::immutable,
})]]
inline constexpr std::int32_t identity(
    [[bioseq_sql::param({.sql_type = bioseq::sql::pg::integer})]]
    std::int32_t value) {
    return value;
}
```

The generator uses Clang's C++ LibTooling and abstract syntax tree (AST) interfaces through `libclang-cpp`. Compiler actions inspect registered module owners under an explicit compilation profile.[14] AST facts describe declarations and types. Compiler token ranges associate annotations and documentation with those declarations. BioSeqDB then parses the

[13] Sayers, "A General Introduction to the E-utilities."

[14] The Clang Team, LibTooling; The Clang Team, Choosing the Right Interface for Your Application.

annotation options as a restricted language and validates their admitted forms. Unknown options, conflicting annotations, duplicate defaults and unsupported C++/SQL conversions stop generation. Volatility and parallel safety remain author-supplied contracts: the generator does not prove them from an arbitrary function body.

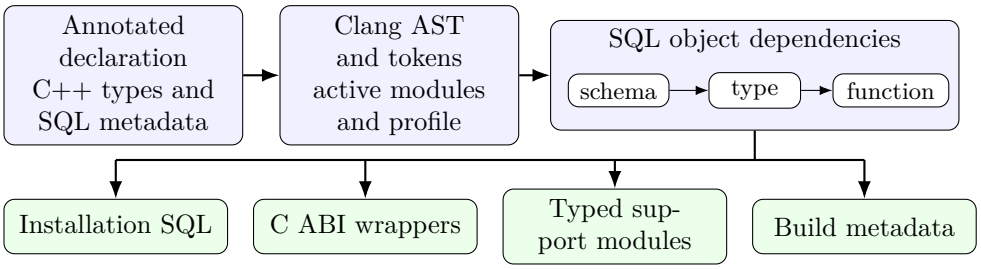


Figure 5.8: One checked interface produces related artifacts. The small dependency chain is illustrative. Typed support includes row codecs, configuration bindings and catalog handles where the declarations require them.

The graph records dependencies among declared SQL objects, each with a stable identity. Tarjan's algorithm identifies strongly connected components, after which a stable topological traversal orders the admitted component graph.[15] Unsupported cycles are rejected. Base types additionally need statement-level ordering between their shell declaration, input/output functions and completed definition.

Generated support code must exist before its consumers compile. The build therefore produces row codecs, configuration bindings and catalog handles before final SQL and wrappers. Final generation checks these intermediate outputs against the active declarations. Separate build profiles keep test-only declarations out of the production interface.

The generated C ABI also centralizes exception translation. C++ exceptions must not escape into PostgreSQL, while a PostgreSQL error can jump past C++ destructors.[16] The boundary preserves captured database diagnostics and leaves C++ exception scopes before raising the SQL error. Calls into PostgreSQL below that boundary still require an explicit bridge or a region without live nontrivial C++ objects.

Language reflection is a related alternative, but it is not the mechanism used here. Reflection was adopted for C++26.[17] Upstream Clang still lists it as unsupported.[18] BioSeqDB therefore uses an external compiler pass, including source locations, preprocessing state and module provenance, together with its own annotation validation.

5.8.2. Controlling specialization cost

Templates specialize kernels for concrete alphabets, storage layouts and algorithm choices. Independent parameters multiply the potential number of instantiations (Figure 5.9). A parameter can create new compiler work even when it does not change a downstream algorithm. Link-time removal of equivalent code cannot recover all the earlier parsing, instantiation and optimization work.

BioSeqDB limits this cost by selecting length buckets after fixing the k-mer size, passing already selected input representations into scheduled work, and assigning expensive special-

[15] Tarjan, "Depth-First Search and Linear Graph Algorithms."
[16] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Using C++ for Extensibility.
[17] Köppe et al., Editors' Report: Programming Languages—C++.
[18] The Clang Team, C++ Support in Clang.

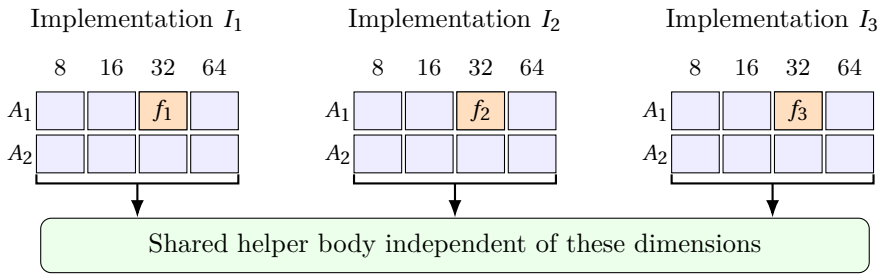


Figure 5.9: Illustrative specialization product: two alphabets, four storage widths in bits, and three implementations produce 24 cells. Each cell denotes one possible function body. Highlighted cells keep alphabet and width fixed while changing implementation. This is not a production support matrix.

izations to concrete module owners. Shared computational modules can be compiled once for compatible consumers. An ordinary benchmark and the extension can consequently reuse the same core without each owning its compilation again. This does not eliminate specialization: it gives the build graph explicit work units and avoids repeating irrelevant choices.

The cost has been substantial during development. Recorded May 2026 runs included a 32-minute debug benchmark build and PostgreSQL translation-unit compilations that exceeded a 50-minute timeout. These are historical observations, not current build timings or measurements of SQL generation alone. Later investigations also found expensive chains of imported modules behind small function bodies. Controlling both template dimensions and module ownership is therefore necessary.

5.8.3. Benchmark specifications and validation

Benchmarking uses a generic framework and adapters for individual workloads. The framework owns configuration validation, matrix expansion, scheduling, measurement and report generation. An adapter defines the experiment: how to prepare its input, invoke an algorithm or SQL statement, evaluate the result and release temporary resources. The same orchestration serves in-process alignment, sequence input and output, and PostgreSQL experiments.

A JSON specification declares named inputs and algorithm parameter schemas. Within each benchmark, `matrix.for` assigns repetition counts to inputs, while `matrix.over` lists algorithms and parameter values. The framework expands the combinations separately for each algorithm and fills omitted parameters from declared defaults. Schema and reference checks reject inconsistent specifications before execution. JSON selects implementations already registered by the adapters.

Listing 5.7: Input and algorithm selection fields from an alignment benchmark specification.

```
"matrix": {
  "for": {"RMQ_REPRO_LONG_CCS": 100},
  "over": {
    "bioseq_ksw2_rmq_scalar_mm": {},
    "minimap2_rmq": {}
  }
}
```

This excerpt selects 100 repetitions of each registered alignment route on the same named input. The empty objects use the declared parameter defaults. Input definitions, metric selection and other run settings are omitted.

The measurement boundary is explicit. The framework times the execution callback, then passes its input and output to the evaluator. This permits reference computations outside that timer. However, adapters may also check results while constructing their output, so elapsed time must be interpreted together with the measured region. Alignment experiments through PostgreSQL check expected matches, coordinates within the configured tolerance and strand. Selected cases additionally validate CIGAR replay. These checks complement authored integration tests of SQL behavior.

Reports retain metric summaries and rule outcomes. Adapters can additionally emit durable raw samples. Rules compare selected configurations within one run, while regression checks use eligible historical captures. Eligibility requirements determine which runs can enter a historical comparison. The configured matrix still determines coverage: a successful timing run does not establish correctness for every query, setting or executor path.

5.9. Authorship, development and AI assistance

I developed BioSeqDB over approximately three years. Early work included Python aligner prototypes written approximately two years ago. I designed the system and implemented the BioSeqDB-specific algorithms, representations and optimizations presented as my contributions in the preceding chapters. Moving from those prototypes to a PostgreSQL extension also required substantial integration, testing and performance work.

These contributions build on published algorithms and existing software. The minimap2 mapping structure, prior SIMD minimizer work and adaptive block alignment are important foundations.[19] The implementation includes ports and adaptations, as well as new work. The basic columnar store comes from Citus and Hydra and retains code written by their authors and maintainers, as described in Section 3.1.[20] PostgreSQL, SeqAn, Highway and oneTBB also provide substantial components. The original contribution lies in the specific algorithms, changes and integration identified in this thesis. It does not imply authorship of the underlying published algorithms or of all source code in the repository.

5.9.1. Where AI tools were used

I used AI coding tools under requirements and designs that I supplied. Their work included:

- Source reorganization. Long files were divided into modules, functions were renamed, and code was adjusted for Clang-based checks. Many present file and function names were produced during this work.
- Development tools. AI helped implement benchmarking utilities and code generators according to my designs, and assisted with implementation and optimization tasks.
- Technical writing. AI generated working documentation.

[19] Li, "Minimap2: pairwise alignment for nucleotide sequences"; Groot Koerkamp and Martayan, "SimdMinimizers: Computing Random Minimizers, fast"; Liu and Steinegger, "Block Aligner: an adaptive SIMD-accelerated aligner for sequences and position-specific scoring matrices."

[20] Citus Data contributors, Citus Columnar storage implementation; Hydra contributors, Hydra Columnar: PostgreSQL columnar storage extension.

The large reorganizations were intended to preserve the algorithms while changing names and file boundaries. They nevertheless introduced bugs. In my assessment, diagnosing and correcting regressions from AI-assisted work consumed approximately two months. Numerical code was particularly sensitive: a seemingly equivalent change to a mask, integer width or coordinate calculation could change results. A refactoring that compiles is therefore insufficient to establish numerical correctness.

AI assistance accelerated parts of development despite these costs. The research questions, project-specific designs and decisions about which ideas to pursue were mine. I directed and corrected the assisted work and retain responsibility for the final implementation and the claims made here.

Automatically generated documentation is not an authoritative specification. Some descriptions were incorrect, and others became stale after later changes. For this chapter, generated descriptions were checked against the implementation, the cited technical sources and the stated tests. Generated names and reports alone do not establish how a feature works.

5.10. Validation limits of the complete system

Validation of query-result caching, described in Section 2.6, remains incomplete. It has targeted tests, including comparisons with caching disabled, but these do not cover all combinations of plans, settings and transaction histories. Incorrect reuse and therefore incorrect query results remain a risk. The cache should be treated as experimental.

Chapter 6

Conclusions

BioSeqDB demonstrates that sequence search can be represented as a PostgreSQL execution operation rather than as a scalar call to an external mapper. The system stores packed biological sequences, constructs persistent minimizer indexes, and runs a minimap2-derived pipeline for seeding, chaining, filtering, and final extension. Planner support and custom scans preserve the batch structure required by SIMD and GPU implementations while presenting typed SQL results that can participate in filters, projections, and joins.

The first research question concerned a shared representation for the algorithmic stages. Alphabet-generic compact ranges cover resident values, while lazy sequence views represent fragmented, sliced, and composed sources. Algorithms read the required region without first converting the complete value to a byte array. The second question concerned heterogeneous execution. Minimizer extraction and chaining provide regular work for the implemented SIMD and GPU routes. Dispatch follows explicit algorithm settings, input compatibility, index form, and resource availability. Final alignment uses banded or unbanded KSW2, adaptive Block Aligner, and GPU routes that may return specialized scripts before device dynamic programming. Optional prefilters may reject unsuitable candidates but do not replace alignment.

The third question concerned database execution. A custom scan owns the search expression, batches input rows, and substitutes match fields into planned expressions. This design also permits multiple alignments for one input row. The fourth question concerned the surrounding data system. Physical sequence storage, persistent indexes, bounded ingestion, external-provider relation definitions, cache coordination, and source-driven SQL generation give the kernels explicit storage and lifecycle contracts.

The remaining limits are operational and algorithmic. Some remote retrieval paths still require record-sized or archive-sized memory. Provider quotas are not coordinated across all PostgreSQL backends. Hardware-specific routes have batch-size thresholds and require broader evaluation on additional devices. GPU chaining may examine a bounded predecessor set, and seed and alignment backends retain route-specific contracts. The present mapper is centered on linear references rather than pangenome graphs. A relational database can nevertheless plan and execute batched sequence analysis while retaining typed data, bounded resources, persistent derived state, and explicit algorithm selection.

References

Aasna, Evelin et al. “DREAM-Stellar: parallel and space efficient exact local alignment.” In: BMC Bioinformatics 27.1, 67 (Feb. 18, 2026). doi: 10.1186/s12859-026-06389-0. url: https://link.springer.com/article/10.1186/s12859-026-06389-0.

Abadi, Daniel, Samuel Madden, and Miguel Ferreira. “Integrating compression and execution in column-oriented database systems.” In: Proceedings of the 2006 ACM SIGMOD International Conference on Management of Data. SIGMOD ’06. Association for Computing Machinery, 2006, pp. 671–682. doi: 10.1145/1142473.1142548. url: https://doi.org/10.1145/1142473.1142548.

Chernishev, George A. et al. “A Comprehensive Study of Late Materialization Strategies for a Disk-Based Column-Store.” In: Proceedings of the 24th International Workshop on Design, Optimization, Languages and Analytical Processing of Big Data. Vol. 3130. CEUR Workshop Proceedings. 2022, pp. 21–30. url: https://ceur-ws.org/Vol-3130/paper3.pdf.

Citus Data contributors. Citus Columnar storage implementation. Version 14.0.0. AGPL-3.0. Feb. 17, 2026. url: https://github.com/citusdata/citus (visited on 07/28/2026).

ClickHouse contributors. ClickBench: analytical database benchmark. Historical PostgreSQL, Citus, and Hydra submissions. Sept. 20, 2023. url: https://github.com/ClickHouse/ClickBench/tree/2e8146c397f81e3e98937205d09e331313b991e3 (visited on 09/09/2026).

cppreference.com contributors. C++ named requirements: StandardLayoutType. Undated online source. url: https://en.cppreference.com/cpp/named_req/StandardLayoutType (visited on 09/15/2026).

Cubukcu, Umur et al. “Citus: Distributed PostgreSQL for Data-Intensive Applications.” In: Proceedings of the 2021 International Conference on Management of Data. SIGMOD ’21. Association for Computing Machinery, 2021, pp. 2490–2502. doi: 10.1145/3448016.3457551. url: https://doi.org/10.1145/3448016.3457551.

Davis, Jeff. Citus 10 brings columnar compression to Postgres. Mar. 6, 2021. url: https://www.citusdata.com/blog/2021/03/06/citus-10-columnar-compression-for-postgres/ (visited on 09/08/2026).

Flach, Quezia et al. “Analysis of Biological Sequence Search Performance in NoSQL Database.” In: International Journal of Computer Applications 176.42 (July 2020), pp. 1–6. doi: 10.5120/ijca2020920416. url: https://doi.org/10.5120/ijca2020920416.

Groot Koerkamp, Ragnar and Igor Martayan. packed-seq: Bitpacked sequence trait and implementation. Version 5.0.0. MIT license. Rust crate developed for SimdMinimizers (SEA 2025). url: https://github.com/rust-seq/packed-seq (visited on 09/15/2026).

Groot Koerkamp, Ragnar and Igor Martayan. packed_seq crate documentation. Version 5.0.0. July 7, 2026. url: https://docs.rs/packed-seq/latest/packed_seq/ (visited on 09/15/2026).
— "SimdMinimizers: Computing Random Minimizers, fast." In: 23rd International Symposium on Experimental Algorithms. Vol. 338. Leibniz International Proceedings in Informatics. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, 20:1–20:19. doi: 10.4230/LIPIcs.SEA.2025.20. url: https://doi.org/10.4230/LIPIcs.SEA.2025.20.
Hydra contributors. Hydra Columnar benchmarks. url: https://github.com/hydradatabase/columnar/blob/d1f7657ee82265b2b9f1c06fda929bb5b942411b/BENCHMARKS.md (visited on 09/09/2026).
— Hydra Columnar: PostgreSQL columnar storage extension. Source snapshot d1f7657ee822. Feb. 10, 2025. url: https://github.com/hydradatabase/columnar (visited on 09/09/2026).
Kallenborn, Felix et al. "GPU-accelerated homology search with MMseqs2." In: Nature Methods 22.10 (Sept. 18, 2025), pp. 2024–2027. doi: 10.1038/s41592-025-02819-8. url: https://doi.org/10.1038/s41592-025-02819-8.
Editors' Report: Programming Languages—C++. WG21 paper N5015. ISO/IEC JTC 1/SC 22/WG 21, Aug. 14, 2025. url: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5015.html (visited on 09/09/2026).
Lanius, Christian. parse_mmi.py: Python parser and lookup functions for minimap2 indexes. Sept. 27, 2021. url: https://gist.github.com/christian-lanius/8b8d6a38e35b93783ff7d6236211ff5e (visited on 09/08/2026).
Li, Heng. minimap2. Version 2.30. url: https://github.com/lh3/minimap2/tree/79c9cc186b95f50bd899f69b48eba995ced810c6 (visited on 07/28/2026).
— "Minimap2: pairwise alignment for nucleotide sequences." In: Bioinformatics 34.18 (2018), pp. 3094–3100. doi: 10.1093/bioinformatics/bty191. arXiv: 1708.01492 [q-bio.GN]. url: https://doi.org/10.1093/bioinformatics/bty191.
Liu, Daniel and Martin Steinegger. "Block Aligner: an adaptive SIMD-accelerated aligner for sequences and position-specific scoring matrices." In: Bioinformatics 39.8, btad487 (2023). doi: 10.1093/bioinformatics/btad487. url: https://doi.org/10.1093/bioinformatics/btad487.
Martayan, Igor et al. "Helicase: Vectorized parsing and bitpacking of genomic sequences." In: bioRxiv (Mar. 2026). Preprint. doi: 10.64898/2026.03.19.712912. url: https://github.com/imartayan/helicase (visited on 09/09/2026).
minimap2 contributors. How to interpret the mmi index file? Issue 820. Discussion updated in 2025. Sept. 26, 2021. url: https://github.com/lh3/minimap2/issues/820 (visited on 09/08/2026).
Mirdita, M. et al. "Fast and sensitive taxonomic assignment to metagenomic contigs." In: Bioinformatics 37.18 (2021), pp. 3029–3031. doi: 10.1093/bioinformatics/btab184. url: https://doi.org/10.1093/bioinformatics/btab184.
Mirdita, Milot, Martin Steinegger, and Johannes Söding. "MMseqs2 desktop and local web server app for fast, interactive sequence searches." In: Bioinformatics 35.16 (2019), pp. 2856–2858. doi: 10.1093/bioinformatics/bty1057. url: https://doi.org/10.1093/bioinformatics/bty1057.

Mondelli, Maria Luiza et al. “BioWorkbench: a high-performance framework for managing and analyzing bioinformatics experiments.” In: PeerJ 6, e5551 (Aug. 2018). doi: 10.7717/peerj.5551. url: https://doi.org/10.7717/peerj.5551.

National Center for Biotechnology Information. FASTA Format for Nucleotide Sequences. June 18, 2025. url: https://www.ncbi.nlm.nih.gov/genbank/fastaformat/ (visited on 09/09/2026).

— NCBI Datasets v2 REST API documentation. Undated online source. url: https://www.ncbi.nlm.nih.gov/datasets/docs/v2/api/ (visited on 07/28/2026).

PostgreSQL Global Development Group. PostgreSQL 17 Documentation: Database Page Layout. Version 17. url: https://www.postgresql.org/docs/17/storage-page-layout.html (visited on 09/08/2026).

— PostgreSQL 17 Documentation: Explicit Locking. Version 17. url: https://www.postgresql.org/docs/17/explicit-locking.html (visited on 09/08/2026).

— PostgreSQL 17 Documentation: Function Optimization Information. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/xfunc-optimization.html (visited on 07/28/2026).

— PostgreSQL 17 Documentation: Generic WAL Records. Version 17. url: https://www.postgresql.org/docs/17/generic-wal.html (visited on 09/08/2026).

— PostgreSQL 17 Documentation: Introduction to Concurrency Control. Version 17. url: https://www.postgresql.org/docs/17/mvcc-intro.html (visited on 09/08/2026).

— PostgreSQL 17 Documentation: Resource Consumption. Version 17. url: https://www.postgresql.org/docs/17/runtime-config-resource.html#GUC-MAX-PARALLEL-WORKERS-PER-GATHER (visited on 09/09/2026).

— PostgreSQL 17 Documentation: SET. Version 17. url: https://www.postgresql.org/docs/17/sql-set.html (visited on 09/09/2026).

— PostgreSQL 17 Documentation: Set Returning Functions. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/functions-srf.html (visited on 07/28/2026).

— PostgreSQL 17 Documentation: Table Access Method Interface Definition. Version 17. url: https://www.postgresql.org/docs/17/tableam.html (visited on 09/08/2026).

— PostgreSQL 17 Documentation: TOAST. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/storage-toast.html (visited on 07/28/2026).

— PostgreSQL 17 Documentation: Using C++ for Extensibility. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/xfunc-c.html (visited on 07/28/2026).

— PostgreSQL 17 Documentation: Writing a Custom Scan Provider. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/custom-scan.html (visited on 07/28/2026).

Python Software Foundation. Python 3 Documentation: Common Sequence Operations. Version 3.14.7. Slice clamping rules are given in note 4 of the operation table. Sept. 15, 2026. url: https://docs.python.org/3/builtins/stdtypes.html#common-sequence-operations (visited on 09/15/2026).

Reinert, Knut et al. “The SeqAn C++ template library for efficient sequence analysis: A resource for programmers.” In: Journal of Biotechnology 261 (Nov. 10, 2017), pp. 157–168. doi: 10.1016/j.jbiotec.2017.07.017. url: https://www.sciencedirect.com/science/article/pii/S0168165617315420.

SAM/BAM Format Specification Working Group. Sequence Alignment/Map Format Specification. Version 1.6. Undated online source. url: https://samtools.github.io/hts-specs/SAMv1.pdf (visited on 07/28/2026).
Sayers, Eric. "A General Introduction to the E-utilities." In: Entrez Programming Utilities Help. Updated 2022-11-17. Bethesda, Maryland: National Center for Biotechnology Information, May 26, 2009. url: https://www.ncbi.nlm.nih.gov/books/NBK25497/ (visited on 09/09/2026).
— "The E-utilities In-Depth: Parameters, Syntax and More." In: Entrez Programming Utilities Help. Updated 2026-03-04. Bethesda, Maryland: National Center for Biotechnology Information, May 29, 2009. url: https://www.ncbi.nlm.nih.gov/books/NBK25499/ (visited on 09/09/2026).
Seabolt, Edward E. et al. IBM Functional Genomics Platform, A Cloud-Based Platform for Studying Microbial Life at Scale. Version 3. Preprint. Mar. 30, 2020. arXiv: 1911.02095 [q-bio.QM]. url: https://arxiv.org/abs/1911.02095v3 (visited on 09/08/2026).
SeqAn Project. Alphabets in SeqAn. Version 3.4.0. url: https://docs.seqan.de/seqan3/3.4.0/tutorial_alphabets.html (visited on 09/09/2026).
— SeqAn Manual. url: https://seqan.readthedocs.io/en/main/ (visited on 09/09/2026).
— SeqAn Tutorial: Sequences In-Depth. Version 1.4.2. String specializations, including packed and external strings. url: https://seqan.readthedocs.io/en/seqan-v1.4.2/Tutorial/SequencesInDepth.html (visited on 09/15/2026).
Shen, Wei, Botond Sipos, and Liuyang Zhao. SeqKit usage documentation: the subseq subcommand. Undated online source. Documentation for the tool described in SeqKit2, iMeta 3(3):e191, 2024. url: https://bioinf.shenwei.me/seqkit/usage/#subseq (visited on 09/15/2026).
Siragusa, Enrico, David Weese, and Knut Reinert. Fast and sensitive read mapping with approximate seeds and multiple backtracking. Version 1. Preprint. Aug. 21, 2012. doi: 10.48550/arXiv.1208.4238. arXiv: 1208.4238 [cs.DS]. url: https://arxiv.org/abs/1208.4238v1 (visited on 09/09/2026).
Steinegger, Martin and Johannes Söding. "Clustering huge protein sequence sets in linear time." In: Nature Communications 9, 2542 (June 29, 2018). doi: 10.1038/s41467-018-04964-5. url: https://doi.org/10.1038/s41467-018-04964-5.
— "MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets." In: Nature Biotechnology 35.11 (Oct. 16, 2017), pp. 1026–1028. doi: 10.1038/nbt.3988. url: https://doi.org/10.1038/nbt.3988.
— "MMseqs2: sensitive protein sequence searching for the analysis of massive data sets." Version 5. In: bioRxiv, 079681 (June 7, 2017). Preprint. doi: 10.1101/079681. url: https://www.biorxiv.org/content/early/2017/06/07/079681.
Tarjan, Robert Endre. "Depth-First Search and Linear Graph Algorithms." In: SIAM Journal on Computing 1.2 (1972), pp. 146–160. doi: 10.1137/0201010. url: https://doi.org/10.1137/0201010.
The Clang Team. C++ Support in Clang. url: https://clang.llvm.org/cxx_status.html (visited on 09/09/2026).
— Choosing the Right Interface for Your Application. url: https://clang.llvm.org/docs/Tooling.html (visited on 09/09/2026).
— LibTooling. url: https://clang.llvm.org/docs/LibTooling.html (visited on 09/09/2026).

Uhl, Tim Niklas, Matthias Schimek, and Daniel Brommer. Concepts in Practice: C++ MPI Bindings for the HPC Ecosystem. From a Standardizable Core to a Composable Interface. 2026. arXiv: 2606.09102 [cs.DC]. url: https://arxiv.org/abs/2606.09102.

UniProt Consortium. FASTA headers. June 10, 2026. url: https://www.uniprot.org/help/fasta-headers (visited on 09/09/2026).

UXL Foundation Contributors. How Task Scheduler Works. url: https://uxlfoundation.github.io/oneTBB/main/tbb_userguide/How_Task_Scheduler_Works.html (visited on 09/09/2026).

— oneTBB Benefits. url: https://uxlfoundation.github.io/oneTBB/main/intro/Benefits.html (visited on 09/09/2026).

— oneTBB Specification: task_arena. url: https://uxlfoundation.github.io/oneTBB/main/specification/source/task_scheduler/task_arena/task_arena_cls.html (visited on 09/09/2026).

— When Task-Based Programming Is Inappropriate. url: https://uxlfoundation.github.io/oneTBB/main/tbb_userguide/When_Task-Based_Programming_Is_Inappropriate.html (visited on 09/09/2026).

Vardanian, Ash. StringZilla: Fast SIMD, SWAR, and GPGPU String Processing. url: https://github.com/ashvardanian/StringZilla (visited on 09/09/2026).